

Git, GitHub, GitLab 이란?

Git 이란?

2005년에 리누스 토르발스에 의해 개발된 ‘분산 버전관리 시스템’.

GitHub 이란?

깃(Git)을 사용하는 프로젝트를 지원하는 웹 호스팅 서비스.

협업 및 코드 공유를 위한 웹 기반 인터페이스를 제공.

GitLab 이란?

깃(Git)을 사용하는 프로젝트를 지원하는 웹 호스팅 서비스.

GitHub과 유사한 기능을 제공하면서, DevOps 라이프사이클 지원에 더 중점을 둠.

GitHub or Git

어디에서나 혼자서도 플레이 가능한 휴대폰 게임
온라인을 기반으로 유저들과 대결하는 휴대폰 게임

Git, GitHub, GitLab

Git은 코드의 버전을 관리하는 도구.

GitHub, GitLab은 이러한 Git저장소를 호스팅하고, 추가적인 협업 및 프로젝트 관리 기능을 제공하는 서비스.

GitHub은 사용자 친화적인 인터페이스와 광범위한 커뮤니티.

GitLab은 포괄적인 DevOps 도구와 더 많은 자체 호스팅 옵션을 강조.

Git 을 사용하는 이유

버전 관리

Git의 가장 큰 장점으로 파일의 버전을 관리할 수 있음.

협업 용이

여러 사람이 동시에 같은 프로젝트에 작업할 때, Git은 코드 충돌을 관리하고 통합하는데 매우 유용함.

분산 환경 지원

중앙 서버에 의존하지 않고 각 사용자의 로컬에서 전체 저장소의 복사본이 존재하여 오프라인 상태에서도 작업이 가능함.

데이터 무결성

Git은 내부적으로 SHA-1 해시를 사용하여 저장소의 무결성을 유지.

백업과 복원 용이

모든 버전이 저장되므로, 데이터 손실의 위험이 줄어듬.

브랜치와 머지

Git은 브랜칭과 머징을 쉽고 빠르게 할 수 있도록 지원.

동시에 다양한 기능을 개발하면서도, 코드 베이스를 깔끔하게 유지할 수 있음.

커뮤니티와 통합

많은 오픈 소스 프로젝트가 Git을 사용하며, GitHub, GitLab 같은 플랫폼들은 Git 저장소를 호스팅하며 협업과 코드 공유를 쉽게 할 수 있음.

GitBash 이란?

Linux 스타일의 셸 경험

Windows 사용자들에게 Linux 또는 Unix 셸에서와 같은 경험을 제공합니다. 이를 통해 Windows에서도 Linux 스타일의 명령줄 인터페이스 사용이 가능함.

Git 명령어 접근

Windows의 기본 명령 프롬프트나 PowerShell에서는 사용할 수 없는 Git 명령어들을 사용할 수 있음.

SSH 지원

Git Bash는 SSH 연결을 지원합니다. 이는 원격 Git 저장소에 안전하게 접속할 수 있게 해주며, 특히 Github, GitLab과 같은 서비스와의 통합에 필수적입니다.

Github 회원가입

Github 회원가입

<https://github.com/>

Git 설치

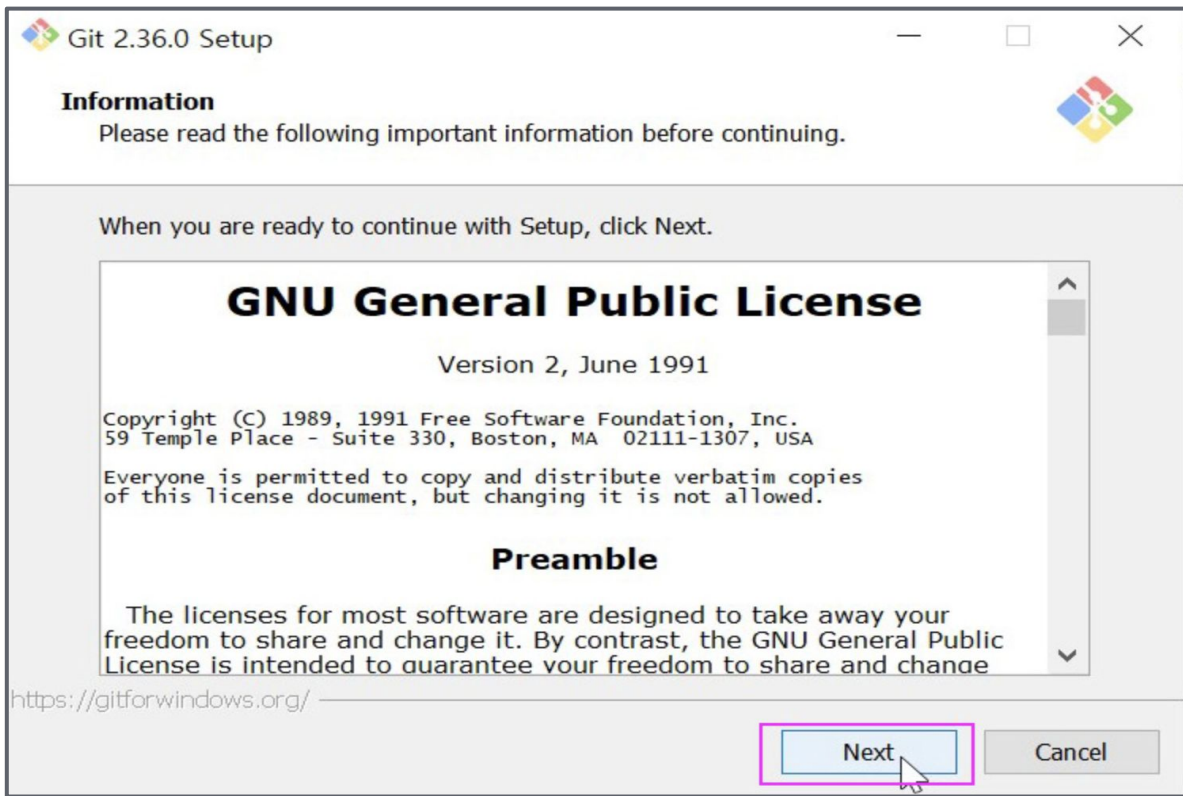
Git Download Link

<https://git-scm.com/>

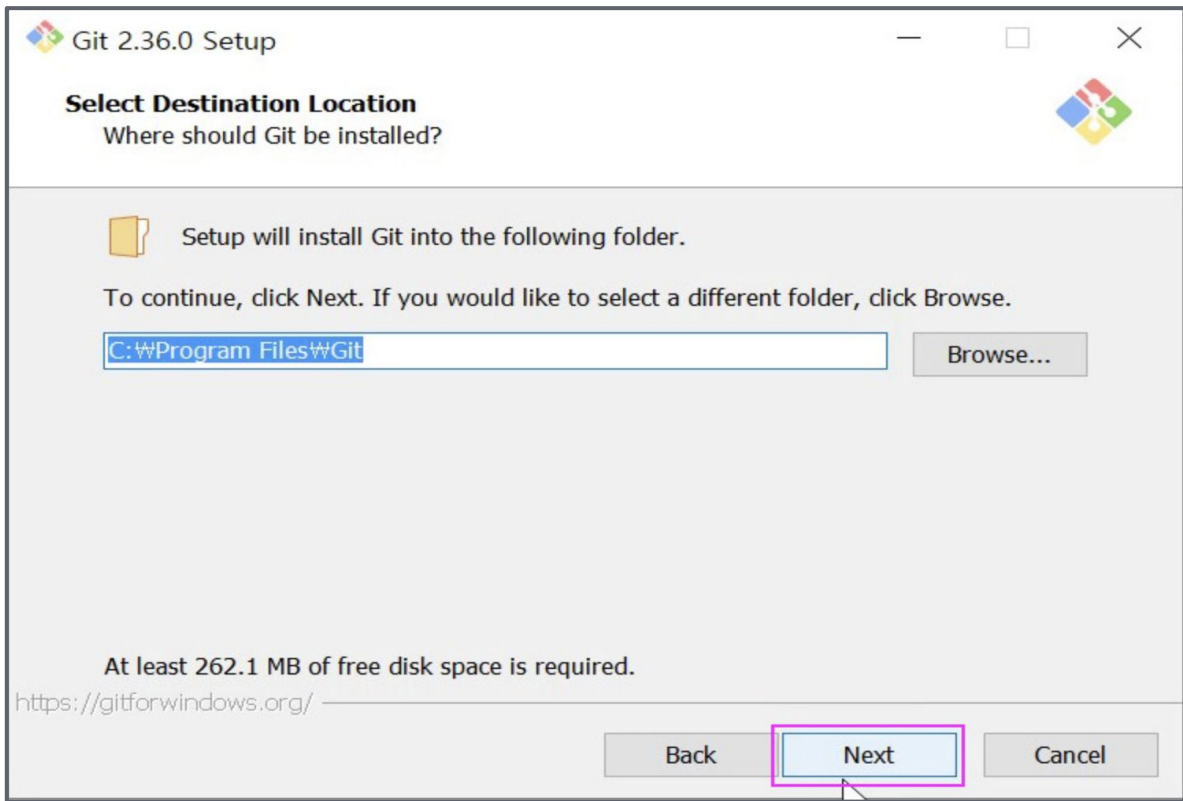
Windows bit 확인

<https://support.microsoft.com/ko-kr/windows/32%EB%B9%84%ED%8A%B8-%EB%B0%8F-64%EB%B9%84%ED%8A%B8-windows-%EC%A7%88%EB%AC%B8%EA%B3%BC-%EB%8C%80%EB%8B%B5-c6ca9541-8dce-4d48-0415-94a3faa2e13d>

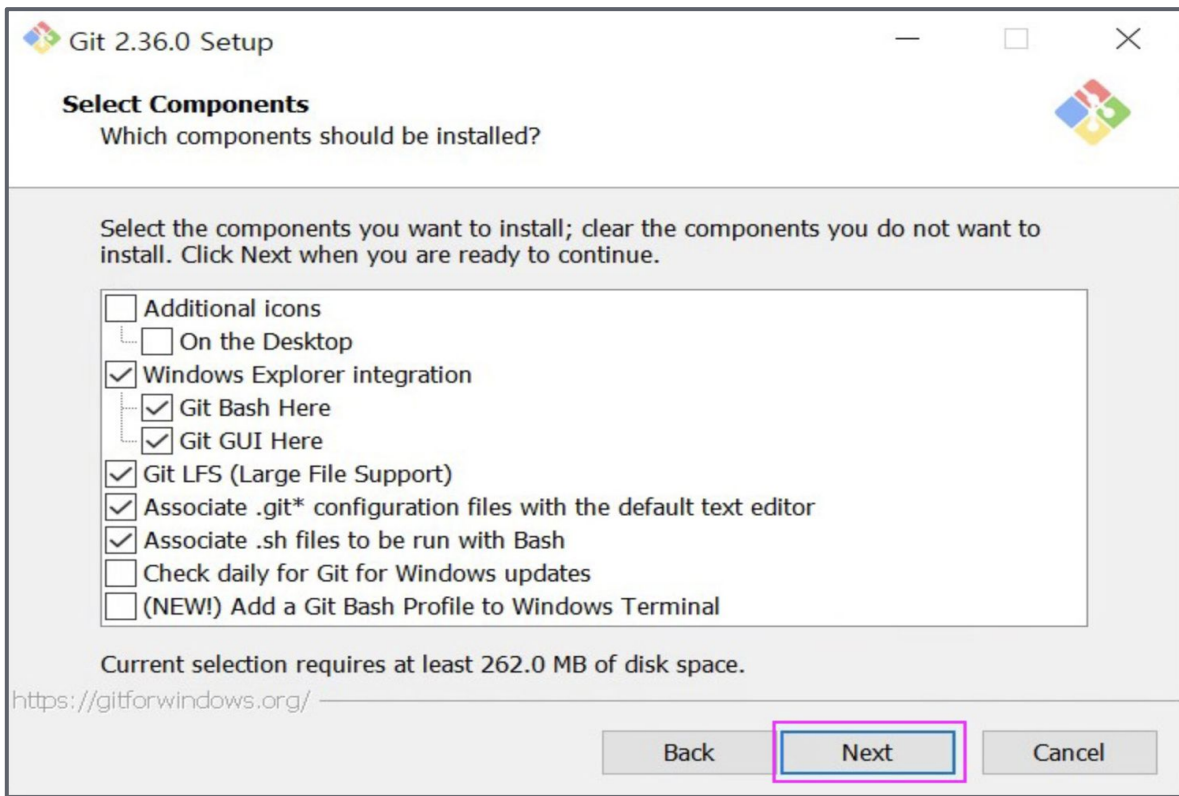
Git 설치



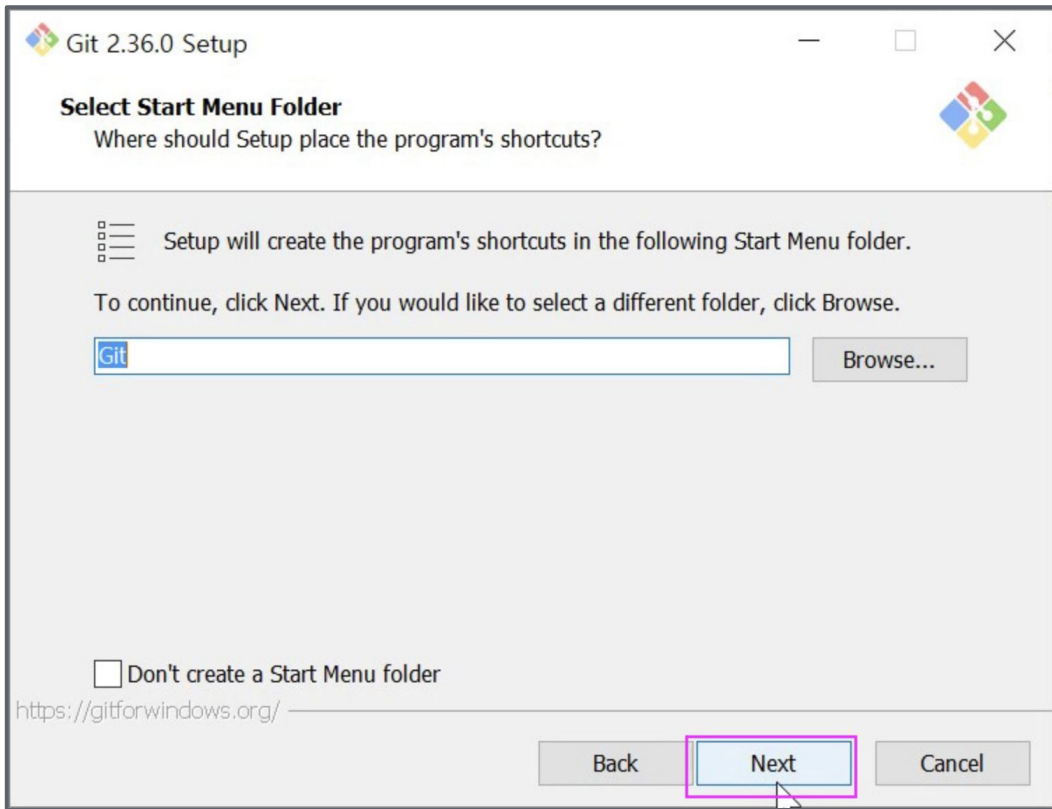
Git 설치



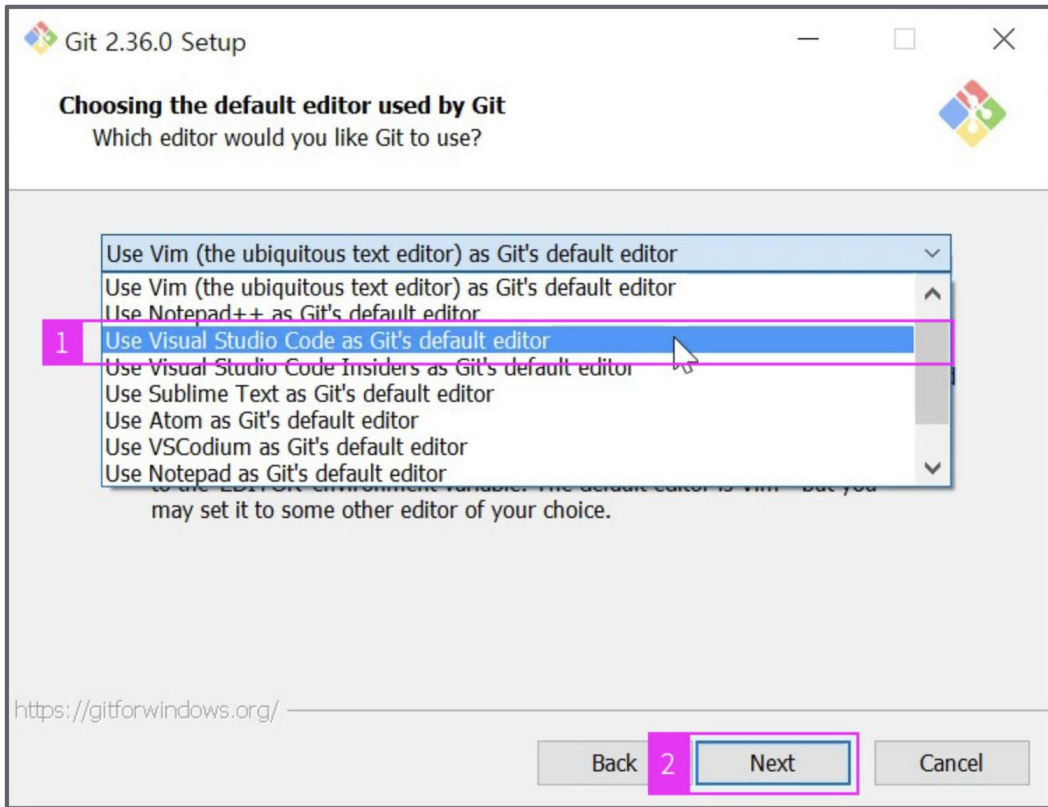
Git 설치



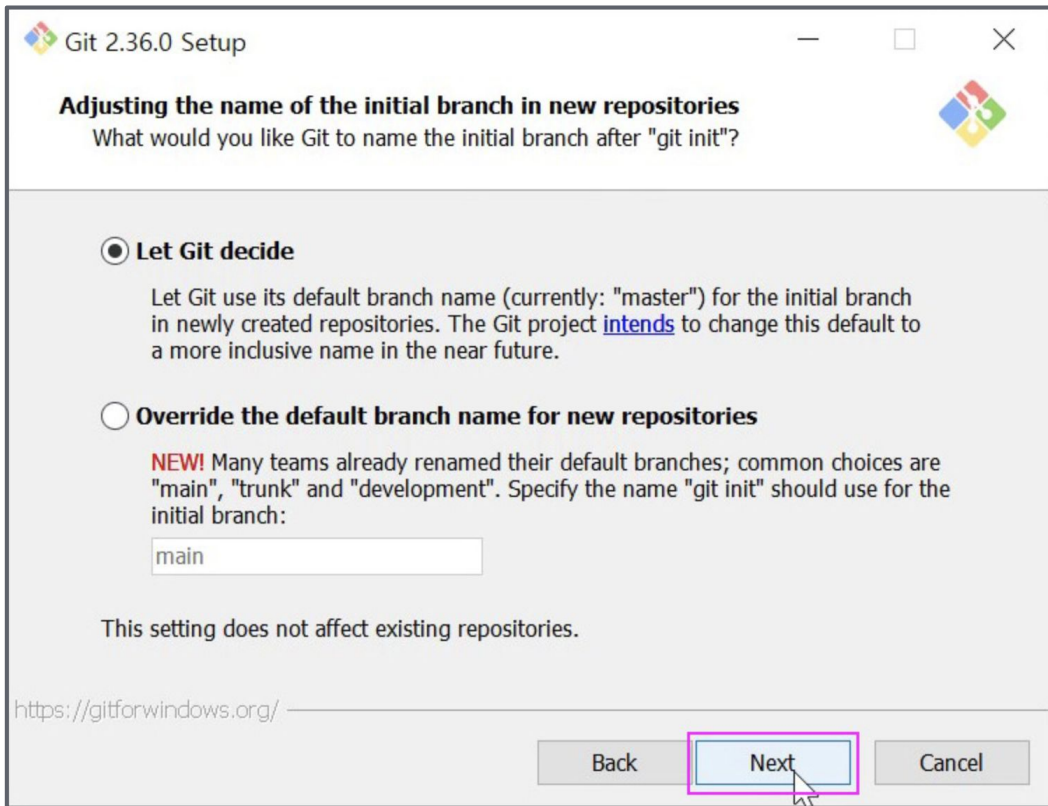
Git 설치



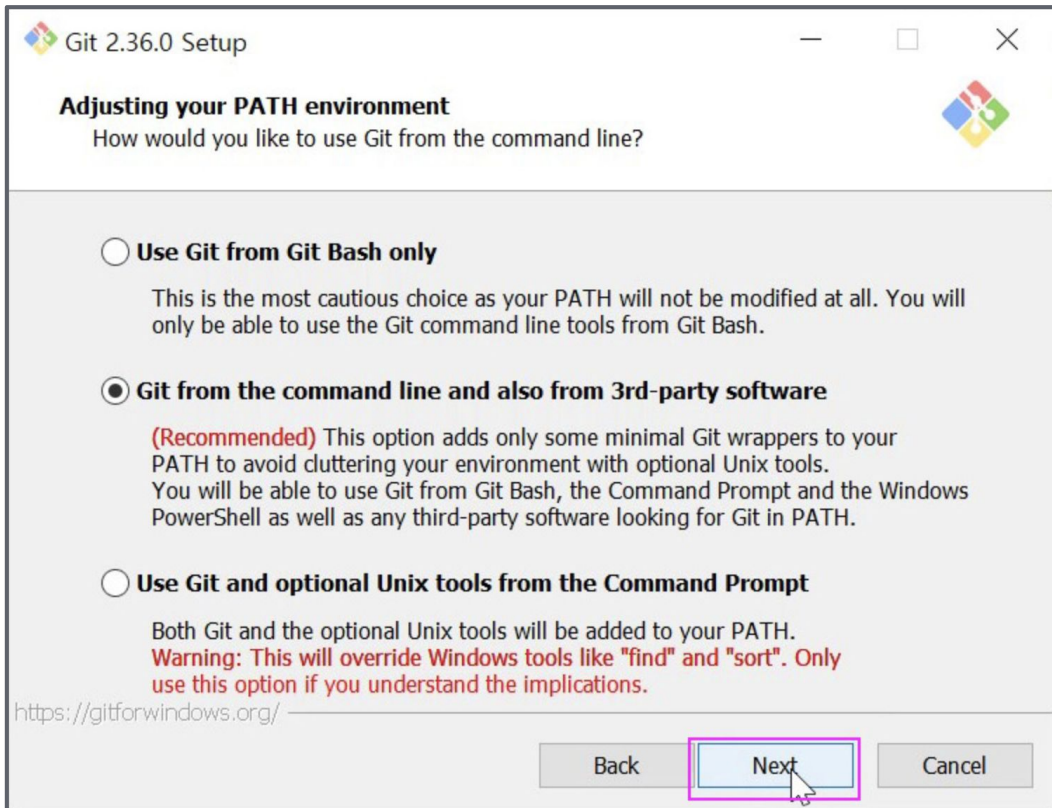
Git 설치



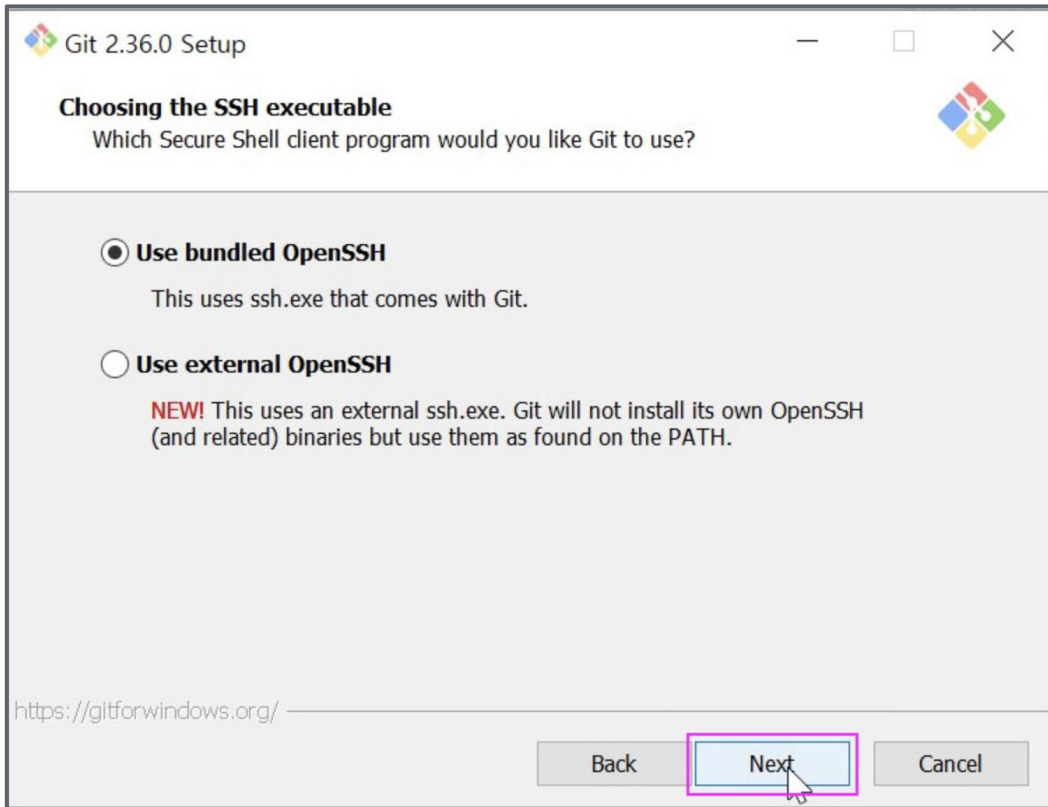
Git 설치



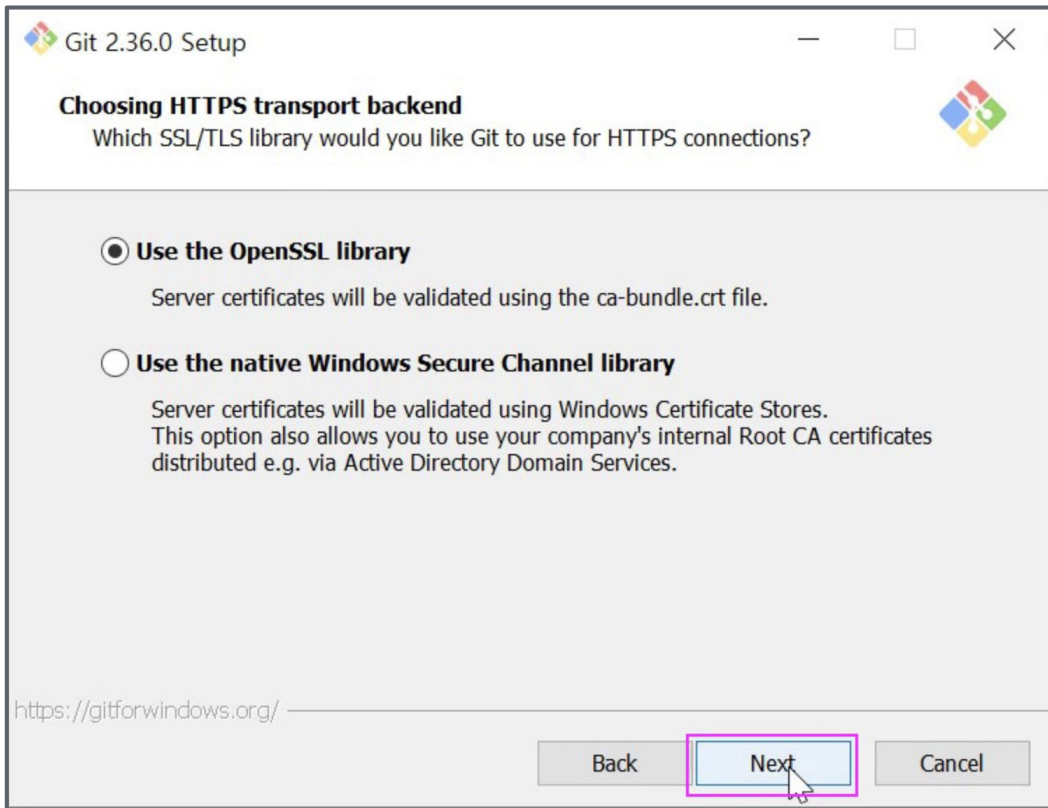
Git 설치



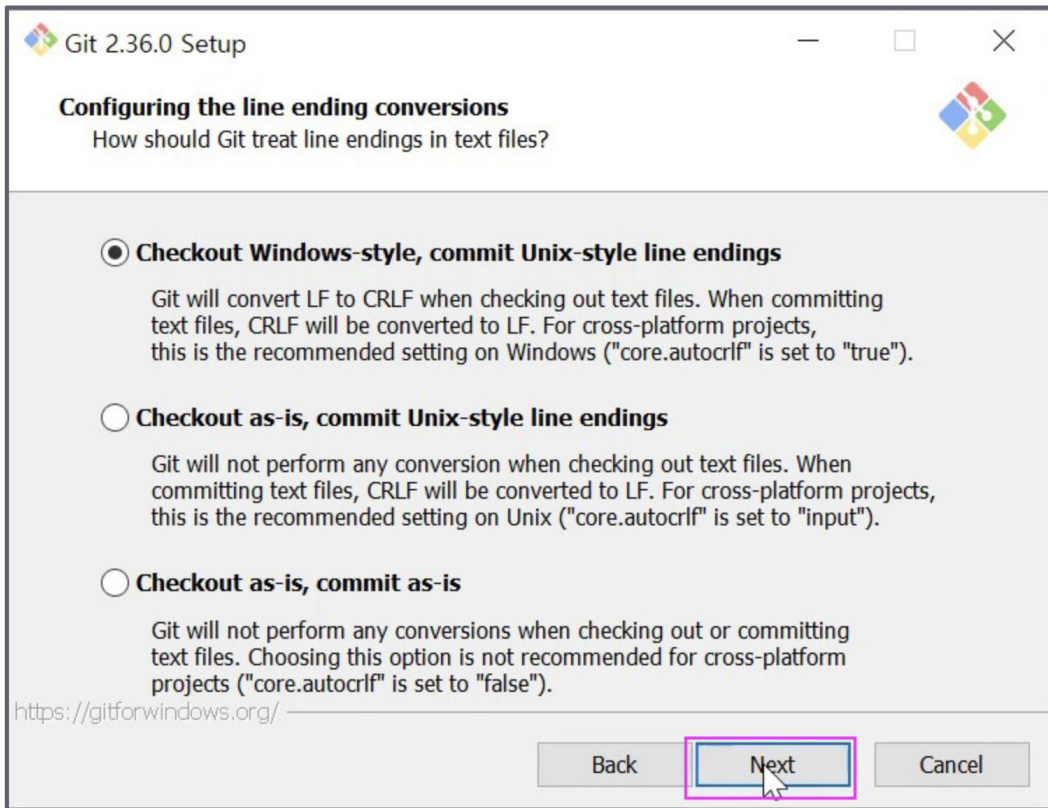
Git 설치



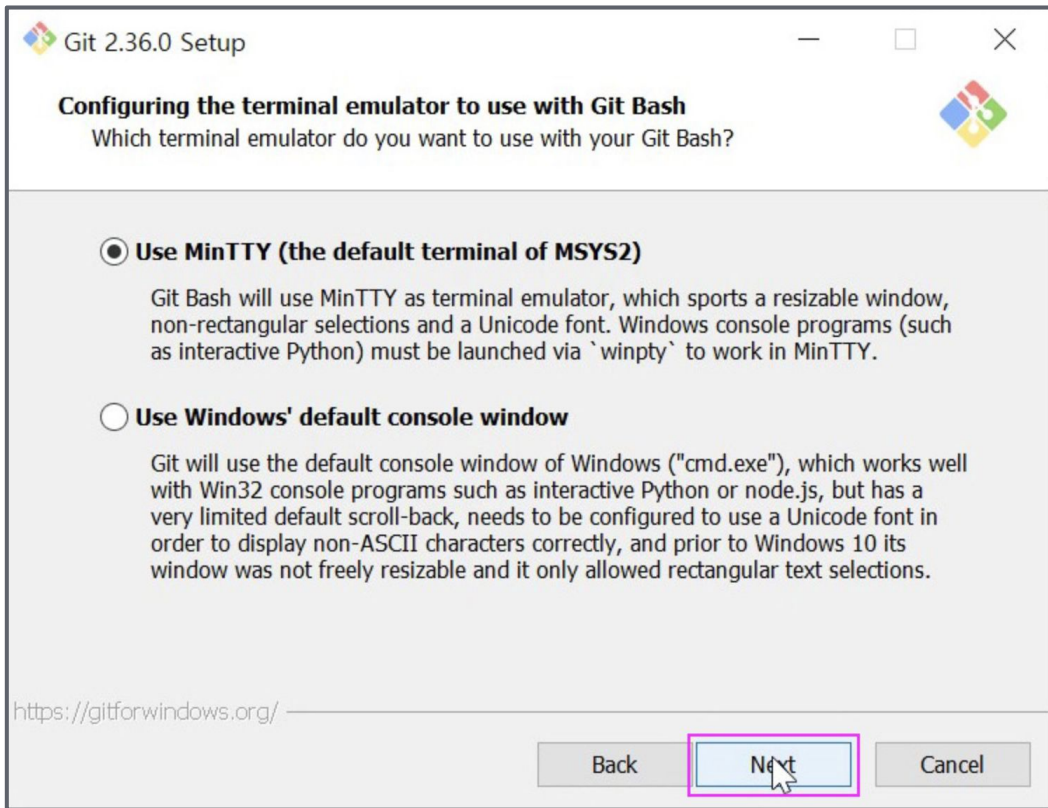
Git 설치



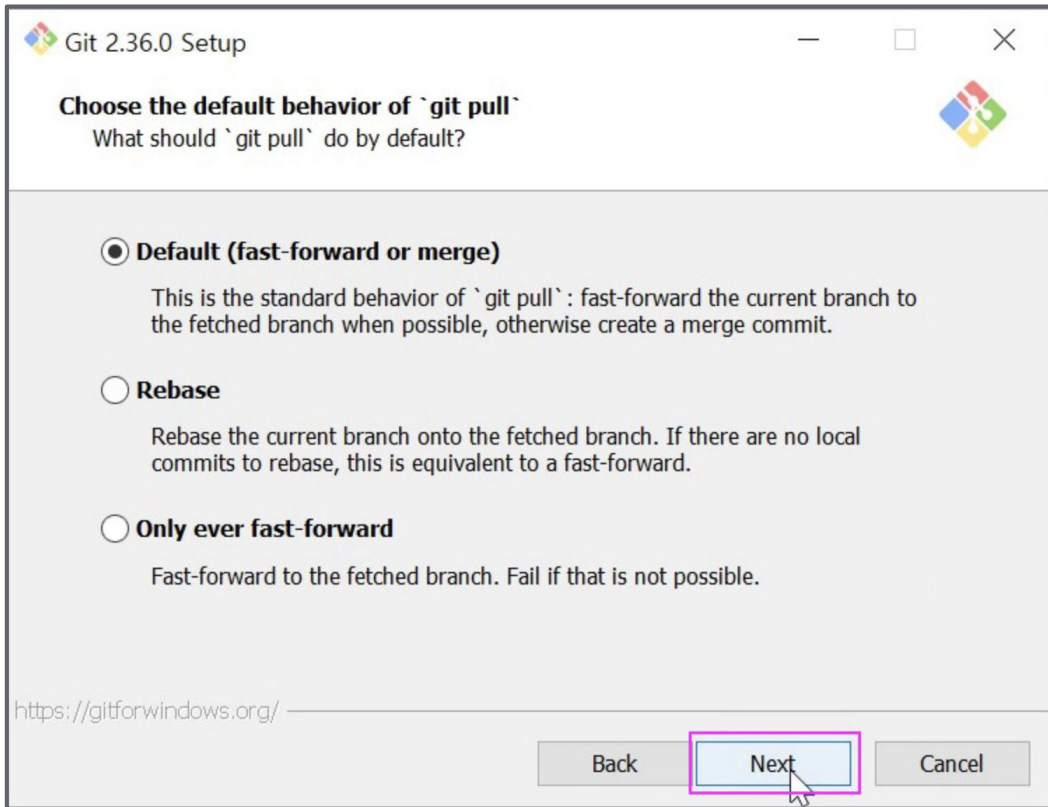
Git 설치



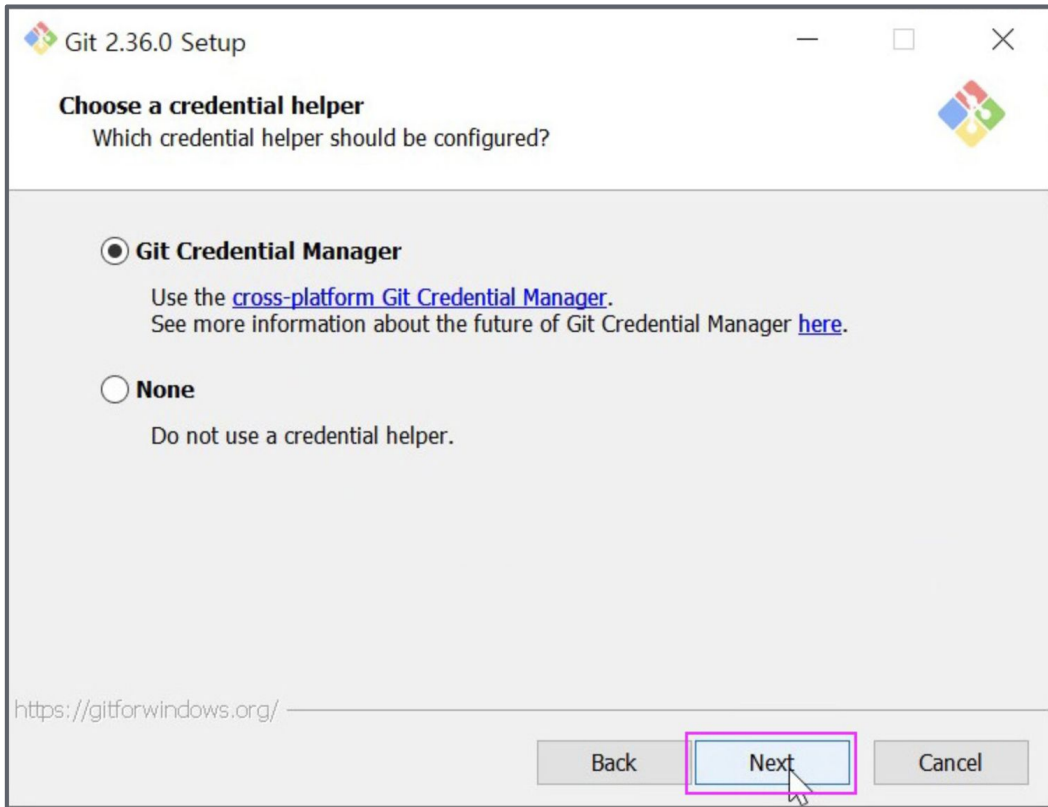
Git 설치



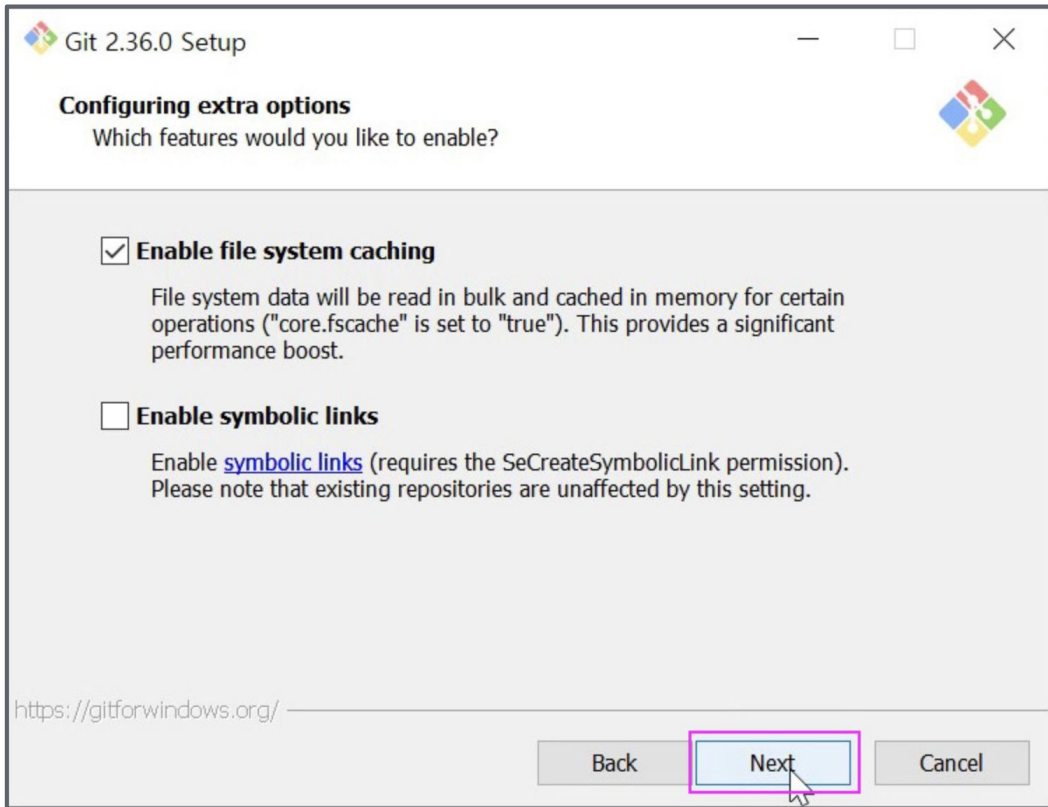
Git 설치



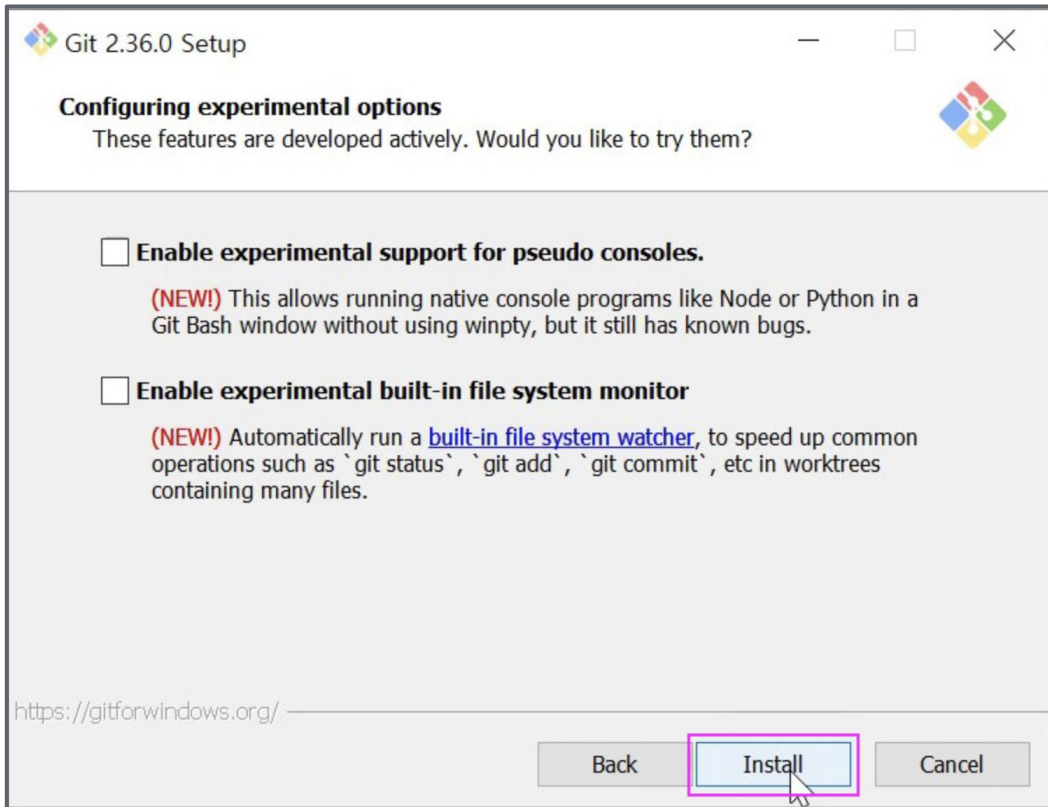
Git 설치



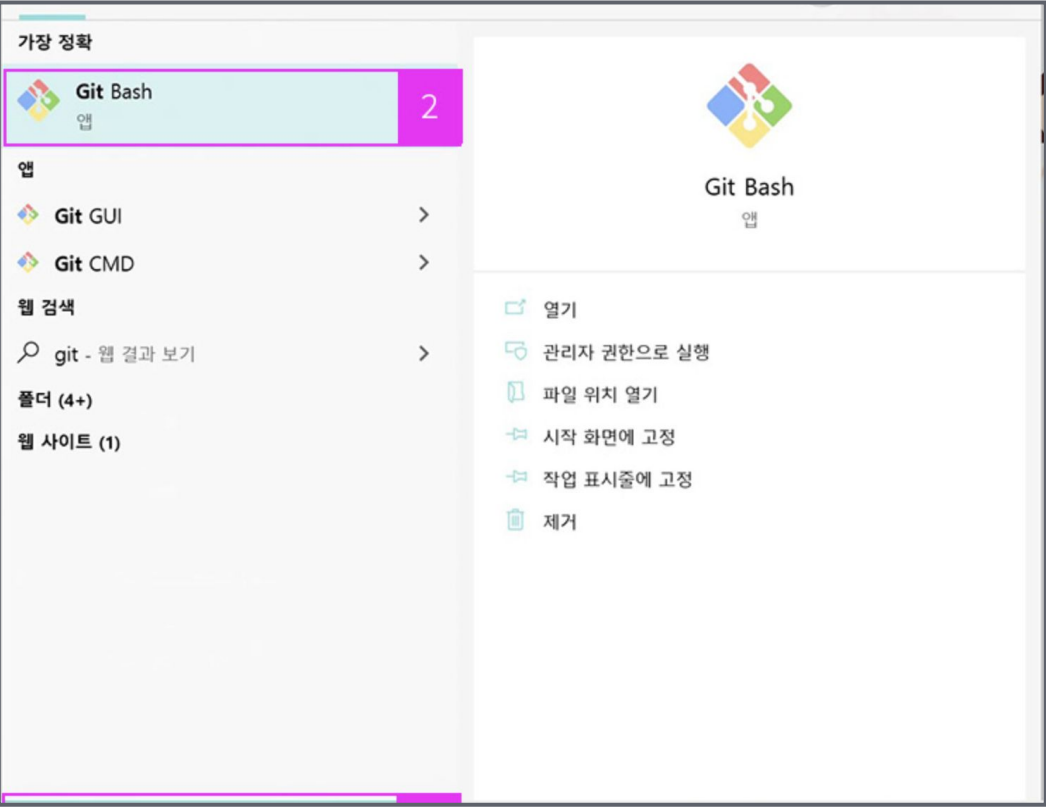
Git 설치



Git 설치



Git 설치

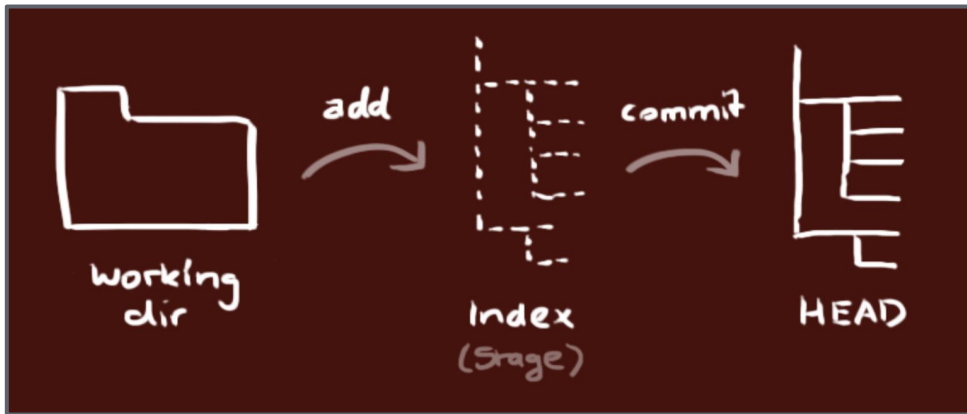


Git 설치 확인

A screenshot of a macOS terminal window. The title bar is light pink and contains three window control buttons (red, yellow, green) on the left and a folder icon followed by the text 'dongbeomkim — -zsh — 97x24' on the right. The terminal content area is white and shows the command 'git --version' being executed. The prompt 'dongbeomkim@dongbeomkim-YW1MV2XN32 ~ %' is visible before the command. A grey cursor block is at the end of the command line.

```
dongbeomkim@dongbeomkim-YW1MV2XN32 ~ % git --version
```

Git 작업 흐름



작업 흐름

git의 로컬 저장소는 세 그루의 나무로 구성되어 있음.

첫번째 작업 디렉토리 (실제 파일)

두번째 인덱스 (준비영역)

세번째 HEAD (최종 확정본)

Github Public 레포지토리 생성

 New repository

Q Type [/](#) to search



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner *

Repository name *

 DB-kim

 /

Great repository names are short and memorable. Need inspiration? How about [literate-fortnight](#) ?

Description (optional)

☒  **Public**

Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**

You choose who can see and commit to this repository.

Initialize this repository with:

☐ **Add a README file**

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None

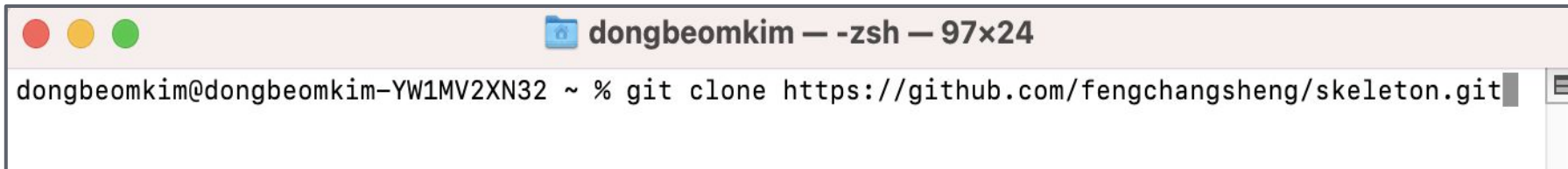
Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Git 저장소 받아오기

저장소 받아오기

로컬 저장소를 복제(clone)하려면 아래의 명령을 실행

```
git clone https://github.com/DB-kim/test.git
```

A terminal window with a title bar that reads "dongbeomkim — -zsh — 97x24". The terminal content shows the command "git clone https://github.com/fengchangsheng/skeleton.git" being entered at the prompt "dongbeomkim@dongbeomkim-YW1MV2XN32 ~ %".

```
dongbeomkim@dongbeomkim-YW1MV2XN32 ~ % git clone https://github.com/fengchangsheng/skeleton.git
```

Git 저장소 만들기

새로운 저장소 만들기

폴더를 생성하고, 폴더 안에서 'git init' 실행

새로운 git 저장소가 만들어짐

원격 레포지토리에 업로드

`git remote add origin <원격 서버 주소>`



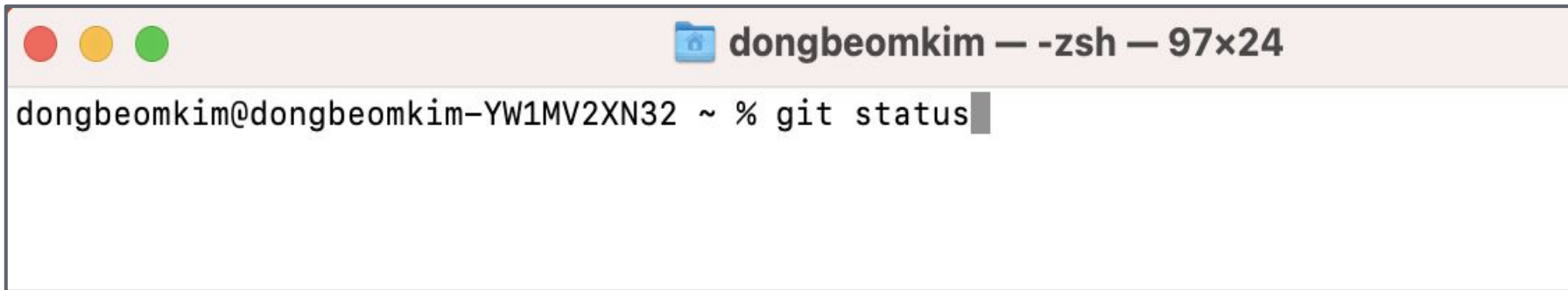
A terminal window titled "dongbeomkim — -zsh — 80x24" with a folder icon. The window shows the output of a login: "Last login: Mon Dec 18 01:58:57 on ttys000" and the current prompt "dongbeomkim@dongbeomkim-YW1MV2XN32 ~ %". The user has entered the command "git init" and the cursor is at the end of the line.

```
Last login: Mon Dec 18 01:58:57 on ttys000
dongbeomkim@dongbeomkim-YW1MV2XN32 ~ % git init
```

Git 디렉토리 상태확인

현재 상태 확인

git status



A terminal window with a title bar containing three colored circles (red, yellow, green) and the text 'dongbeomkim — -zsh — 97x24'. The terminal content shows the prompt 'dongbeomkim@dongbeomkim-YW1MV2XN32 ~ %' followed by the command 'git status' and a cursor.

```
dongbeomkim@dongbeomkim-YW1MV2XN32 ~ % git status
```

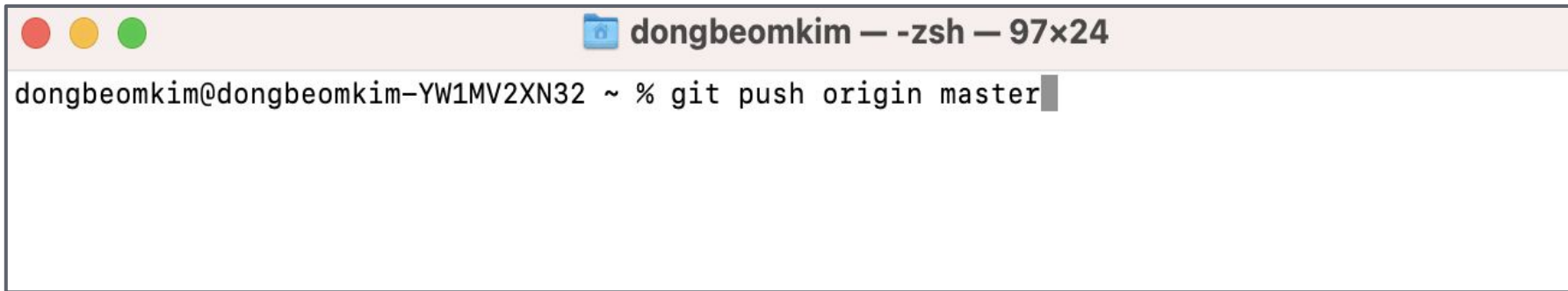
Git 변경 내용 발행 하기

원격 서버 업로드

`git push origin master`

기존 원격 저장소를 복제한게 아니라면

`git remote add origin <원격 서버 주소>`

A terminal window with a title bar that reads "dongbeomkim — -zsh — 97x24". The terminal content shows the command "dongbeomkim@dongbeomkim-YW1MV2XN32 ~ % git push origin master" followed by a cursor. The window has three colored window control buttons (red, yellow, green) on the left.

```
dongbeomkim@dongbeomkim-YW1MV2XN32 ~ % git push origin master
```

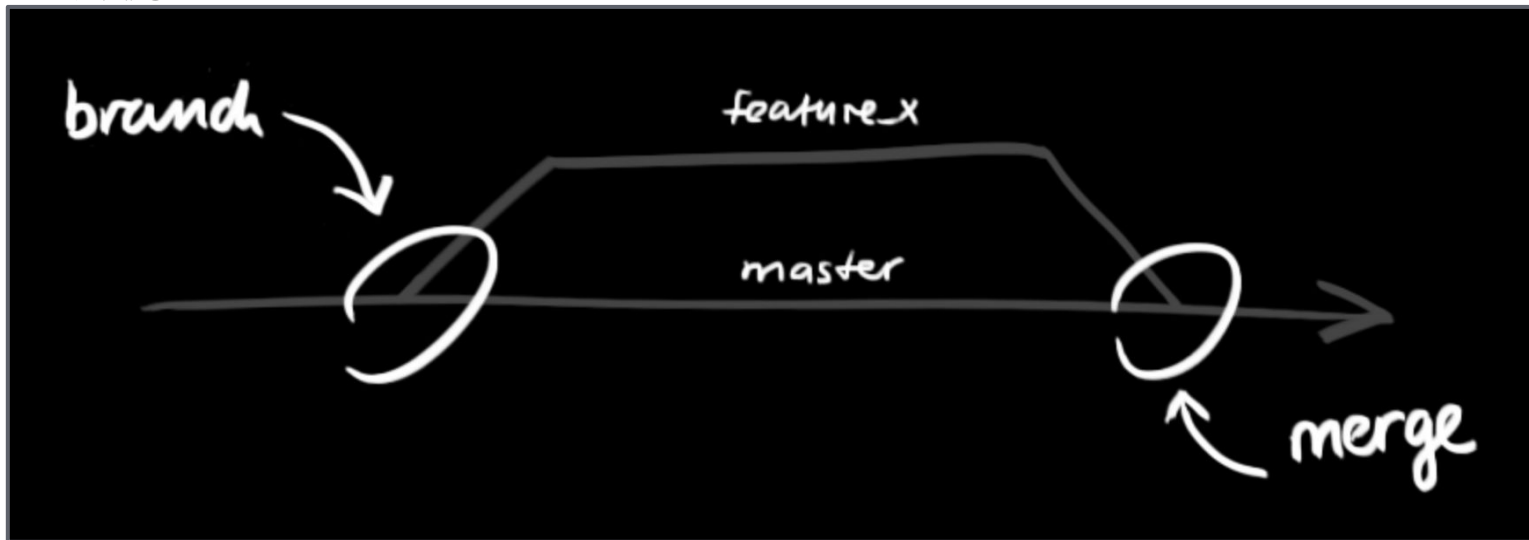

Git 가지(branch)치기

branch란?

가지는 안전하게 격리된 상태에서 무언가를 만들 때 사용함.

다른 가지를 이요하여 개발을 진행하고, 개발 완료시 master

가지에 병합

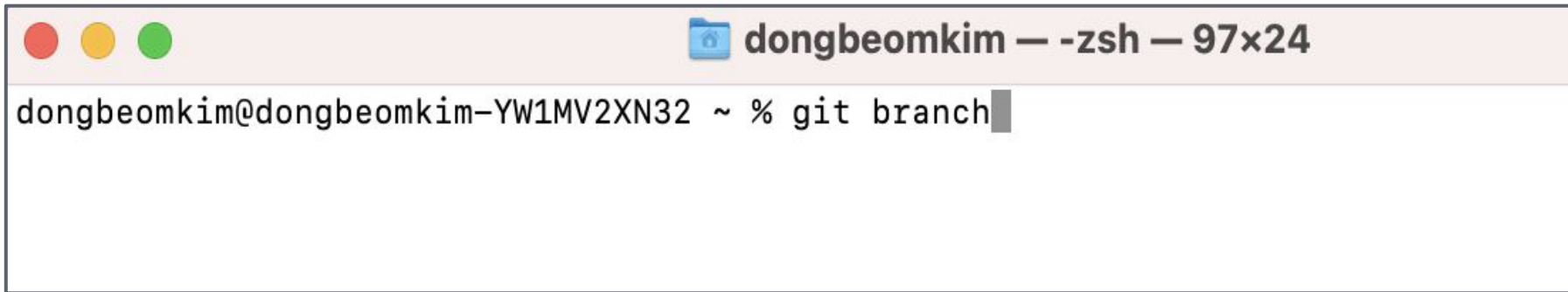


Git Branch 확인하기

Branch 확인

git branch

*이 현재 내가 위치한 브랜치를 뜻함

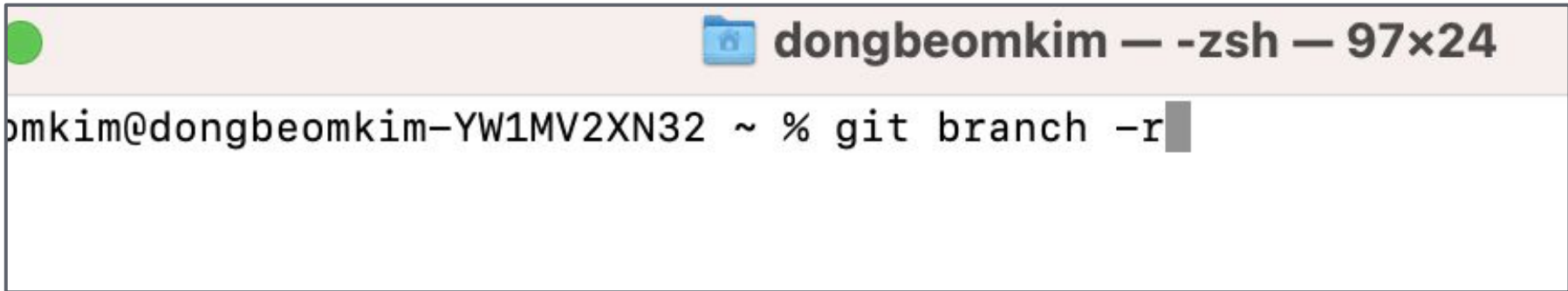
A terminal window with a title bar that reads 'dongbeomkim — -zsh — 97x24'. The window contains the command 'git branch' and its output, which is not visible in the image. The terminal window has a light gray background and a dark gray border.

```
dongbeomkim@dongbeomkim-YW1MV2XN32 ~ % git branch
```

Git 원격 저장소의 브랜치 확인

원격 저장소 **branch** 확인

git branch -r

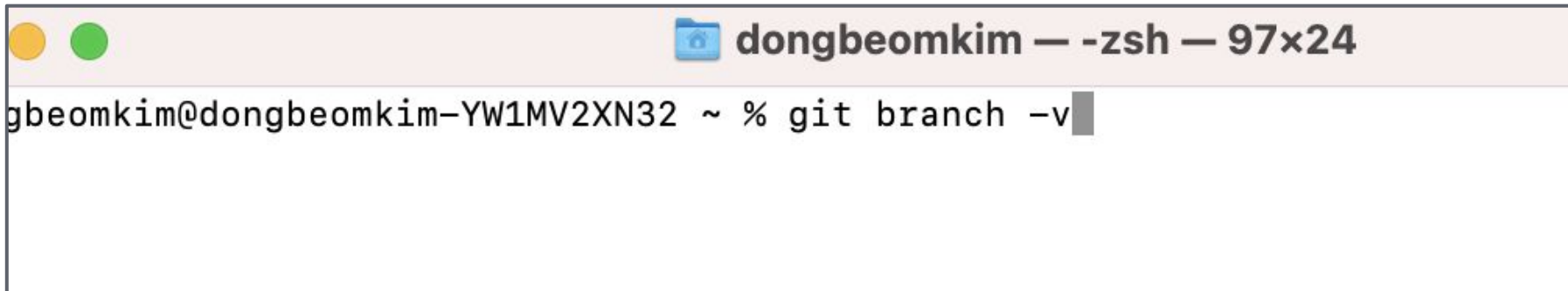
A screenshot of a terminal window. The title bar at the top is light pink and contains a blue folder icon, the text 'dongbeomkim', and the window title '- zsh - 97x24'. The terminal content shows the prompt 'dongbeomkim@dongbeomkim-YW1MV2XN32 ~ %' followed by the command 'git branch -r' which has been executed. A cursor is visible at the end of the command line.

```
dongbeomkim@dongbeomkim-YW1MV2XN32 ~ % git branch -r
```

Git 브랜치 마지막 커밋 메세지 확인

branch 커밋 메세지 확인

git branch -v



A screenshot of a macOS terminal window. The title bar at the top shows three window control buttons (yellow, green, red) on the left and a folder icon followed by the text 'dongbeomkim — -zsh — 97x24' on the right. The terminal content shows the prompt 'gbeomkim@dongbeomkim-YW1MV2XN32 ~ %' followed by the command 'git branch -v' with a cursor at the end.

```
gbeomkim@dongbeomkim-YW1MV2XN32 ~ % git branch -v
```

Git 브랜치 생성 및 이동

branch 생성하기

git branch test

branch 이동하기

git checkout test



A screenshot of a terminal window with a title bar that reads "dongbeomkim — -zsh — 97x24". The terminal content shows the prompt "dongbeomkim@dongbeomkim-YW1MV2XN32 ~ %" followed by the command "git branch test" and a cursor. The window has three colored window control buttons (red, yellow, green) in the top-left corner.

Git 브랜치 생성 및 이동(동시)

branch 생성 및 이동

git checkout -b 브랜치이름

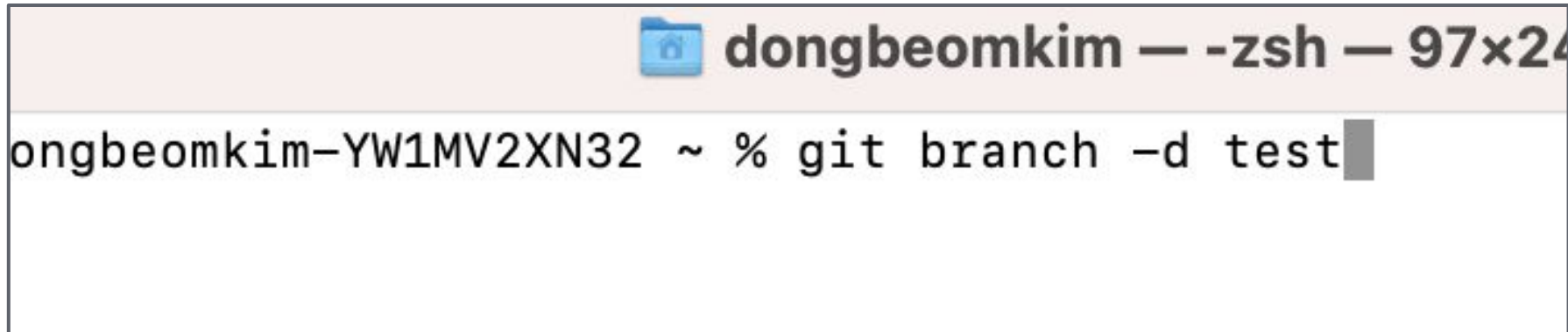
A screenshot of a macOS terminal window. The title bar at the top shows three colored window control buttons (red, yellow, green) on the left and the text 'dongbeomkim — -zsh — 97x24' on the right. The main area of the terminal is white and contains the text 'dongbeomkim@dongbeomkim-YW1MV2XN32 ~ % git checkout -b test2' followed by a grey cursor block.

```
dongbeomkim@dongbeomkim-YW1MV2XN32 ~ % git checkout -b test2
```

Git 브랜치 삭제

branch 삭제하기

git branch -d test



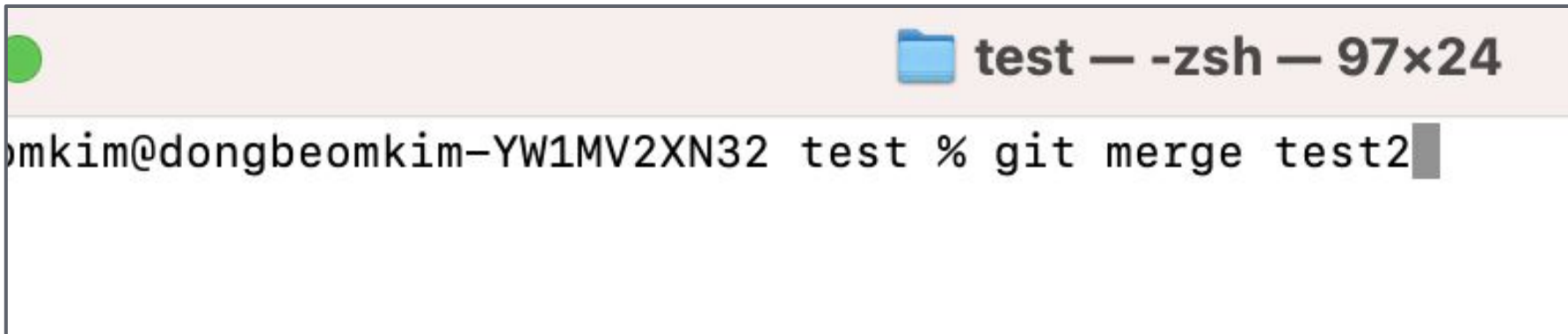
```
dongbeomkim — -zsh — 97x24  
ongbeomkim-YW1MV2XN32 ~ % git branch -d test
```

Git 병합

test2 branch 를 master에 머지하기

git checkout master

git merge test2

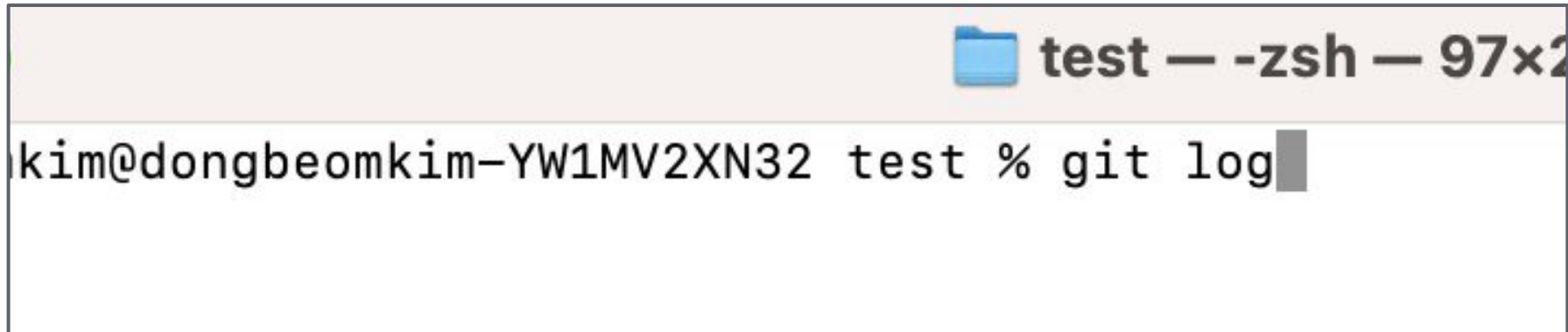
A screenshot of a terminal window. The title bar is light pink and contains a blue folder icon, the text "test", and the window title "test — -zsh — 97x24". The terminal content shows a green cursor at the end of the command "omkim@dongbeomkim-YW1MV2XN32 test % git merge test2".

```
omkim@dongbeomkim-YW1MV2XN32 test % git merge test2
```


Git 로그 확인하기

로그 확인하기

git log



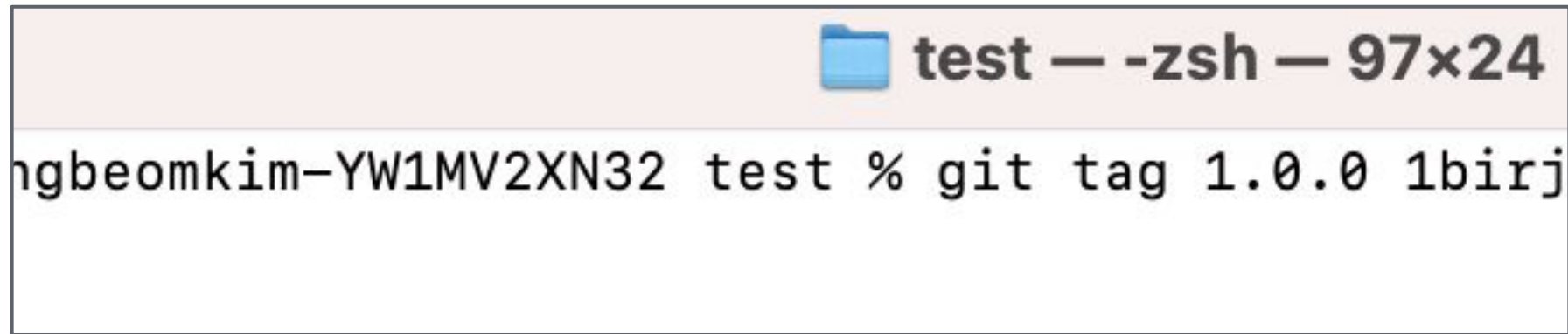
```
kim@dongbeomkim-YW1MV2XN32 test % git log
```

Git 꼬리표 (tag) 달기

tag 추가하기

새로운 버전을 출시할 때마다 tag를 달면 형상 식별하기 좋음

```
git tag 1.0.0 1234567
```



Git 병합 실습하기

branch 생성

두 개(A,B)의 branch에서 각각 새로운 내용(추가)을 push 한다.

branch 병합하기

두 개(A,B) 중 A부터 master 브랜치에 차례대로 merge 한다.

Git log 옵션

그래프 log 확인하기

git log --graph

모든 branch 커밋 이력 조회

git log --oneline --decorate --graph --all

commit 내용 삭제하고 보기

git log --graph --oneline

각 커밋 정보를 한 라인으로 요약해서 보기

git log --pretty=oneline

GitHub에서 확인하기

repository -> Insight -> Network

Git add 취소하기

```
// 모든 파일이 Staged 상태로 바뀐다.  
$ git add *  
// 파일들의 상태를 확인한다.  
$ git status  
On branch master  
Changes to be committed:  
(use "git reset HEAD <file>..." to unstage)  
    renamed:    README.md -> README  
    modified:   CONTRIBUTING.md
```

```
// CONTRIBUTING.md 파일을 Unstage로 변경한다.  
$ git reset HEAD CONTRIBUTING.md  
Unstaged changes after reset:  
M    CONTRIBUTING.md
```

Git commit 취소하기

```
// commit 목록 확인
```

```
$ git log
```

```
// [방법 1] commit을 취소하고 해당 파일들은 staged 상태로 워킹 디렉터리에 보존
```

```
$ git reset --soft HEAD^
```

```
// [방법 2] commit을 취소하고 해당 파일들은 unstaged 상태로 워킹 디렉터리에 보존
```

```
$ git reset --mixed HEAD^ // 기본 옵션
```

```
$ git reset HEAD^ // 위와 동일
```

```
$ git reset HEAD~2 // 마지막 2개의 commit을 취소
```

```
// [방법 3] commit을 취소하고 해당 파일들은 unstaged 상태로 워킹 디렉터리에서 삭제
```

```
$ git reset --hard HEAD^
```

Git commit 메세지 수정하기

```
$ git commit --amend
```

Git untracked 파일 삭제하기

```
$ git clean -f // 디렉터리를 제외한 파일들만 삭제  
$ git clean -f -d // 디렉터리까지 삭제  
$ git clean -f -d -x // 무시된 파일까지 삭제
```


Git ignore

Git Ignore이란?

git으로 추적하지 않을 파일 및 디렉토리의 목록을 지정하는 파일.

사용 방법

.gitignore 파일을 루트 디렉토리에 생성

파일에는 Git에서 무시할 파일 또는 패턴을 작성한다.

이점

Git으로 추적하지 않을 파일을 명시함으로써, Git 저장소를 깔끔하게 유지하고 불필요한 파일을 커밋하지 않을 수 있다.

Git ignore 실습

Git Ignore 파일 업데이트

스프레드 시트의 ignore 파일 업데이트

적용 테스트

target 파일을 기준으로 적용 테스트 진행

```
1  # Compiled class file
2  *.class
3
4  # Log files
5  *.log
6  log/
7  logs/
8
9  # IntelliJ IDEA
10 .idea/
11 *.iml
12 *.ipr
13 *.iws
14
15 # Eclipse
16 .classpath
17 .project
18 .settings/
19
20 # Maven
21 target/
22 pom.xml.tag
23 pom.xml.releaseBackup
24 pom.xml.versionsBackup
25 pom.xml.next
```

Git untracked 파일 삭제하기

```
$ git clean -f // 디렉터리를 제외한 파일들만 삭제  
$ git clean -f -d // 디렉터리까지 삭제  
$ git clean -f -d -x // 무시된 파일까지 삭제
```

Git Rebase

Rebase란?

말 그대로 **base**를 다시 선정하는 기능

브랜치의 이력을 변경하거나 다른 브랜치에 합치기 위해 사용하는 명령어로 기존 커밋을 재배포하거나 커밋을 합치는 작업을 수행할 때 사용됨.

사용 이유

현재 브랜치에서 새로운 기능을 개발하는 도중에 업데이트된 기반 브랜치의 변경사항을 가져오고 싶을 때 유용합니다.

브랜치를 깔끔하게 관리하고 커밋 이력을 정리하거나 수정할 수 있도록 도와줍니다.

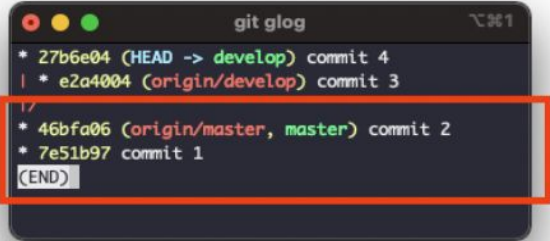
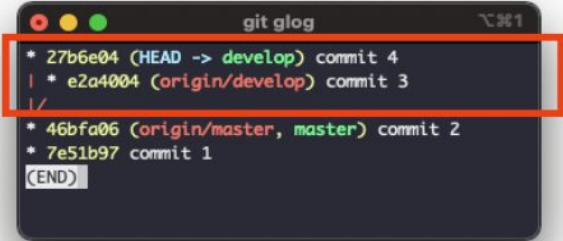
merge vs rebase

사전적 의미 **merge**는 합치다. **rebase**는 재배포하다.

Git Fast-Forward

Fast-Forward란?

자손 관계 여부를 나타냄

Fast-Forward 관계가 맞는 경우	Fast-Forward 관계가 아닌 경우
	
commit 1과 commit 2는 자손 관계	commit 3과 commit 4는 자손 관계가 아님
이런 경우를 Fast-Forward 관계 라고 부릅니다.	이런 경우를 Fast-Forward 관계가 아니라고 합니다.

Git Fast-Forward 관계일 때 Merge

Merge

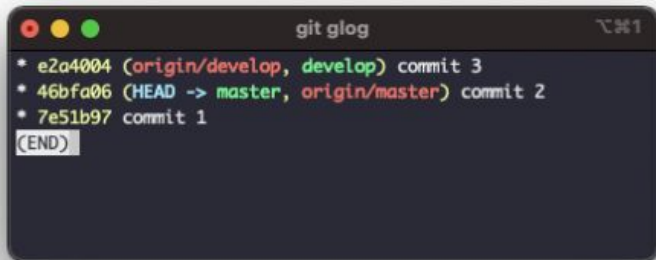
git checkout master

git merge develop

결과

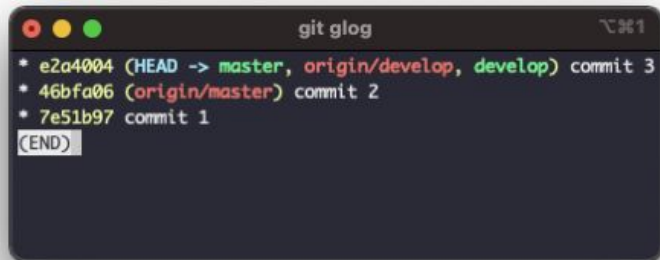
조상의 Commit을 그대로 이어감.

merge 전



```
git log
* e2a4004 (origin/develop, develop) commit 3
* 46bfa06 (HEAD -> master, origin/master) commit 2
* 7e51b97 commit 1
(END)
```

merge 후



```
git log
* e2a4004 (HEAD -> master, origin/develop, develop) commit 3
* 46bfa06 (origin/master) commit 2
* 7e51b97 commit 1
(END)
```

Git Fast-Forward 관계일 때 Rebase

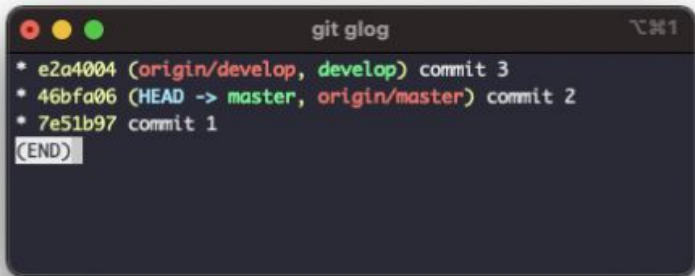
Merge

git checkout main
git rebase develop

결과

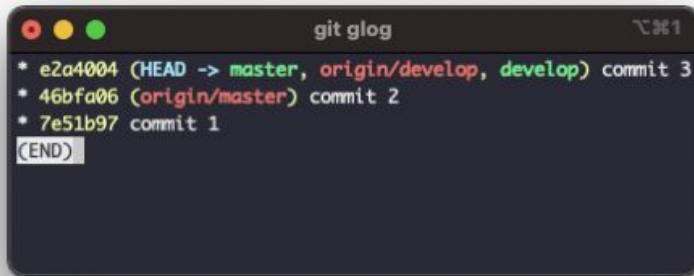
B(develop)을 유지하면서 A(master)를 변경시키기 때문에
조상 Commit을 따라가기만 함.

merge 전



```
git log
* e2a4004 (origin/develop, develop) commit 3
* 46bfa06 (HEAD -> master, origin/master) commit 2
* 7e51b97 commit 1
(END)
```

merge 후



```
git log
* e2a4004 (HEAD -> master, origin/develop, develop) commit 3
* 46bfa06 (origin/master) commit 2
* 7e51b97 commit 1
(END)
```

Git Fast-Forward 관계가 아닐 때 Merge

Merge

git checkout main

git merge develop

결과

commit 1, 2가 Fast-Forward 관계가 아님.

이런 경우에는 merge 커밋이 함께 생성되고 그래프가 만들어짐

merge 전



```
git glog
* 7bfbd9e (HEAD -> master, origin/master) commit 4
| * e2a4004 (origin/develop, develop) commit 3
|/
* 46bfa06 commit 2
* 7e51b97 commit 1
(END)
```

merge 후



```
git glog
* 987eac6 (HEAD -> master) Merge branch 'develop' into mas
ter
| \
| * e2a4004 (origin/develop, develop) commit 3
| | 7bfbd9e (origin/master) commit 4
|/
* 46bfa06 commit 2
:
```


Git Fast-Forward 관계가 아닐 때 Rebase

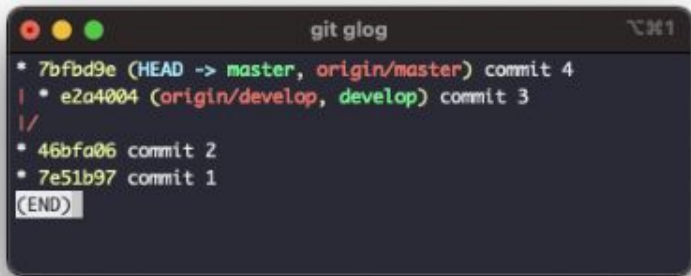
Merge

git checkout main
git rebase develop

결과

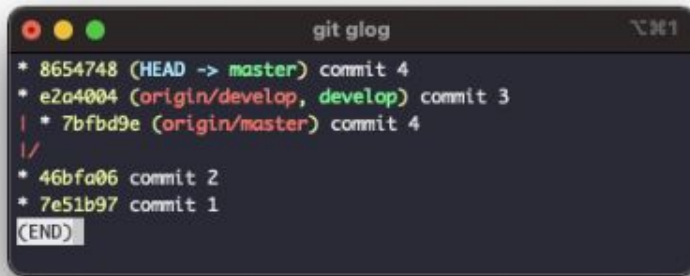
B(develop)을 유지하면서 A(master)를 변경시키기 때문에 A를 B의
조상으로 만들어줌
commit 3 위에 commit 4가 재배치가 이루어짐.

rebase 전



```
git log
* 7bfd9e (HEAD -> master, origin/master) commit 4
| * e2a4004 (origin/develop, develop) commit 3
|/
* 46bfa06 commit 2
* 7e51b97 commit 1
(END)
```

rebase 후



```
git log
* 8654748 (HEAD -> master) commit 4
* e2a4004 (origin/develop, develop) commit 3
| * 7bfd9e (origin/master) commit 4
|/
* 46bfa06 commit 2
* 7e51b97 commit 1
(END)
```

Git Merge vs Rebase

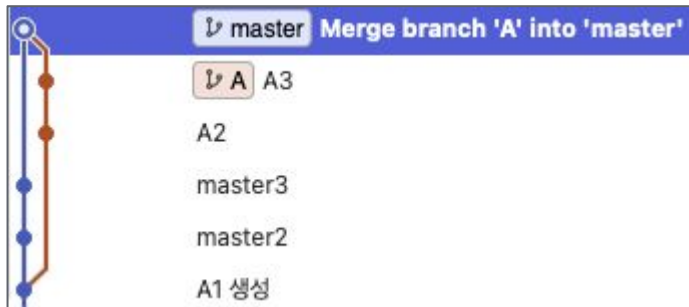
Merge vs Rebase

merge/rebase 둘다 모두 현재 브랜치에서 다른 브랜치의 소스를 적용하는 방법.

차이점

merge는 merge commit 메시지와 함께 세모 또는 네모를 그린다.

rebase는 그래프에서 일자를 그린다.



Git squash

Git squash란?

여러 개의 커밋을 하나로 압축하는 작업을 의미함.

커밋 히스토리를 깔끔하게 정리하거나, 여러 개의 작은 커밋을 하나의 의미 있는 커밋으로 결합할 수 있음.

사용 이유

프런트, 서버 사이드 소스 코드 커밋, 테스트 코드 커밋, 리뷰 대응 커밋 등 한 이슈를 수정하기 위해 여러 커밋을 하게 됨.

이러한 커밋 내역이 쌓이면 나중에 이력 추적이 어려워짐.

협업을 진행하거나, 회사에서 소스코드를 수정할 때 작업이 완료되면 만든 브랜치에 수정한 내용을 커밋합니다. 하지만 보통 커밋 한번에 모든 것을 수정할 수 없을 때가 있

주의사항

원격 저장소에 이미 푸쉬가 되어있는 경우에는 강제 **Squash**를 사용하려면 강제 푸쉬가 필요. 협업시 강제 푸쉬는 위험요소 중 하나이기 때문에 주의해야합니다.

주의사항

원격 저장소에 이미 푸쉬가 되어있는 경우에는 강제 **Squash**를 사용하려면 강제 푸쉬가 필요. 협업시 강제 푸쉬는 위험요소 중 하나이기 때문에 주의해야합니다.

Git squash 실습 - 최신 n개 커밋 병합

현재 커밋 로그 확인

git log --pretty=oneline

브랜치 생성

git branch feature/rebase-example

브랜치 이동

git checkout feature/rebase-example

메서드 생성

main 클래스에 메서드 추가 생성 (스프레드 시트 참고)

커밋 메시지는 메서드 하나당 아래와 같이 설정

first commit

second commit

third commit

3개 커밋 로그 확인

git log --pretty=oneline

```
dongbeomkim@dongbeomkim-YW1MV2XN32 java-maven-junit-skeleton % git log --pretty=oneline
a2df845f73aef014b14710faae021a0740712091 (HEAD -> feature/rebase-example) third commit
1bca6683a3b9bd91cbc1ba8624fe2152dc968284 second commit
c81baa2ef0427efb93dc64ff488fe67ce1d863b4 first commit
28839a1d1dca160375eaca1da740e325cb859f0c (origin/main, main) 웹훅 테스트
7e6ed17ce69a6867d5c3574c19699e50fe412fbc Create webhook.yml
```

Git squash 실습 - 최신 n개 커밋 병합

Rebase 실행

`git rebase -i <합치려는 Commit 전 Hash Commit>`

i 옵션은 대화형 모드(interactive mode)를 의미함.

커밋 병합

3개 중 합칠 로그에는 pick

나머지 2개는 squash 옵션인 s로 변경해준다.

커밋 메세지 병합

3개의 커밋 메세지를 하나로 수정 해준다.

원격 저장소 푸쉬

`git push --set-upstream origin feature/rebase-example`

Rebase 도중 취소

`git rebase --abort`

```
dongbeomkim@dongbeomkim-YW1MV2XN32 java-maven-junit-skeleton % git log --pretty=oneline
a2df845f73aef014b14710faae021a0740712091 (HEAD -> feature/rebase-example) third commit
1bca6683a3b9bd91cbc1ba8624fe2152dc968284 second commit
c81baa2ef0427efb93dc64ff488fe67ce1d863b4 first commit
28839a1d1dca160375eaca1da740e325cb859f0c (origin/main, main) 웹훅 테스트
7e6ed17ce69a6867d5c3574c19699e50fe412fbc Create webhook.yml
```

```
pick e9db56c first
pick 4362953 second
pick f23cba1 third

# Rebase ae4bb9f..f23cba1 onto ae4bb9f (3 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
```

Git ignore

Git Ignore이란?

git으로 추적하지 않을 파일 및 디렉토리의 목록을 지정하는 파일.

사용 방법

.gitignore 파일을 루트 디렉토리에 생성

파일에는 Git에서 무시할 파일 또는 패턴을 작성한다.

이점

Git으로 추적하지 않을 파일을 명시함으로써, Git 저장소를 깔끔하게 유지하고 불필요한 파일을 커밋하지 않을 수 있다.

Git ignore 실습

Git Ignore 파일 업데이트

스프레드 시트의 ignore 파일 업데이트

적용 테스트

target 파일을 기준으로 적용 테스트 진행

```
1  # Compiled class file
2  *.class
3
4  # Log files
5  *.log
6  log/
7  logs/
8
9  # IntelliJ IDEA
10 .idea/
11 *.iml
12 *.ipr
13 *.iws
14
15 # Eclipse
16 .classpath
17 .project
18 .settings/
19
20 # Maven
21 target/
22 pom.xml.tag
23 pom.xml.releaseBackup
24 pom.xml.versionsBackup
25 pom.xml.next
```

Git Commit Lint

Commit Lint란?

커밋 메시지의 형식과 내용을 일관성 있게 유지하기 위해 사용되는 도구입니다. 커밋 메시지는 프로젝트 가독성, 검색성, 변경 이력 추적 등에 중요한 역할을 하므로, 명확한 내용을 갖추는 것이 좋습니다.

기능

사전에 정의한 규칙에 따라 커밋 메시지를 작성할 수 있습니다.

Lint 도구 종류

commitlint, husky, lint-staged, prettier 등등

Git Commit Lint 실습

Commitlinter-maven-plugin 사용

플러그인 추가(스프레드 시트)

패턴 정의

pom.xml을 통해 정규식을 넣어 commit 정책을 정한다.

실행 명령어

```
mvn commitlinter:validate
```

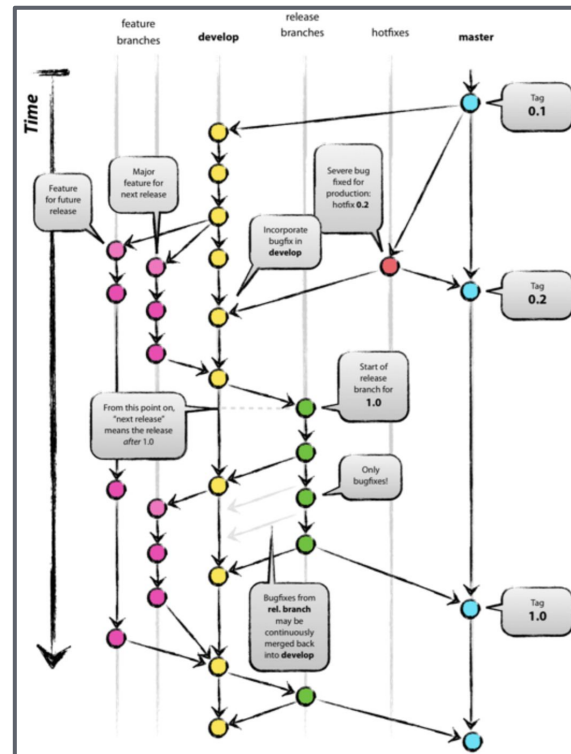
최근 commit 메시지 변경

```
git commit --amend
```

Git WorkFlow

git workflow

각 브랜치 의미 파악하기



Git Actions - 트리거 설정 예시

```
name:   
on:  
  push:  
    branches:  
      - main # 원하는 브랜치 이름으로 변경할 수 있습니다.
```

```
name:   
on:  
  pull_request:  
    branches:  
      - main # 원하는 브랜치 이름으로 변경할 수 있습니다.
```

Git Actions - 트리거 설정 예시

```
name: [REDACTED]
on:
  schedule:
    - cron: '0 0 * * *' # 매일 00:00 시에 실행하도록 설정
```

Git Actions - jobs 설정 예시

```
jobs: [redacted]  
  build:  
    runs-on: ubuntu-latest  
    steps:  
      - name: Checkout code  
        uses: actions/checkout@v2
```

...

Git Actions 실습 - Push Action 실행

Repository 생성

<https://github.com/><username>

Skeleton Project Clone

<https://github.com/gsaslis/java-maven-junit-skeleton>

Git 현재 원격 저장소 확인

```
git remote -v
```

기존 원격 저장소 제거

```
git remote remove origin
```

원격 저장소 추가

```
git remote add origin <원격 저장소 URL>
```

강제 덮어 쓰기

```
git branch -M main
```

신규 원격 저장소 Push

```
git push -u origin main
```

Git Actions 실습 - Workflow 작성

Git Workflow 파일 생성

프로젝트 루트 디렉토리에 '.github/workflows' 디렉토리 생성

```
mkdir .github  
cd .github  
mkdir workflows  
cd workflows
```

yml 파일 생성

```
touch test.yml
```

Workflow 정의

<https://docs.google.com/spreadsheets/d/1bFSiFU8wRLnYlfcicnsGh2DEAQR0qNLnUaO-QO05Wf4/edit#gid=1037294118>

데모 파일 test.yml 삽입

```
vi test.yml -i
```

Push 실행

```
git status  
git add .  
git commit -m 'Workflows 추가'  
git push
```

<https://github.com/dong-beom/git-study/actions>

actions 정상 실행 확인

remote Error

```
git config --global user.name "username"  
git config --global user.email "email@email.com"
```

Git Actions 실습 - Push Demo 리뷰

name: GitHub Actions Demo

run-name: `{{ github.actor }}` is testing out GitHub Actions 🚀

on: [push]

jobs:

Explore-GitHub-Actions:

runs-on: ubuntu-latest

steps:

- **run:** echo "🎉 The job was automatically triggered by a `{{ github.event_name }}` event."
- **run:** echo "🐧 This job is now running on a `{{ runner.os }}` server hosted by GitHub!"
- **run:** echo "🔍 The name of your branch is `{{ github.ref }}` and your repository is `{{ github.repository }}`."
- **name:** Check out repository code
uses: actions/checkout@v3
- **run:** echo "💡 The `{{ github.repository }}` repository has been cloned to the runner."
- **run:** echo "💻 The workflow is now ready to test your code on the runner."
- **name:** List files in the repository
run: |
ls `{{ github.workspace }}`
- **run:** echo "🍏 This job's status is `{{ job.status }}`."

1. Push 이벤트 발생시 자기소개 문구가 실행되는 워크플로우 작성 (형식 자유)

Git Actions 실습 - Java 실행

name: Java Maven Test

on: push

jobs:

builds:

runs-on: ubuntu-latest

steps:

- **name:** Checkout Code
- **uses:** actions/checkout@v2
- **name:** Set up JDK 11
- uses:** actions/setup-java@v2
- with:**
 - java-version: 11
 - distribution: 'adopt'

- **name:** Build with Maven

run: mvn clean install

- **name:** Run tests

run: mvn test

Git Actions 실습 - Test Code 실행 문제

- name: Build with Maven

run:

- name: Run tests

run:

Git Actions 실습 - Test Code 실행 답안

- **name:** Build with Maven

run: mvn clean install

- **name:** Run tests

run: mvn test

Git Actions 활용 - 코드 정적 분석

정적 분석이란?

소스 코드의 구조, 문법, 규칙, 디자인 패턴 등을 분석하여 잠재적인 오류, 버그 취약성, 일반적인 코드 품질 문제를 탐지하는 자동화 테스트 기법

코딩 스타일과 가이드라인을 준수할 수 있도록 지원

식별 종류

1. 코드 오류 : 컴파일 오류, 문법 오류, 타입 오류 등
2. 버그와 취약점 : 잠재적 버그나 보안 취약점을 탐지
3. 코드 스타일 : 코드 스타일 가이드라인 준수
4. 성능 및 최적화 : 코드의 성능 문제나 비효율적인 부분을 식별

정적 분석 Tool

Coverity, SonarQube 등등

SpotBugs

자바용 오픈소스 정적 분석 도구로 자바 소스 코드를 분석하는 방식으로 만들어졌기 때문에 소스 파일이 없는 외부 패키지도 분석이 가능함

각종 빌드 도구(Maven, Gradle 등), IntelliJ, Eclipse에 연동하여 쉽게 사용이 가능하다.

Git Actions 활용 - SQL Injection 코드 추가

SQL Injection

악의적인 사용자가 보안상의 취약점을 이용하여, 임의의 SQL 문을 주입하고 실행되게 하여 데이터베이스가 비정상적인 동작을 하도록 조작하는 행위

SQL Injection Class 생성

Injection 위험이 있는 코드 삽입 (스프레드 시트)

Git Actions 활용 - SpotBugs 추가

SpotBugs Plug-In 추가

pom.xml에 SpotBugs 디펜던시 추가 (스프레드 시트)

Maven Reload, update

IntelliJ : Maven Reload

Eclipse : Maven Update

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.3</version>
      <configuration>
        <source>1.8</source>
        <target>1.8</target>
      </configuration>
    </plugin>

    <plugin>
      <groupId>com.github.spotbugs</groupId>
      <artifactId>spotbugs-maven-plugin</artifactId>
      <version>4.3.0</version>
    </plugin>
  </plugins>
</build>
```

Git Actions 활용 - SpotBugs WorkFlow 적용 및 실행

- **name:** Run Static Code Analysis

run: mvn spotbugs:check

SpotBugs Check 옵션

빌드 중에 코드에 대한 정적 분석을 수행

결과를 통해 코드의 오류 및 취약점을 탐지

Git Actions 심화 - Slack Webhook

 **에드** 오후 12:59
우와 최신기술~

 **GitHub** 앱 오후 1:00
Issue closed by knae11
[#7 \[issue\] test issue for slack notification 3](#)
woowacourse-teams/2021-gpu-is-mine | 오늘 오후 12:55

 **GitHub** 앱 오후 1:06
Issue opened by knae11
× [#8 \[issue\] test issue opening](#)
hi~
Labels
documentation
woowacourse-teams/2021-gpu-is-mine | 오늘 오후 1:06

Issue closed by knae11
[#8 \[issue\] test issue opening](#)
woowacourse-teams/2021-gpu-is-mine | 오늘 오후 1:06

 **배럴** 오후 1:09
테스트 해봤어요~
깃헙 아이디 연동해 놓으면 여기 글쓰기로도 issue open, close 등등 다 되네요~~ 코기최고다!! 🍌🍌
여기에서 `/github help` 로 입력하면 명령어 도움말 확인가능~!
👍 1 🗨

Git Actions 심화 - Slack Webhook

Webhook yml 파일 생성

GitHub를 통해 생성

워크플로우 코드는 스프레드 시트 참고

Slack 채널 생성

Git Action Notification 용 채널 생성

Incoming Webhooks 설정

Webhook URL 복사

Features

App Home

Org Level Apps

Incoming Webhooks

Interactivity & Shortcuts

Slash Commands

Webhook URLs for Your Workspace

To dispatch messages with your webhook URL, send your [message](#) in JSON as the body of an [application/json](#) POST request.

Add this webhook to your workspace below to activate this curl example.

Sample curl request to post to a channel:

```
curl -X POST -H 'Content-type: application/json' --data '{"text":"Hello, World!"}'  
https://hooks.slack.com/services/T059J2TM41M/B05ANAC49PA/a9ZG1lzT5n6EF1PkEdS  
Gjx3j
```

Copy

Webhook URL

Channel

Added By

https://hooks.slack.com/services/

Copy

#demo

김동범DongBeom Kim

May 29, 2023



Add New Webhook to Workspace

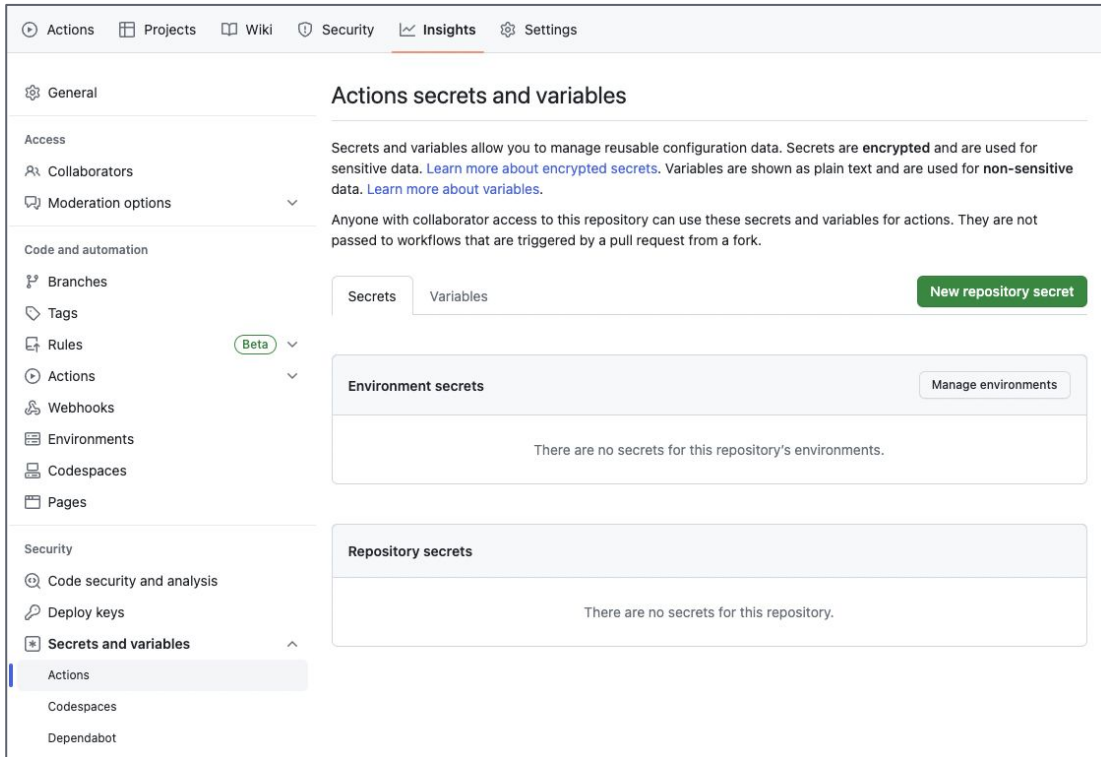
Git Actions 심화 - Slack Webhook

Webhook URL 설정

Slack의 Webhook URL을 Repository에 설정

실행

Actions 동작 테스트



Git Actions 심화 - 배포 자동화 예시

```
- name: Deploy to AWS S3
  uses: jakejarvis/s3-sync-action@master
  with:
    args: --acl public-read --delete
  env:
    AWS_S3_BUCKET: ${ secrets.AWS_S3_BUCKET }
    AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
    AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
    SOURCE_DIR: 'dist' # 빌드된 애플리케이션 폴더 경로
    DEST_DIR: '/' # S3에 배포될 폴더 경로
```

Git Actions 심화 - S3 정적 웹사이트 배포

S3 버킷 생성

AWS Console에 접속하여 S3 버킷을 생성한다.

퍼블릭 액세스 설정

퍼블릭 액세스가 가능하도록 설정한다.

버킷 생성

퍼블릭 액세스 버킷을 생성한다.

HTML 파일 업로드

Free HTML 파일을 업로드 한다.

정적 웹사이트 호스팅

정적 웹사이트 호스팅 설정을 한다.

index.html 을 메인 페이지로 설정

Error 페이지가 존재할 경우 error 페이지 설정

동작 확인

정적 웹사이트 URL을 통해 접속한다.

Git Actions 심화 - S3 Workflow 설계

권한 사용자 생성

IAM을 통해 github-actions 라는 user를 생성한다.

사용자 권한 매핑

github-actions 사용자에게 S3 Full 권한을 부여한다.

사용자 액세스 키 발급

github-actions 사용자의 액세스 키, 시크릿 키를 발급한다.

Github 키 등록

github secret 설정을 통해 키, 시크릿 키를 등록한다.

Workflow 설계

s3-upload.yml 파일을 추가하여 Workflow를 추가한다.(스프레드 시트)

파일 정상 업로드 확인

index.html의 내용을 변경하여 Actions 동작 유무를 확인한다.

Git Actions Demo

1. Pull Request 발생시 Actions 작성 (자유 형식)
2. 각 스크럼 별 시연

Agile - Scrum 이란?

Agile의 정의

지속적으로 개선하는 데 도움이 되는 프로젝트 관리 철학

Scrum의 정의

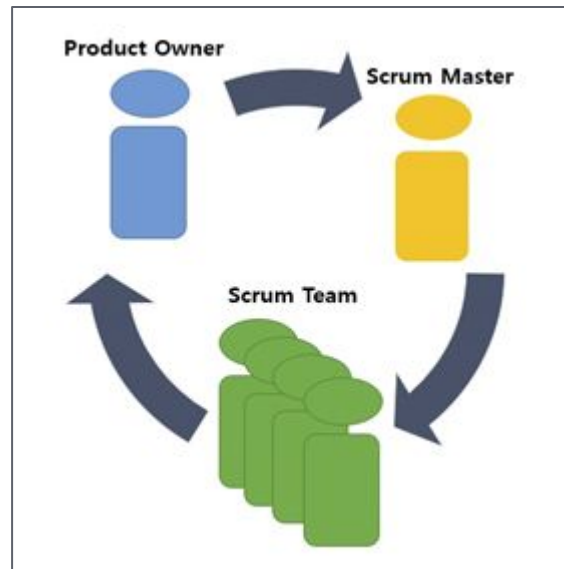
애자일 소프트웨어 개발 방법론 중 하나로, 프로젝트 관리와 개발 프로세스를 위한 프레임워크

Scrum 구성원

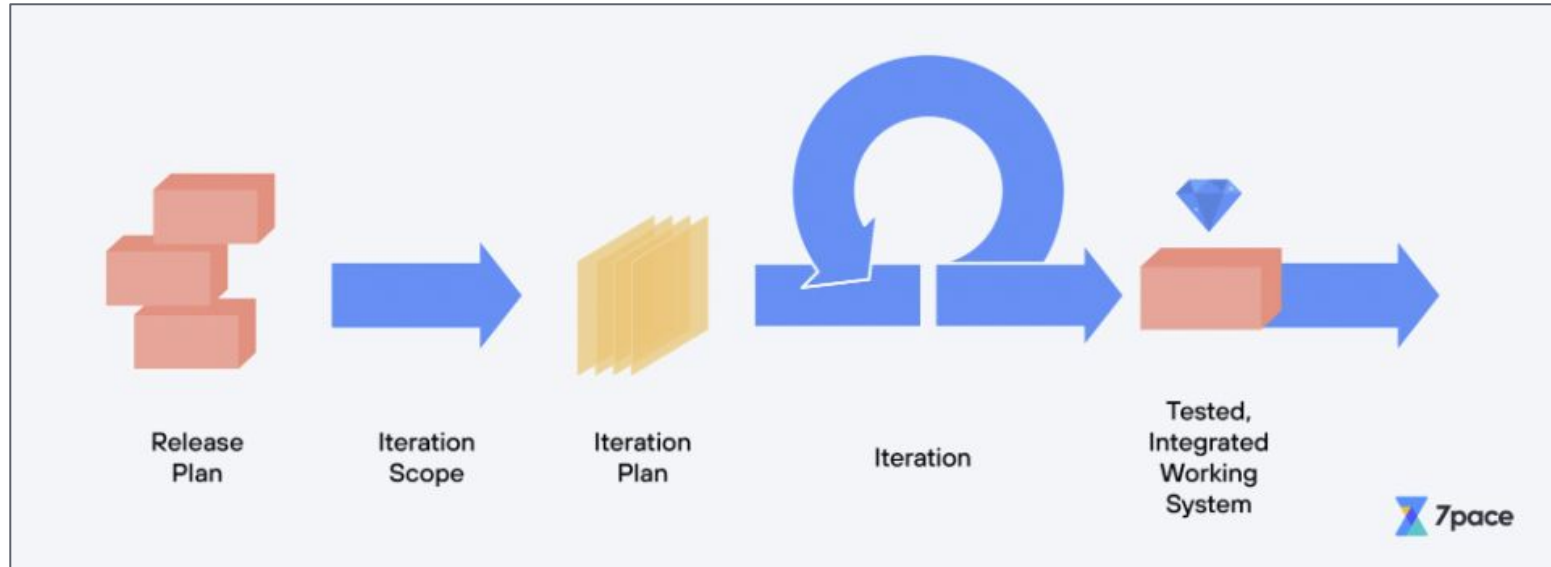
PO : 제품의 품질을 책임지며, 서비스 기획 업무 수행

Scrum Master : Scrum의 리더이며, 스크럼 운영을 책임진다.

Scrum Team : 개발해야되는 목표 기능의 전문가



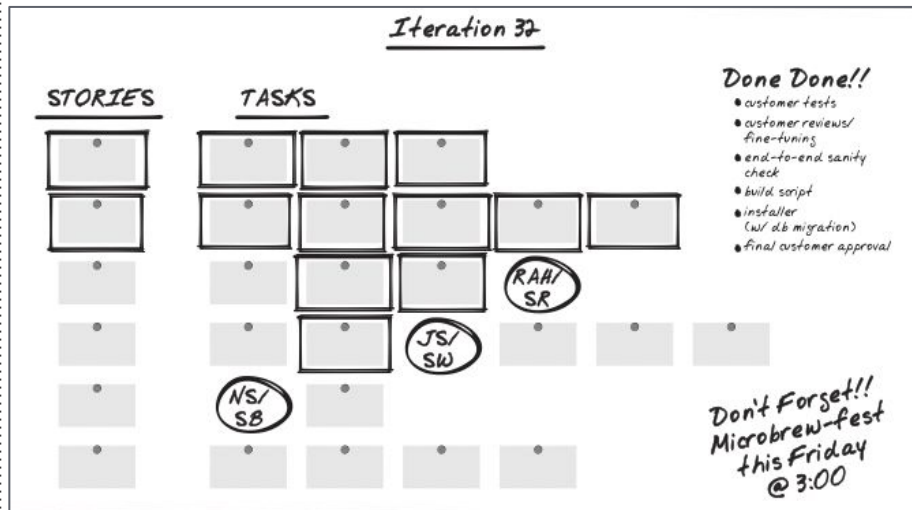
Agile - Iteration



Agile - Iteration Planning

Iteration Planning이란?

애자일 소프트웨어 개발 방법론에서 사용되는 개념으로, 팀이 반복 주기 동안 수행할 작업을 계획하고 구체화하는 활동입니다. 이를 통해 팀은 반복 주기 동안 어떤 작업을 완료할 것인지를 명확히 정의하고, 목표를 설정하며, 작업을 우선순위에 따라 정렬하여 수행할 수 있습니다.



Agile - Iteration Planning

1. Iteration 목표(Goal) 설정

해당 Iteration에 이를 목표 설정 (릴리즈 기준)

2. BackLog 작성

최대한 작은 단위의 BackLog 설정

Desc에 DoD(definition of done) 명시

DONE

adjective

1. Unit tests passed
2. Code reviewed
3. Acceptance criteria met
4. Functional Tests passed
5. Non-Functional requirements met
6. Product Owner accepts the User Story

Backlog 4

8h 19m

Fullscreen mode refactoring

everhour/everhour-web#624

opened by yurytolochko

frontend

JSON: discriminator

everhour/everhour-framework#13

opened by yurytolochko

Migrate to recent angular2 version

everhour/everhour-web#640 opened by yurytolochko

frontend

Data refactoring

0 of 2

everhour/everhour-browser-extension#167 opened by yurytolochko

8h 19m

extension

Agile - Iteration Planning

3. BackLog별 Story Point 작성

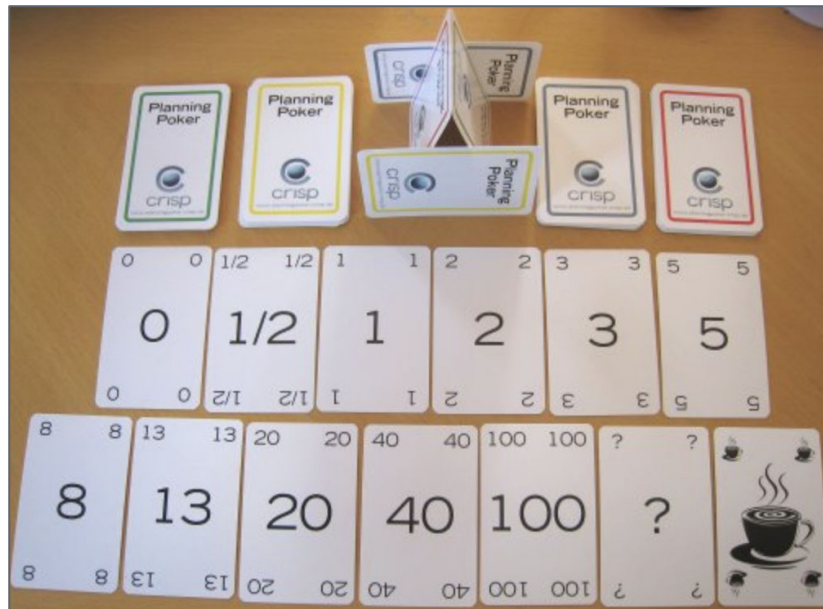
편의상 1h을 1로 산정하여 작성

피보나치 수열을 사용하여 작성 - 플래닝 포커

상대적인 차이를 만들기 때문에 피보나치 수열을 이용

4. Iteration 주기에 맞는 스토리 포인트 배정

2주를 기준으로 작업 계획



Git Projects

Git Projects란?

GitHub에서 제공하는 프로젝트 관리 도구

종류

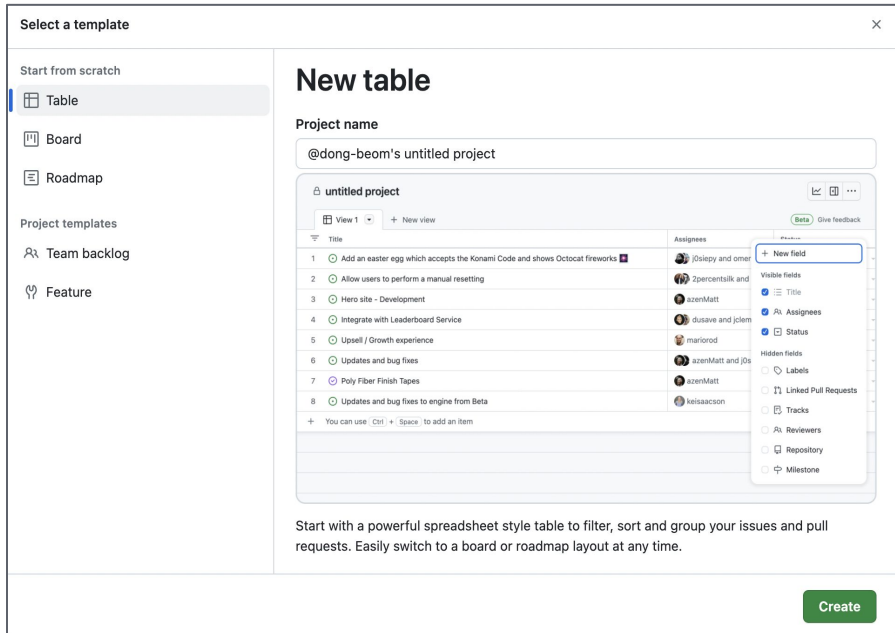
Table : 스프레드시트 스타일 레이아웃

Board : column 스타일 레이아웃

Roadmap : 간트 차트 형식의 로드맵 레이아웃

Team backlog : BackLog 기반의 칸반 보드

Feature : 기능 단위의 스프레드시트



☰

dong-beam

/ Projects

/ @dong-beam's untitled project

🔍

Search or jump to...

/

🔔

+

🚩

🔒 @dong-beam's untitled project

📊 View 1

+ New View

🔍

Filter by keyword or by field

Discard

Save

↩

	Title	...	Assignees	...	Status	...	+
1	🕒 Test #3						+ New field
2	🕒 test1 #2		🚩 dong-beam		Done		Visible fields
3	🕒 test #1		🚩 dong-beam		Todo		☒ 📄 Title ☒ 👤 Assignees ☒ 📋 Status
+ You can use Control + Space to add an item							
Hidden fields							
☐ 🏷 Labels ☐ 🔗 Linked pull requests ☐ 👤 Reviewers ☐ 📁 Repository ☐ 📌 Milestone							

Git Projects - Board

dong-beom / Projects / @dong-beom's demo project

@dong-beom's demo project

View 1

View 2

View 3

+ New View

Filter by keyword or by field

Todo 1

This item hasn't been started

git-study #6

SSO 연동 개발 자체

In Progress 1

This is actively being worked on

git-study #5

ERD 작성 및 문서화

Done 1

This has been completed

git-study #4

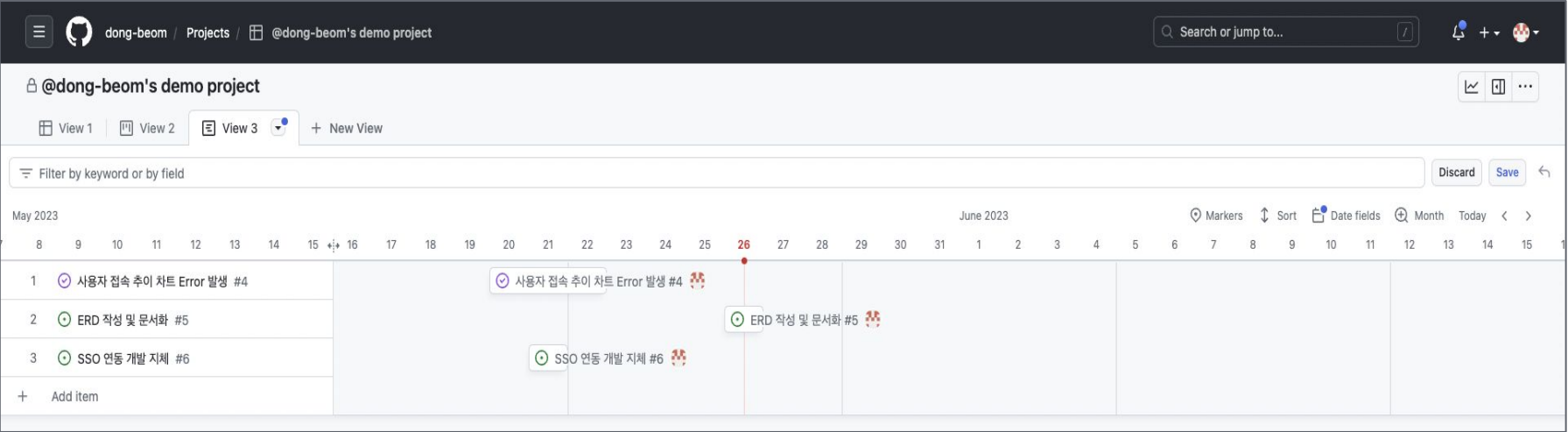
사용자 접속 추이 차트 Error 발생

+ Add item

+ Add item

+ Add item

Git Projects - Roadmap



Git Projects - View 종류별 장단점

Table

장점 : - 프로젝트의 요구사항, 작업 항목, 작업자, 우선순위 등을 명확하게

정의하고 관리할 수 있음.

단점 : - 큰 규모의 개발 프로젝트의 경우 복잡성이 증가함.

변경사항이 발생시 실시간 업데이트, 유지보수가 어려움

*Board

장점 : - 칸반보드는 작업을 시각적으로 표현하여 프로젝트의 전반적인

개요를 쉽게 파악가능함.

- 다양한 열(column)에 따라 작업의 상태를 표시하여

프로젝트의

진행을 한눈에 파악 가능함.

- 우선 순위와 작업 흐름을 관리하는데 유용함.

- 팀원들 간의 협업과 의사소통을 강화하는데 도움이 됩니다.

단점 : - Table과 마찬가지로 큰 규모의 프로젝트일 경우 복잡성이

높을 경우 다양한 열

(column)을

Roadmap

장점 : - 제품의 비전과 전략을 시각화하여 전달하는 데 용이함.

- 이해관계자들과의 커뮤니케이션에 유용함.

- 상위 경영진과 전략 공유에 용이함.

- 장기적인 목표에 대한 적절한 대응이 가능함.

단점 : - 시각화 도구이기 때문에 상세한 작업내용 표현에는 불리함.

- 주기적인 일정 관리를 하지 못하면 Roadmap이 정확하지

않거나

사용하지 않게됨.

시연

Iteration Planning 실습

1. 임의의 프로젝트 선정
2. 2주간의 #1 Iteration Planning을 진행한다. (자유 형식)

Git Projects - 이상적인 프로젝트 관리 방법

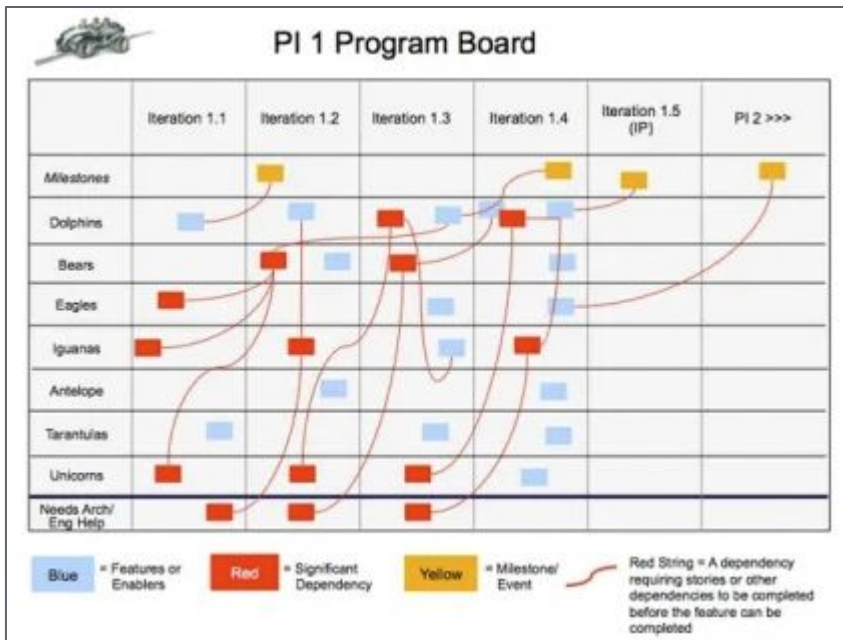
PI Planning

팀단위의 Roadmap을 구성하고 팀 간의 Dependency를

Check한다.

Dependency의 관계를 바탕으로 팀안에서 개발 우선순위를

조정한다.



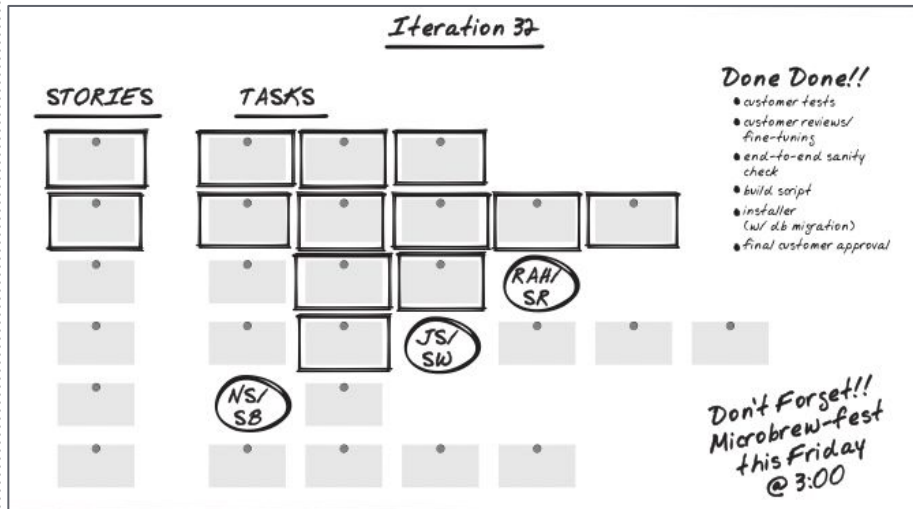
Git Projects - 이상적인 프로젝트 관리 방법

Iteration Planning

Roadmap을 바탕으로 각 스크럼별 백로그를 작성한다.

이터레이션 주기는 대부분 2주로 설정한다.

작성한 백로그는 스크럼별 칸반보드로 관리한다.



개인별 회고

Git Projects - Table 실전 예제

내용

SaaS BI Tool을 만드는 스크럼이라 가정

SaaS Tool 소개

사용자 트래킹 대시보드

마일스톤 작성

스크럼 Goal에 대한 마일스톤 작성

1. 사용자 추이 분석
2. 서비스 안정성 확보

이슈 작성

1. 사용자 접속 추이 차트 Error 발생
2. ERD 작성 및 문서화
3. SSO 연동 개발 지체

Assignees

이슈 담당자 지정

Status 설정

Todo, inprogress, done

이슈별 Label 설정

항목별로 적절한 Label 설정

마일스톤 매핑

이슈별 마일스톤 매핑

개선 사항 공유

스크럼에 예제 같은 이슈가 100개가 있다는 가정하에 개선하면
줄을 점 공유 - 5m

Git Projects - Table 실전 예제

코드 Push

*커밋 내역에 이슈 번호 첨부

Pull Request

해당 이슈 브랜치에서 main 브랜치 PR 요청

PR Review & Merge

코드 리뷰 후 Merge 진행

이슈 Close

이슈 Close 후 마일스톤, 이슈 상태 업데이트 확인

개선 사항

1. 해당 이슈의 완료 기준이 명확하지 않다.
2. main 브랜치의 merge가 자유롭다
3. 일일이 이슈의 설정값을 매칭하여 시간이 많이 소요된다.

Git Projects - Table 실습

남은 2개의 이슈를 배운 부분을 바탕으로 자유로운 형식으로 **Close**까지 처리 - 15m

Git Projects - Table 개선사항

해당 이슈의 완료 기준이 명확하지 않다.

Desc의 상세화가 필요

DoD(Definition of Done) 정의

main 브랜치의 merge가 자유롭다.

merge 정책필요

Ex) PR Default Reviewer 추가

Ex) Require approvals 설정

이슈 설정 정보 매핑 시간 단축

이슈 자동매핑 workflow 사용

Ex) PR 승인시 해당 이슈 status done 처리

Git Organization

Organization이란?

여러 사용자 및 프로젝트를 한 곳에서 중앙 집중적으로 관리할 수 있는 기능을 제공.

기능

- 팀 단위 멤버를 관리할 수 있음. 멤버별 권한과 역할을 할당할 수 있으며,

작업을 조직화할 수 있음.

- 멤버 및 팀에 대한 권한을 설정할 수 있어 보안과 접근 제어를 강화할 수

있으며, 프로젝트 공개여부를 관리하여 프로젝트에 필요 보안 수준을

유지할 수 있음.

- 이슈 트래커를 통해 버그 리포트, 기능 요청, 작업 항목 등을 관리할 수

있으며, 팀원들과의 토론과 작업 상태 추적을 위한 댓글 기능을

제공합니다.

Git Organization 실습

우리는 당신이 여기를 좋아할 것이라고 생각합니다.

조직의 개요에서 시작하는 데 도움이 되는 몇 가지 작업을 제안했습니다.

👤 당신의 사람들을 초대

첫 번째 회원 초대

GitHub 사용자 이름 또는 이메일 주소로 사람을 찾으세요.

구성원의 권한 사용자화

코드에 대한 모든 사람의 기본 권한을 설정합니다.

🔗 협업 코딩

[협업 코딩에 대해 자세히 알아보기 →](#)

풀 요청 만들기

리포지토리에 대한 변경 사항을 제안하고 협업합니다.

분기 보호 규칙 만들기

하나 이상의 브랜치에 대해 특정 워크플로를 적용합니다.

⚙️ 자동화 및 CI/CD

[자동화 및 CI/CD에 대해 자세히 알아보기 →](#)

새 문제 자동 할당

GitHub Actions로 작업을 자동으로 할당해 보세요.

지속적 통합 테스트 실행

CI 워크플로우를 사용하여 코드를 검증하십시오.

Organization 실습

Scrum 구성 : <https://docs.google.com/spreadsheets/>

Organization을 구성하여 Scrum 모의 운영을 한다.

Iteration Goal

개인정보 페이지 만들기

구성 완료 조건

1. 구성원 별 Rule 부여
2. Pull Request Action 1개 이상 생성
3. Branch 보호 규칙 1개 이상 생성
4. Actions를 사용하여 신규 Issue 자동 할당 규칙 1개 이상 생성
5. 코드 검증 Actions 1개 이상 생성

"인프라 가난" 시나리오 (비용 절감형)

핵심 제약: 스테이징 (Staging) 환경 없음, 테스트 서버 1대뿐.

- **상황 제시:** "우리 회사는 초기 스타트업이라 클라우드 비용 절감이 최우선입니다. 상용 (Production) 서버와 동일한 사양의 스테이징 서버가 없고, 오직 개발용 공용 서버 1대만 있습니다."
- **발생하는 문제:** 여러 명의 개발자가 각자 자기 브랜치를 테스트하고 싶은데, 서버는 1대라 누가 배포하면 앞선 사람의 테스트 내용이 덮어씌워집니다.
- **유도 질문:** * "서버가 하나뿐인 상황에서, 상용 배포 전 최종 검증을 위한 '배포 대기열'을 깃 브랜치로 어떻게 관리할 것인가?"
 - "로컬 환경과 상용 환경의 차이로 발생하는 버그를 잡기 위해 깃 태그(Tag)를 어떻게 활용할 것인가?"
 - **힌트:** **Integration** 브랜치를 두어 순서대로 머지하거나, **Docker**와 깃 브랜치를 연동해 임시 환경을 띄우는 전략 등을 고민하게 합니다.

2. "인력 불균형" 시나리오 (리뷰 병목형)

핵심 제약: 시니어 1명 vs 주니어 5명, 혹은 QA 조직 전무.

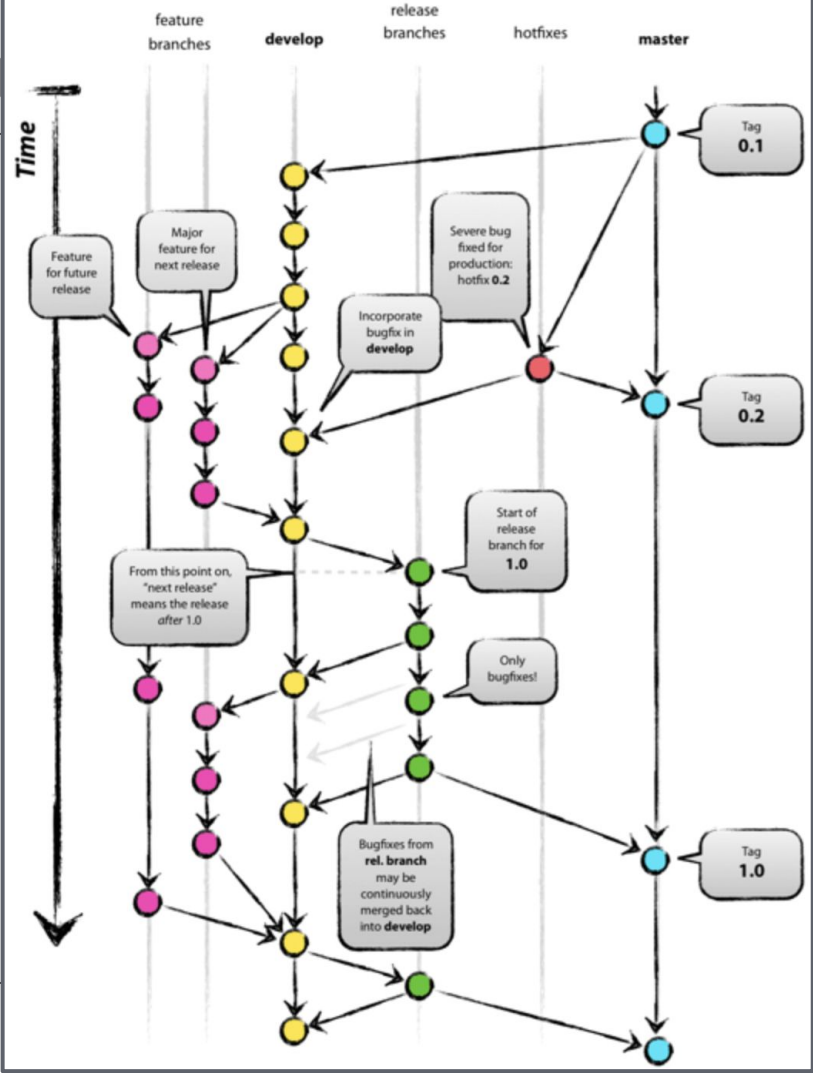
- **상황 제시:** "팀에 코드의 전체 맥락을 짚을 수 있는 시니어는 단 한 명뿐입니다. 주니어 5명이 쏟아내는 PR을 시니어가 다 봐줄 수 없어 배포가 계속 밀리고 있습니다. 게다가 QA 팀이 따로 없어 개발자가 직접 검수해야 합니다."
- **발생하는 문제:** 시니어의 승인(Approve)을 기다리느라 **develop** 브랜치에 코드가 쌓이고, 나중에 한꺼번에 머지하려니 충돌(Conflict) 지옥이 펼쳐집니다.
- **유도 질문:** * "시니어의 병목을 줄이기 위해 PR의 조건(Condition)을 어떻게 유연하게 가져갈 것인가? (예: UI 수정은 자율 머지, 핵심 로직은 필수 리뷰 등)"
 - "QA가 없는 상황에서 상용 서버의 안정성을 담보하기 위해 릴리스 브랜치(Release Branch)에서 어떤 활동을 강제할 것인가?"

3. "레거시 & 모놀리스" 시나리오 (공포의 머지형)

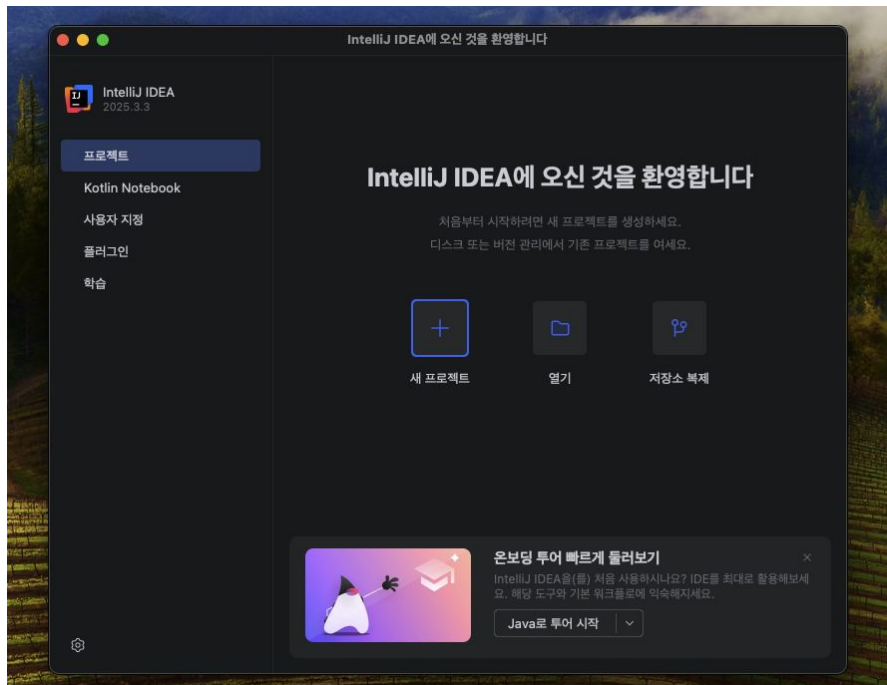
핵심 제약: 빌드 시간 30분, 한번 배포하면 되돌리기(Rollback) 매우 힘들.

- **상황 제시:** "서비스가 거대한 모놀리식 아키텍처라 한 번 빌드하고 배포하는 데 30분이 걸립니다. 게다가 DB 스키마 변경이 잦아 롤백이 거의 불가능에 가깝습니다."
- **발생하는 문제:** 작은 실수 하나가 전체 서비스 마비로 이어지며, 복구하는 데도 최소 30분이 소요됩니다.
- **유도 질문:** * "배포 후 사고를 방지하기 위해 **main** 브랜치 머지 직전에 어떤 '최종 방어선' 프로세스를 깃 전략에 녹일 것인가?"
 - "기능별로 배포 여부를 결정할 수 있도록 'Feature Toggle' 개념을 깃 브랜치와 어떻게 연계할 것인가?"

Git WorkFlow - 복기



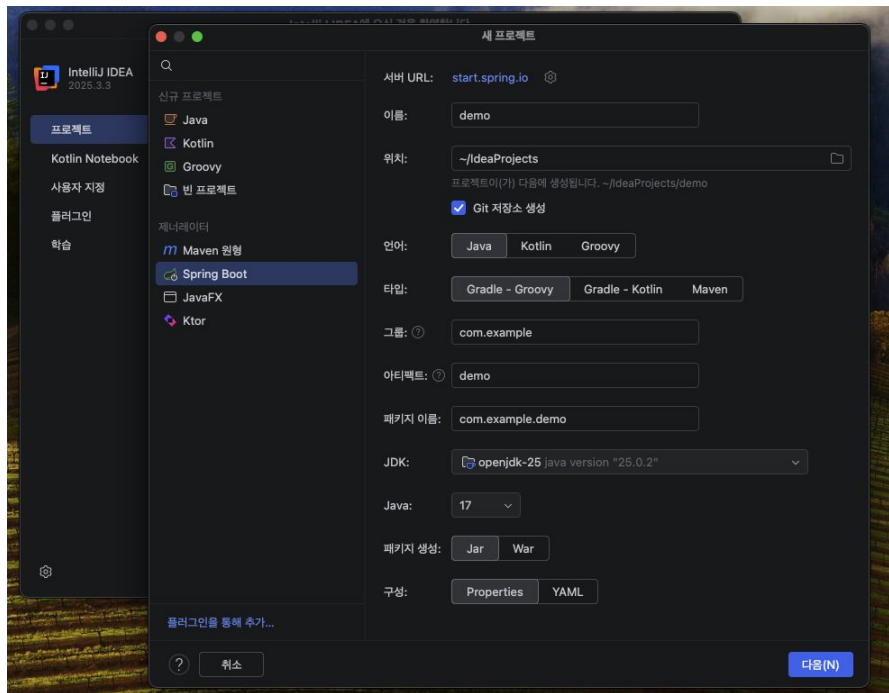
Git + IntelliJ 실습



IntelliJ 설치

IntelliJ 설치 - <https://www.jetbrains.com/idea/>

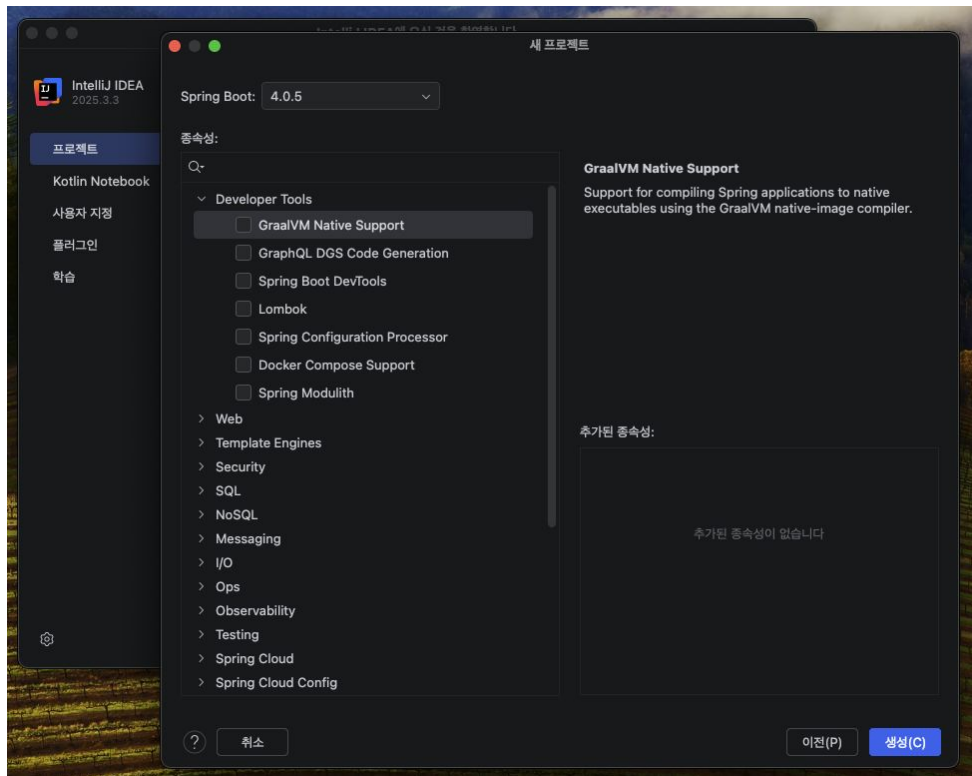
Git + IntelliJ 실습



IntelliJ 프로젝트 생성

새 프로젝트 생성 클릭 후 Spring Boot 프로젝트 생성
(Java 버전은 상관없음)

Git + IntelliJ 실습



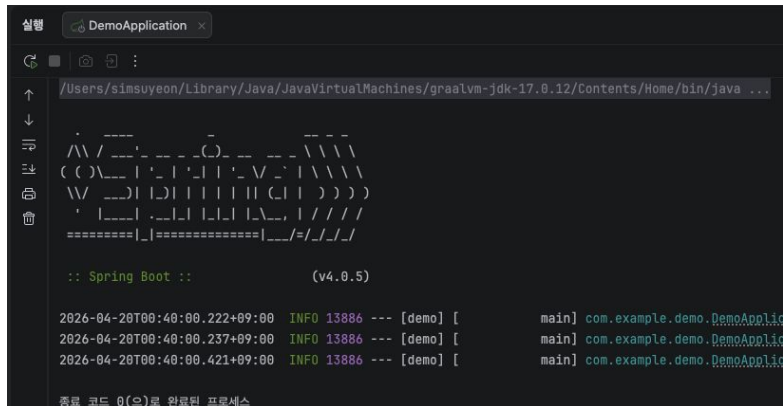
IntelliJ 프로젝트 생성

종속성 없이 생성 버튼 클릭

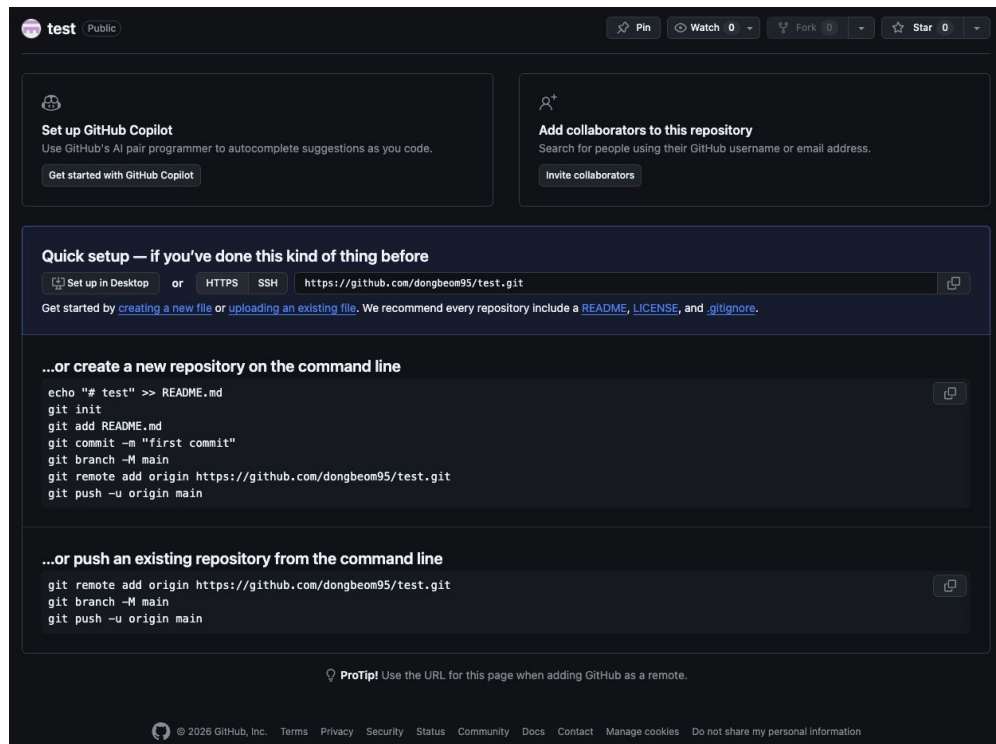
The screenshot displays an IDE interface with three main components:

- Project Structure (Left):** A tree view showing a project named 'demo' with subdirectories like 'idea', 'gradle', 'src', 'main', 'java', 'com.example.demo', 'resources', 'test', and 'build.gradle'.
- Code Editor (Top):** A file named 'HELP.md' is open, containing a 'Getting Started' section with links to 'Official Gradle documentation', 'Spring Boot Gradle Plugin Reference Guide', and 'Create an OCI image'. It also includes a 'Reference Documentation' section with a link to 'Gradle Build Scans'.
- Run Configuration (Bottom):** A dropdown menu is visible, showing options for running 'DemoApplication.main()' as a '실행' (Run) configuration, a '디버깅' (Debug) configuration, or a '커버리지로 실행' (Run with Coverage) configuration.

생성된 SpringBoot 프로젝트 실행 후 정상 동작 확인



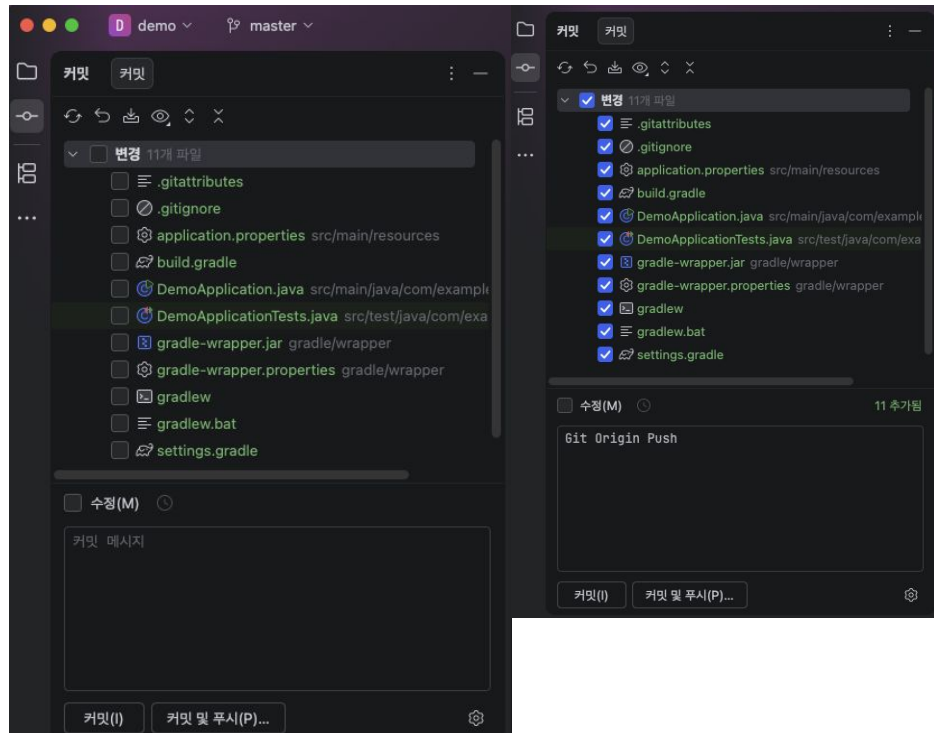
Git + IntelliJ 실습



GitHub Repository 생성

IntelliJ 프로젝트를 업로드할 Repo 생성

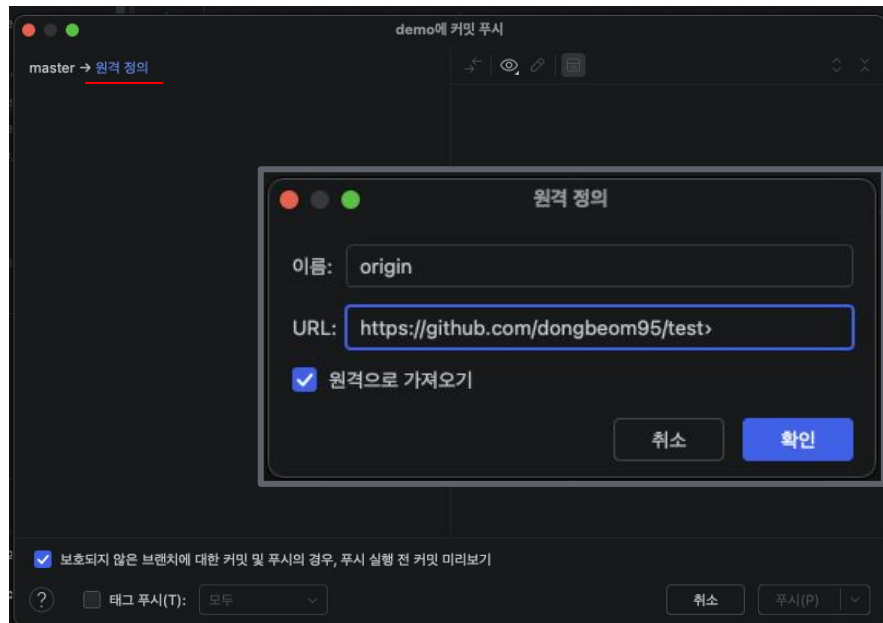
Git + IntelliJ 실습



IntelliJ 프로젝트 Git Push

왼쪽 위 카테고리에서 커밋 내역 확인 후 커밋 및 푸시 진행
추가된 모든 파일 선택

Git + IntelliJ 실습



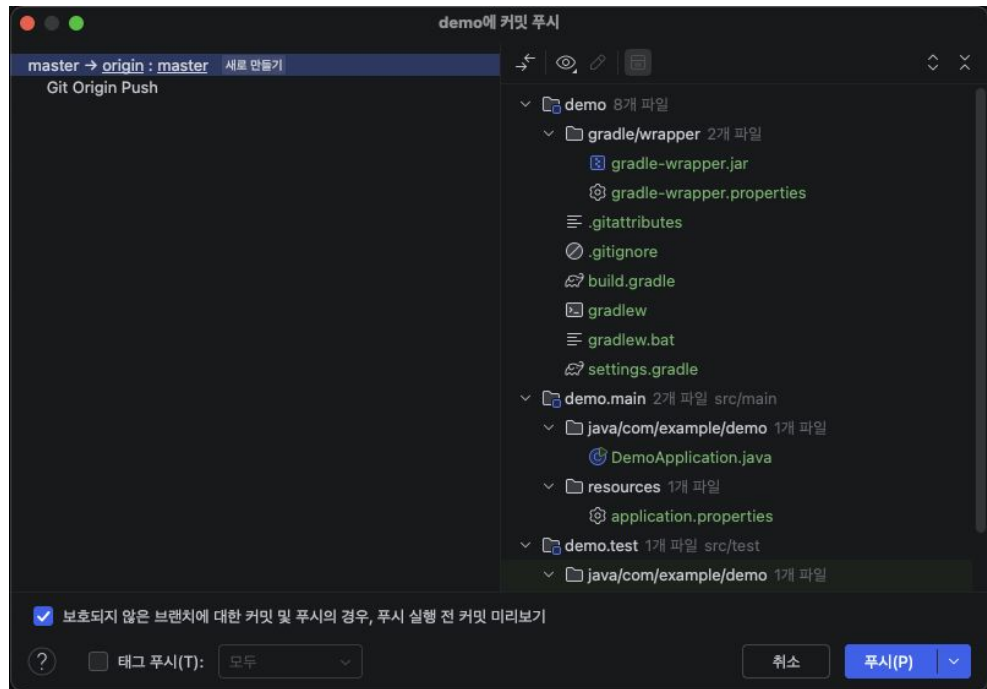
IntelliJ 프로젝트 Git Push

업로드할 GitHub 주소 정의 -> 원격 정의 클릭

(113 페이지에서 생성한 Repo Git 주소를 입력)

Github에 만든 레포지토리 주소 입력 후 확인 버튼 클릭

Git + IntelliJ 실습

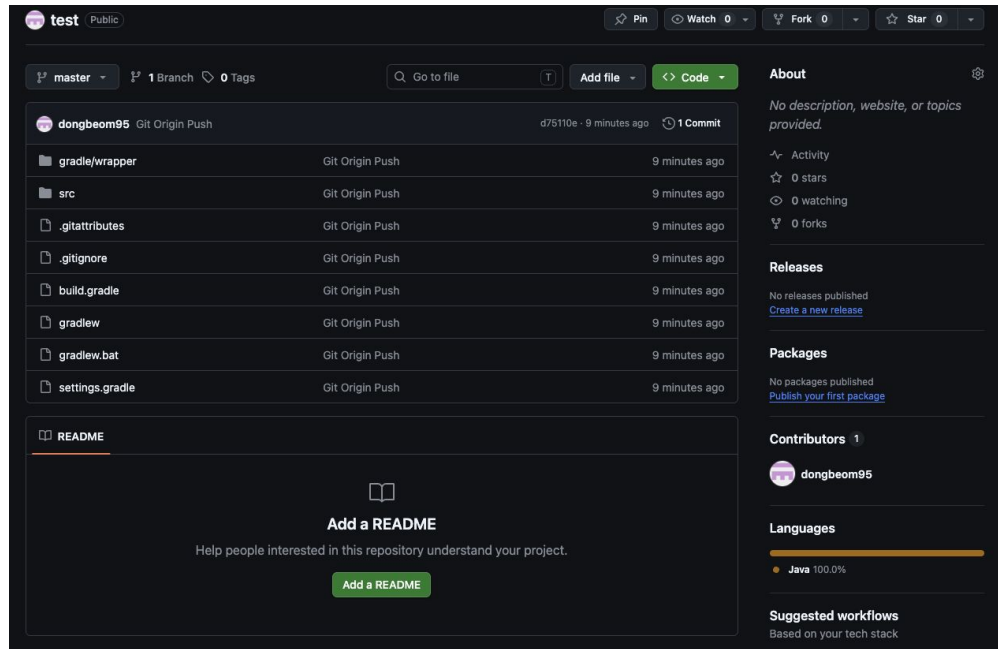


IntelliJ 프로젝트 Git Push

Git Origin 연동 완료 시 커밋 내역 표시

커밋 내역 정상 포함 확인 후 푸시

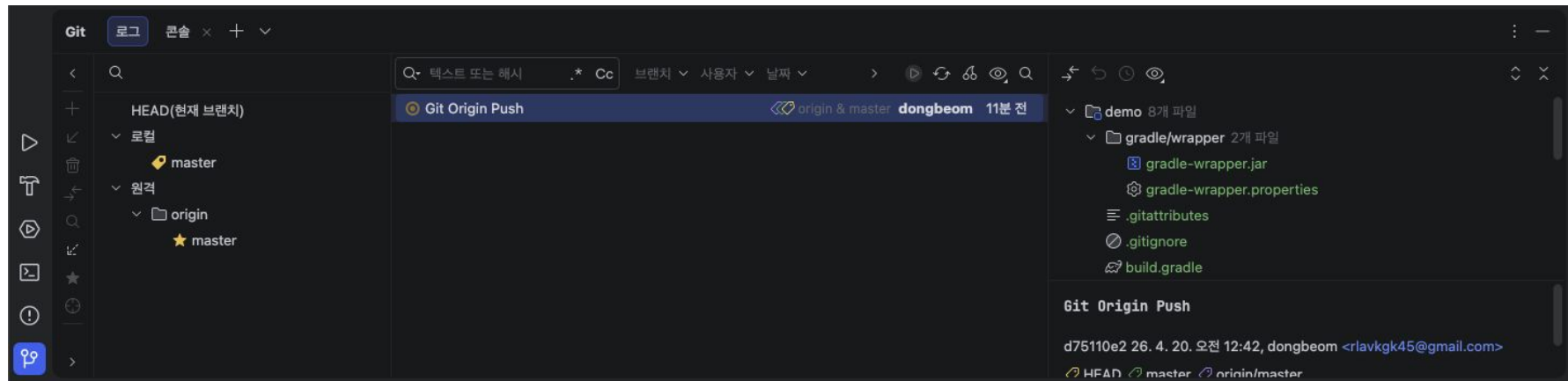
Git + IntelliJ 실습



GitHub Push 확인

IntelliJ 프로젝트 정상 푸쉬 확인

Git + IntelliJ 실습

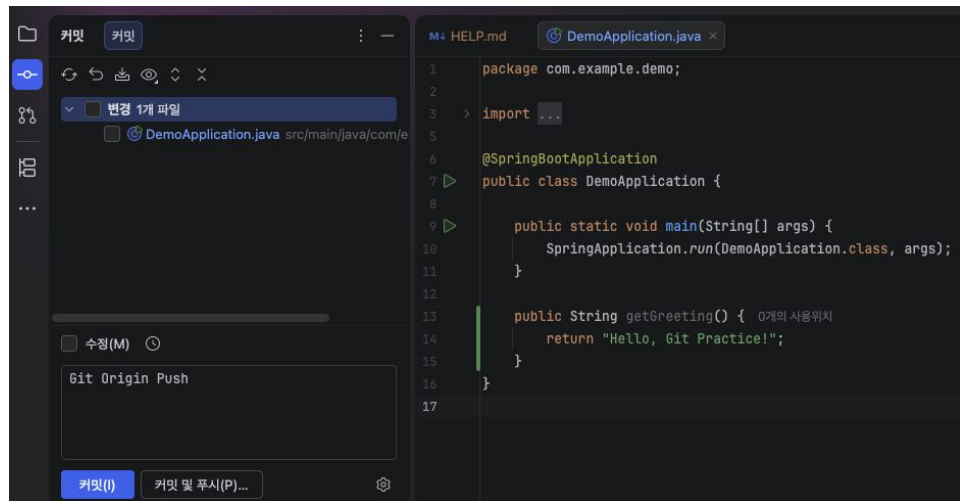


GitHub Log 확인

왼쪽 아래의 Git 아이콘 클릭

로컬, 원격 Log 확인

Git + IntelliJ Conflict 실습

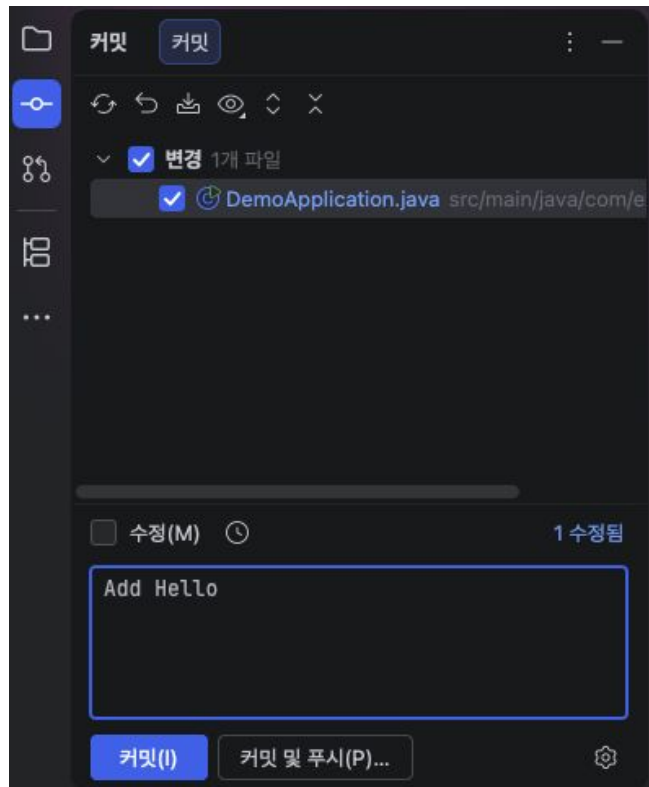


DemoApplication.java 메서드 추가

Main 브랜치에서 메서드 추가 후 커밋 푸쉬

```
public String getGreeting() {  
    return "Hello, Git Practice!";  
}
```

Git + IntelliJ Conflict 실습

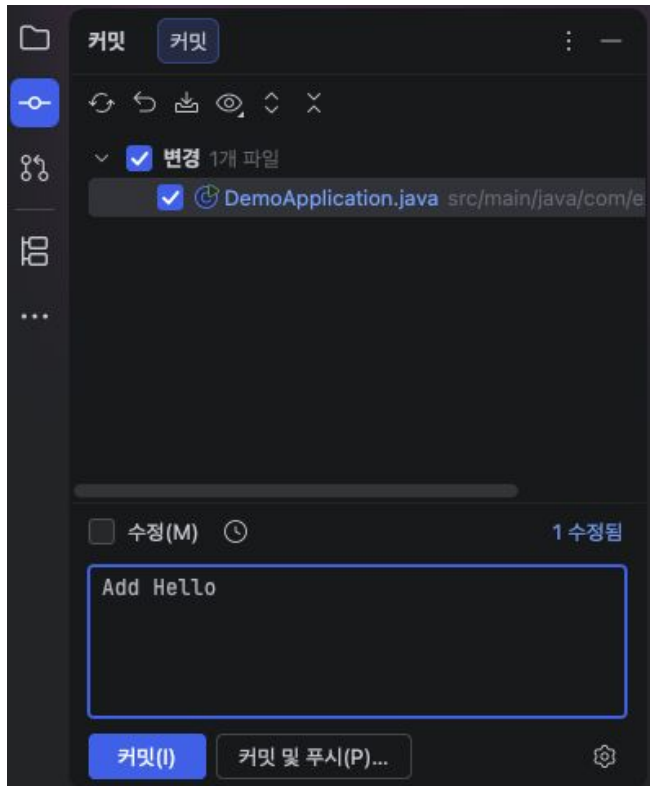


메서드 추가

변경사항 선택후 커밋 메세지 입력

커밋 푸쉬 진행

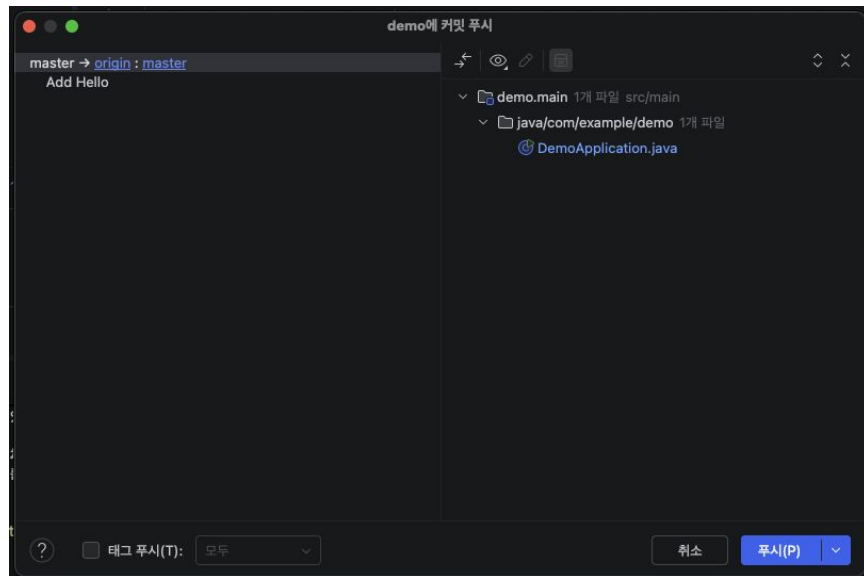
Git + IntelliJ Conflict 실습



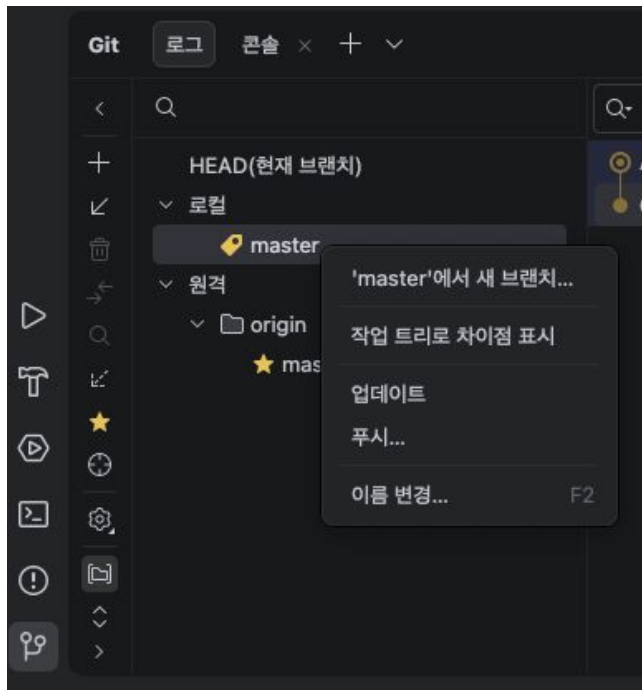
메서드 추가

변경사항 선택후 커밋 메세지 입력

커밋 푸쉬 진행 (master 브랜치)



Git + IntelliJ Conflict 실습



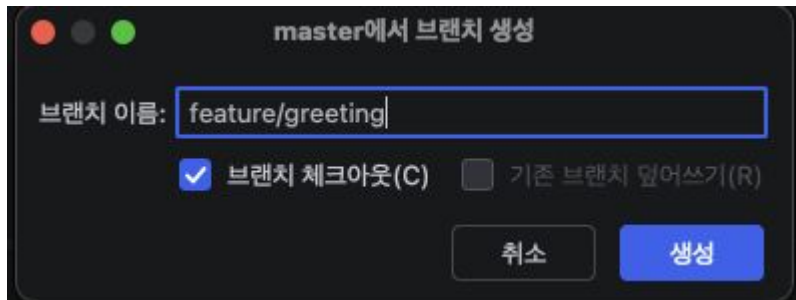
브랜치 생성

Master 브랜치에서 브랜치 분기

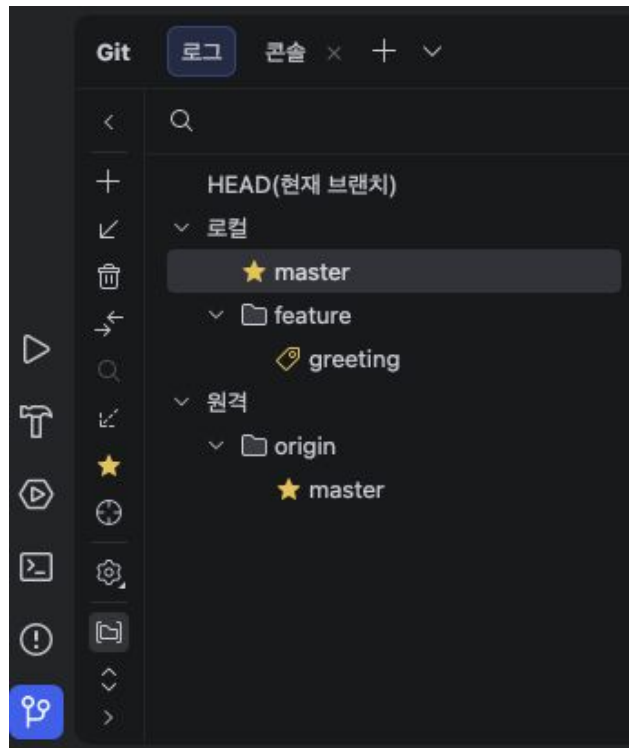
(원격 remote 브랜치가 아닌, 로컬 master 브랜치에서 분기)

로컬 브랜치 분기 이유는 원격에 확실하게 올라갈 브랜치인지 아직 불분명하기 때문

feature/greeting 브랜치 분기



Git + IntelliJ Conflict 실습

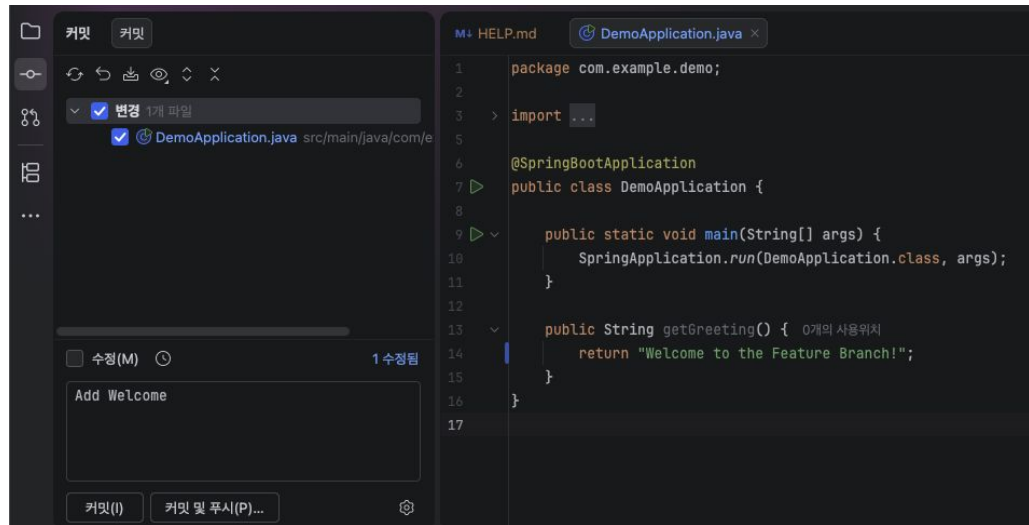


브랜치 확인

feature/greeting 브랜치 로컬 생성 확인

책갈피 표시는 현재 내가 위치한 브랜치를 표기함

Git + IntelliJ Conflict 실습

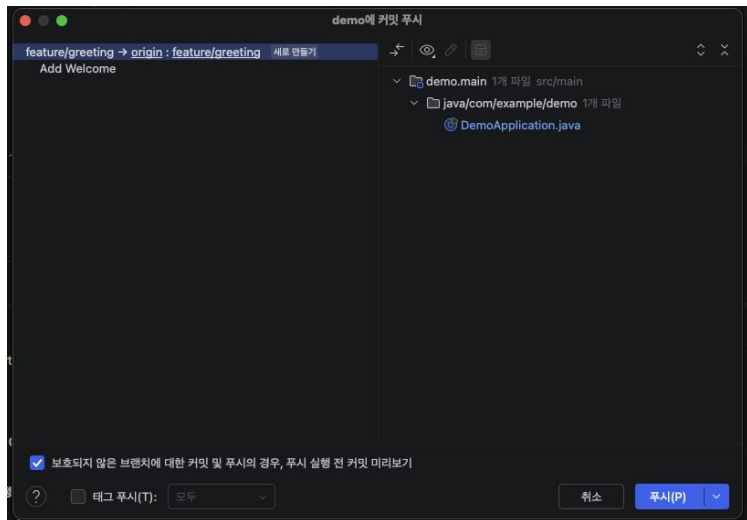


충돌 메서드 생성 (현재 브랜치 feature/greeting)

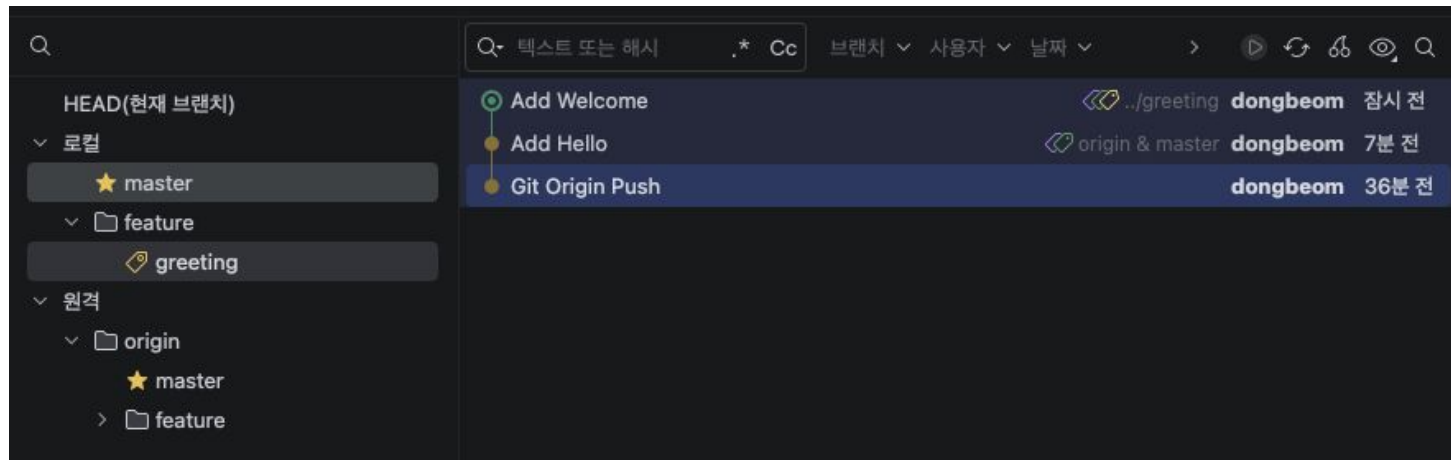
"Hello, Git Practice!" 이었던 return 값을

"Welcome to the Feature Branch!"

으로 변경 후 커밋 푸쉬 진행 (같은 메서드에 다른 내용)



Git + IntelliJ Conflict 실습



Git Log 확인

greeting 브랜치 클릭후 오른쪽 Git Log 확인

다른 색상(초록)의 feature/greeting 브랜치 커밋 푸쉬 내역 확인

Git + IntelliJ Conflict 실습

A. `feature/greeting` 브랜치에서 수정할 코드:

```
1 public String getGreeting() {  
2     return "Welcome to the Feature Branch!";  
3 }
```

B. `master` 브랜치에서 수정할 코드 (A와 다른 내용):

```
1 public String getGreeting() {  
2     return "Hello from the Main Master Branch!";  
3 }
```

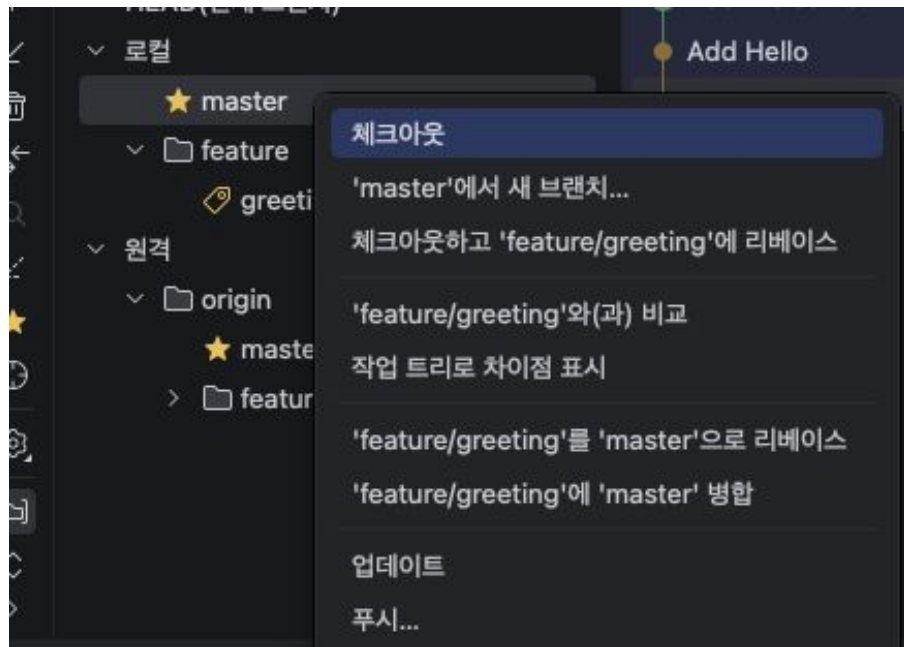
현재 상황

같은 `getGreeting` 메서드에서 서로 다른 내용을 푸쉬한 상태

`feature/greeting` -> Welcome to the Feature ...

`master` -> Hello from the Main ...

Git + IntelliJ Conflict 실습



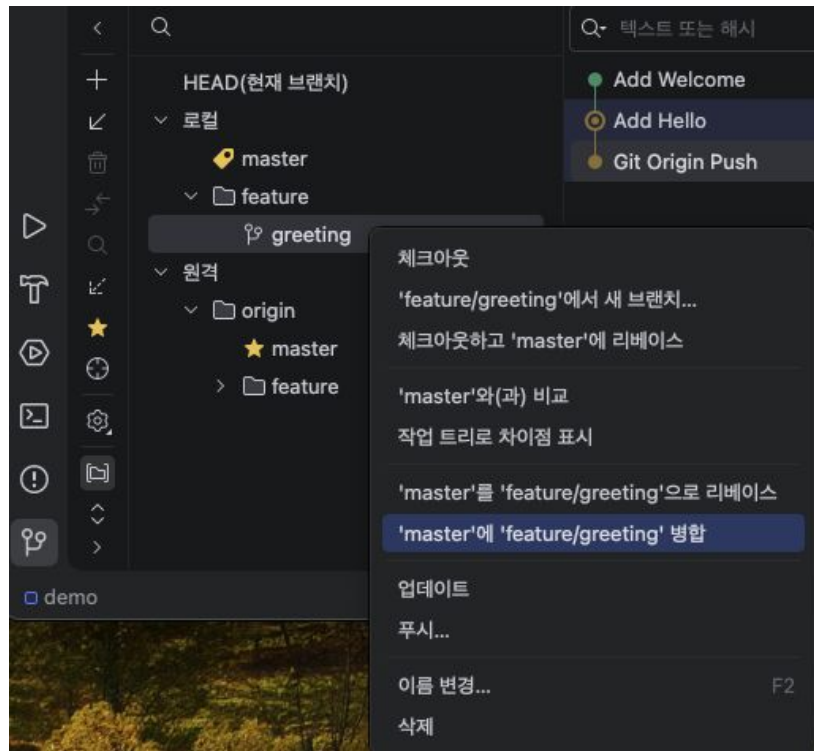
Merge 진행

Master 브랜치로 이동 (git checkout master)

머지를 당할 base 브랜치로 이동 후 머지해야됨

feature/greeting -> master 로 머지시 master 브랜치에서 머지 진행

Git + IntelliJ Conflict 실습

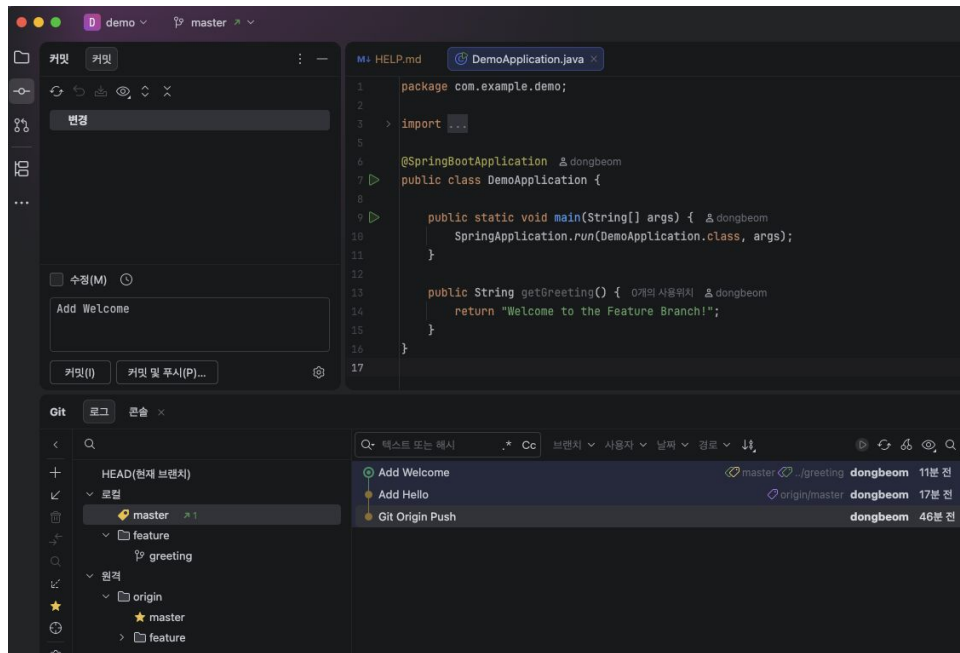


Merge 진행

master 브랜치 정상 이동 확인 (čekalpi 아이콘 이동 확인)

feature/greeting 브랜치의 오른쪽 마우스 클릭 후 Merge 진행

Git + IntelliJ Conflict 실습



Merge 후 상태

master 브랜치에서 feature/greeting 브랜치를 머지 한 상태

메서드 내용이 feature/greeting 내용으로 바뀌고 Conflict는 발생하지 않음

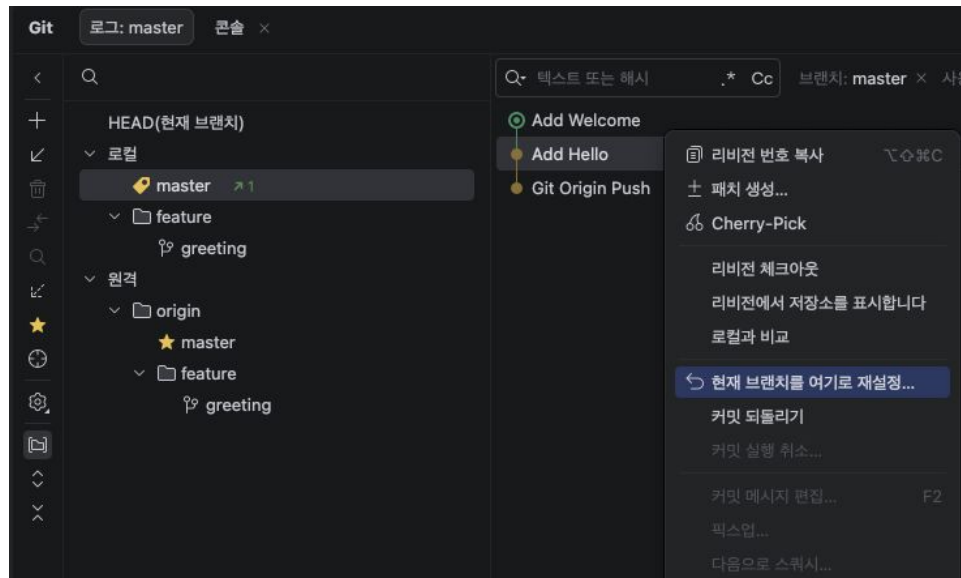
Conflict이 없는 상태로 머지완료 시 왼쪽 상단과 왼쪽 하단의 master 브랜치

업로드 표시가 나타남 1개의 푸시 대기가 있다는 의미

서로 같은 메서드의 다른 내용을 머지 했음에도 왜 발생하지 않았을까 ?

-> 고민 포인트

Git + IntelliJ Conflict 실습

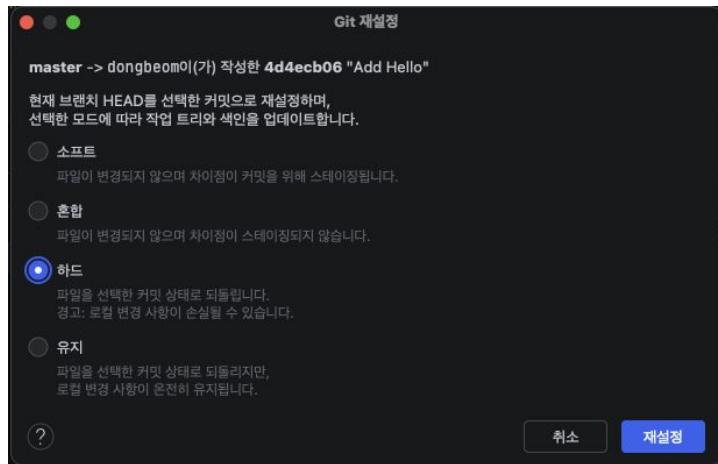


Merge 되돌리기

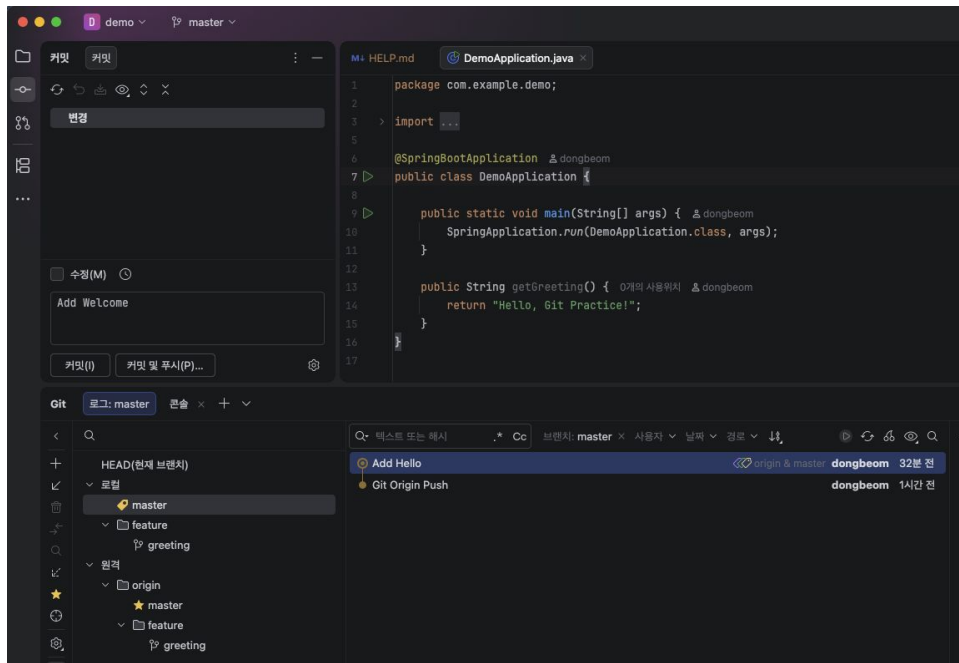
실수로 발생한 Merge 내역을 되돌리기

하드 옵션을 통한 로컬 머지 사항 되돌리기 재설정 실행

Fast-forward 머지가 되면 git merge --abort를 사용할 수 없음
(뒤에서 해당 내용 설명)



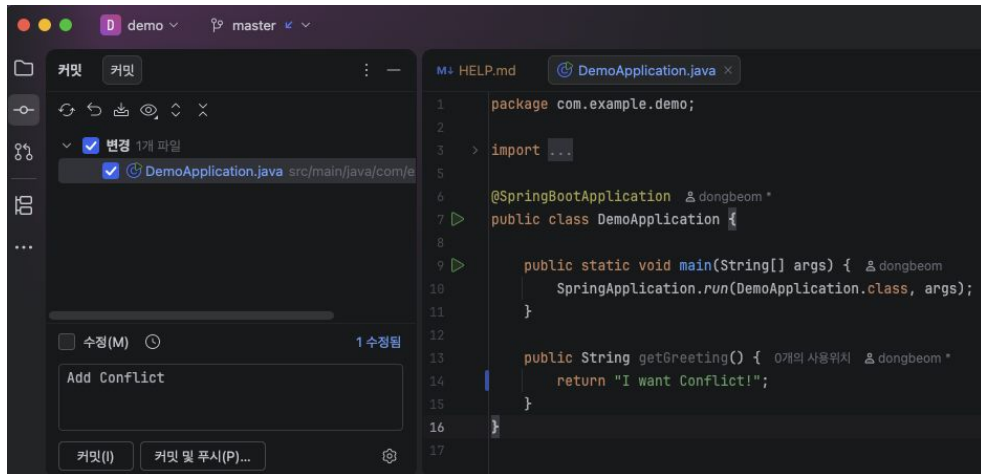
Git + IntelliJ Conflict 실습



Merge 되돌리기

Master 브랜치에 feature/greeting 로그가 없어지고
푸쉬 1 아이콘 삭제 (정상 되돌리기 완료)

Git + IntelliJ Conflict 실습



Master 브랜치 Conflict 발생 유도

Conflict가 발생하지 않은 이유는 Master 브랜치의 최신내역에서 feature/greeting 브랜치가 분기되어 작성되었기 때문

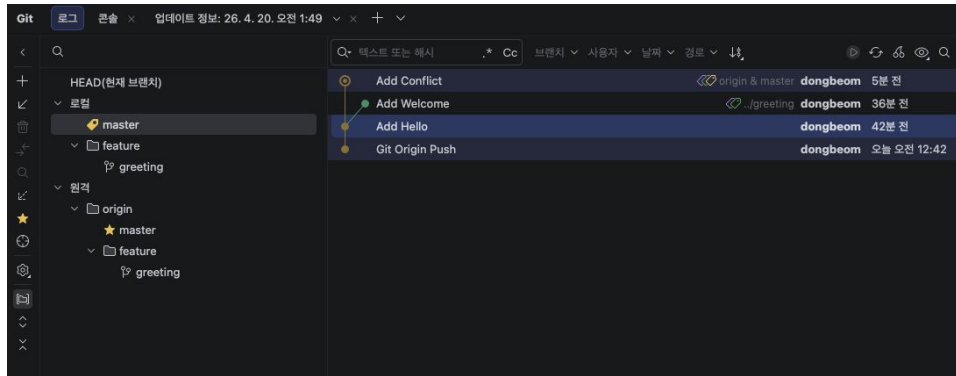
-> Conflict 발생을 위해서는 Main브랜치 (base) 도 분기를 해주어야함
Main 브랜치 메서드 변경 후 푸쉬 진행

```
"Hello, Git Practice!"
```

를 아래 로 변경 후 푸쉬 진행

```
I want Conflict!
```


Git + IntelliJ Conflict 실습

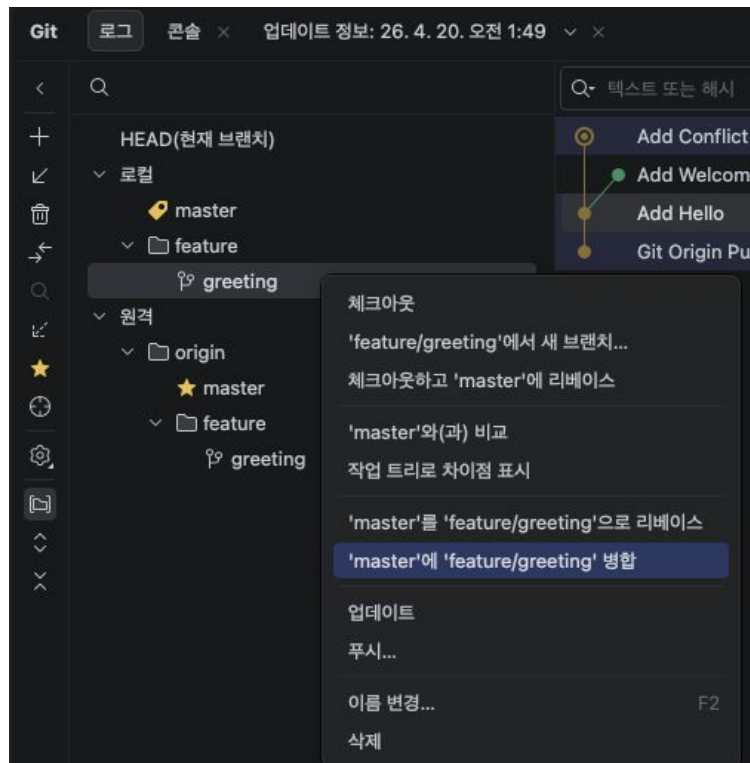


Master 브랜치 Conflict 발생 유도

Master 브랜치의 I want Conflict! 가 더 최신 커밋으로 반영됨.

(base 브랜치인 master와 feature/greeting의 내용이 서로 완벽히 다름)

Git + IntelliJ Conflict 실습

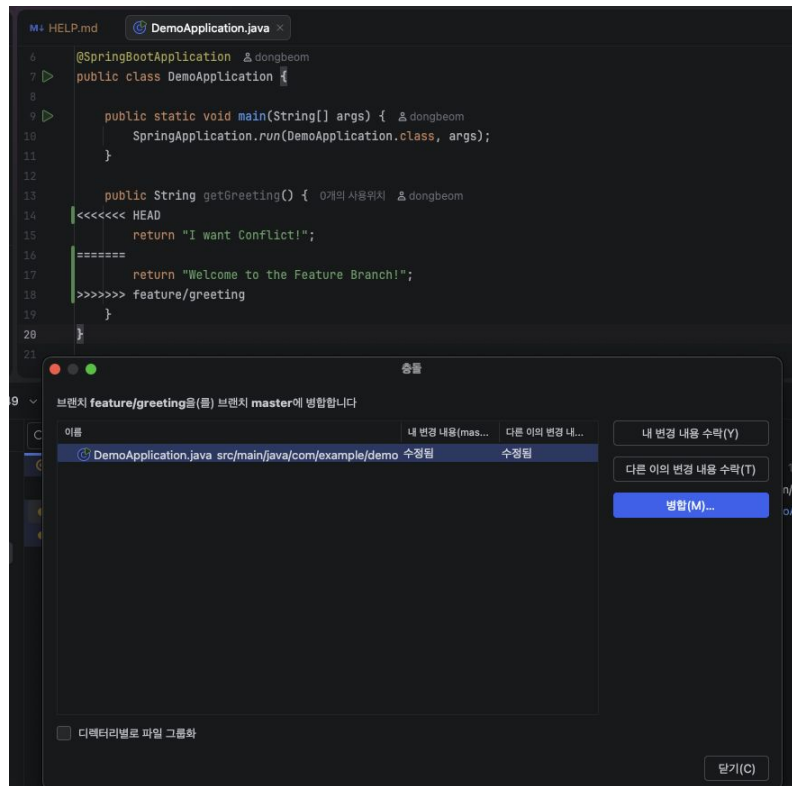


Merge 진행

다시 master 브랜치에 위치한 후 (책갈피 표시 확인)

feature/greeting 브랜치 병합 진행

Git + IntelliJ Conflict 실습

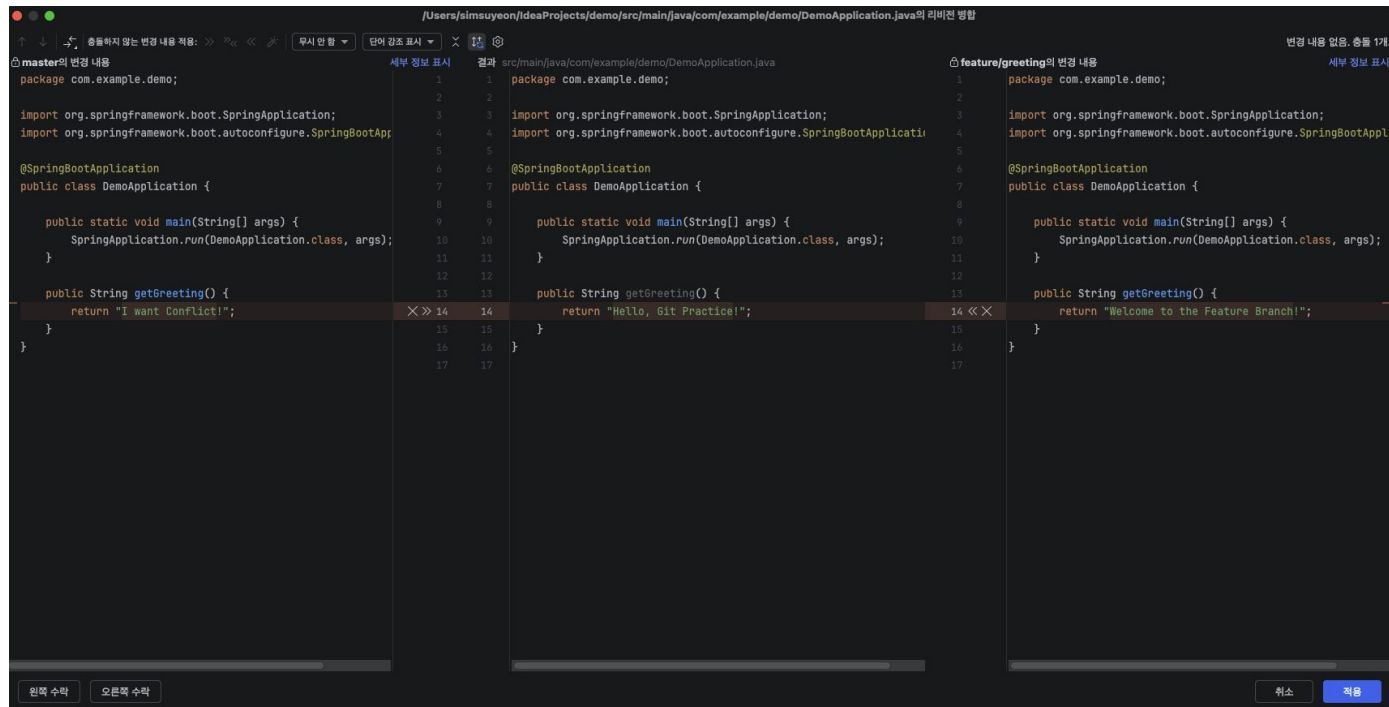


Conflict 발생

Conflict 팝업이 발생하며 Conflict 발생

충돌이 발생한 파일을 더블 클릭하여 충돌 내용 비교 진행

Git + IntelliJ Conflict 실습



Conflict 발생

중간 코드는 최종 완성 본

왼쪽은 master의 내용

오른쪽은 feature/greeting 내용

어느 브랜치의 내용을 합칠지 조정 후

적용

Git + IntelliJ Conflict 실습

<pre>11 11 } 12 12 13 13 public String getGreeting() { 14 14 return "I want Conflict!"; 15 15 } 16 16 } 17 17 }</pre>	<pre>11 11 } 12 12 13 13 public String getGreeting() { 14 14 return "Hello, Git Practice!"; 15 15 } 16 16 } 17 17 }</pre>	<pre>11 11 } 12 12 13 13 public String getGreeting() { 14 14 return "Welcome to the Feature Branch!"; 15 15 } 16 16 } 17 17 }</pre>
---	---	---

/Users/simsu/ideaProjects/demo/src/main/java/com/example/demo/DemoApplication.java의 리비전 병합

↑ ↓ ↺ 충돌하지 않는 변경 내용 적용: >> << 무시 안 함 단어 강조 표시

master의 변경 내용 세부 정보 표시 결과 src/main/java/com/example/demo/DemoApplication.java feature/greeting의 변경 내용 세부 정보 표시

package com.example.demo;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class DemoApplication {

public static void main(String[] args) {

SpringApplication.run(DemoApplication.class, args);

}

public String getGreeting() {

return "I want Conflict!";

}

}

package com.example.demo;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class DemoApplication {

public static void main(String[] args) {

SpringApplication.run(DemoApplication.class, args);

}

public String getGreeting() {

return "Welcome to the Feature Branch!";

}

}

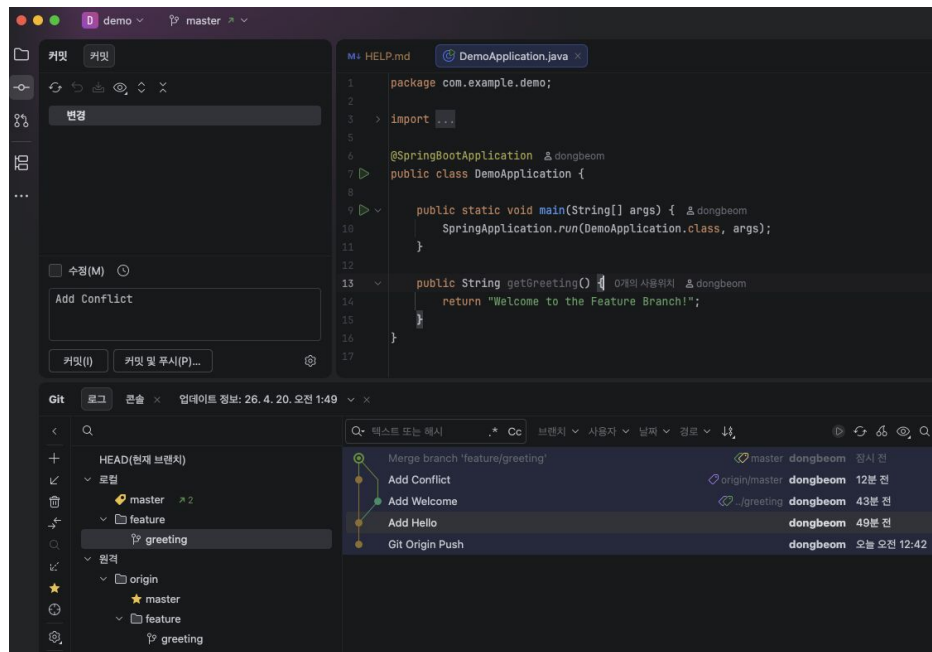
모든 충돌이 해결되었습니다.

Conflict 처리

I want Conflict가 아닌 (master 내용)

Welcome 내용으로 반영 진행 후 머지

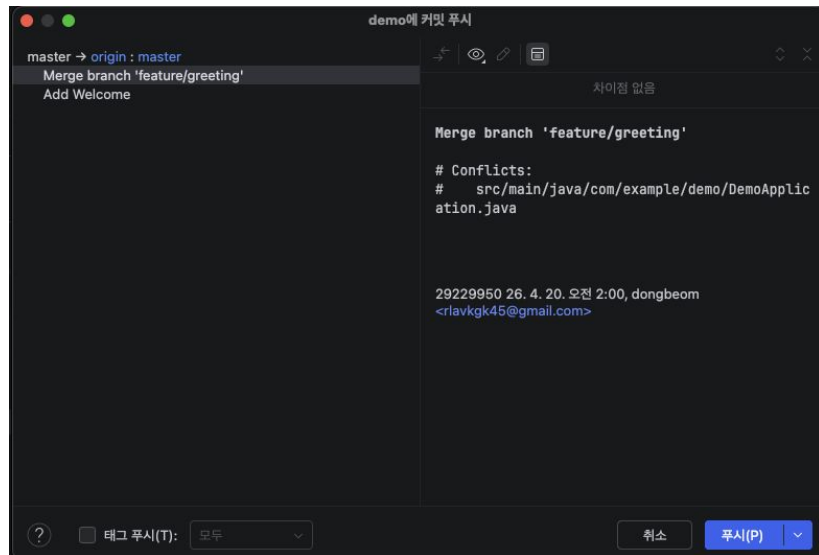
Git + IntelliJ Conflict 실습



Merge Push

정상 내용 반영 확인 후 푸쉬 진행

커밋 내역은 Conflict 처리로 자동 설정됨.



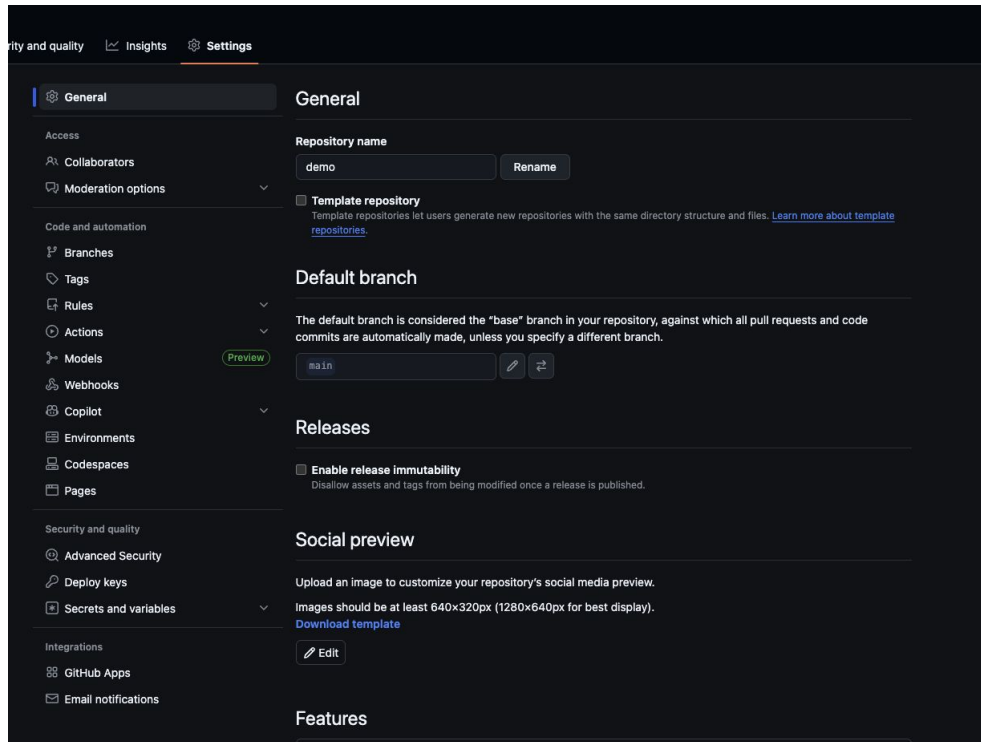
GitHub 정적 웹사이트 배포

```
<!DOCTYPE html>
<html>
<head>
  <title>비전공자 수업 데모</title>
  <style>
    body { font-family: sans-serif; text-align: center;
padding-top: 50px; }
    h1 { color: #2c3e50; }
    p { color: #7f8c8d; }
  </style>
</head>
<body>
  <h1>🎉 축하합니다! 배포에 성공하셨습니다!</h1>
  <p>CodeRabbit + Slack + GitHub Pages 조합을
완성하셨습니다.</p>
  <p>이 웹사이트는 전 세계 어디서든 이 URL로 접속할 수
있습니다.</p>
</body>
</html>
```

정적 웹사이트 index.html 생성

1. 정적 웹사이트 배포를 할 브랜치 생성 (이름 상관 없음)
2. 왼쪽의 index.html 파일 루트 경로 생성 후 푸쉬

GitHub 정적 웹사이트 배포



Settings -> Pages

레포지토리에서 **Pages** 설정

GitHub 정적 웹사이트 배포

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

Security and quality

Advanced Security

Deploy keys

Secrets and variables

Integrations

Git-Hub Apps

Email notifications

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Build and deployment

Source

Deploy from a branch

Branch

Your GitHub Pages site is currently being built from the `feature/ai-test` branch. [Learn more about configuring the publishing source for your site.](#)

feature/ai-test / (root) Save

Learn how to [add a Jekyll theme](#) to your site.

Custom domain

Custom domains allow you to serve your site from a domain other than `dongbeom95.github.io`. [Learn more about configuring custom domains.](#)

Save Remove

☒ **Enforce HTTPS**

— Required for your site because you are using the default domain (`dongbeom95.github.io`)

HTTPS provides a layer of encryption that prevents others from snooping on or tampering with traffic to your site. When HTTPS is enforced, your site will only be served over HTTPS. [Learn more about securing your GitHub Pages site with HTTPS.](#)

Visibility [GitHub Enterprise](#)

With a GitHub Enterprise account, you can restrict access to your GitHub Pages site by publishing it privately. You can use privately published sites to share your internal documentation or knowledge base with members of your enterprise. You can try GitHub Enterprise risk-free for 30 days. [Learn more about the visibility of your GitHub Pages site.](#)

Start free for 30 days

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

웹사이트 배포

정적 웹사이트를 **Push**한 브랜치 선택 후 **Save** 클릭

GitHub 정적 웹사이트 배포

GitHub Pages

[GitHub Pages](#) is designed to host your personal, organization, or project pages from a GitHub repository.

Your site is live at <https://dongbeom95.github.io/demo/>

Last [deployed](#) by  [dongbeom95](#) 2 minutes ago

 Visit site

Unpublish site

배포 확인

1 분 뒤 사이트 URL 생성 후 확인

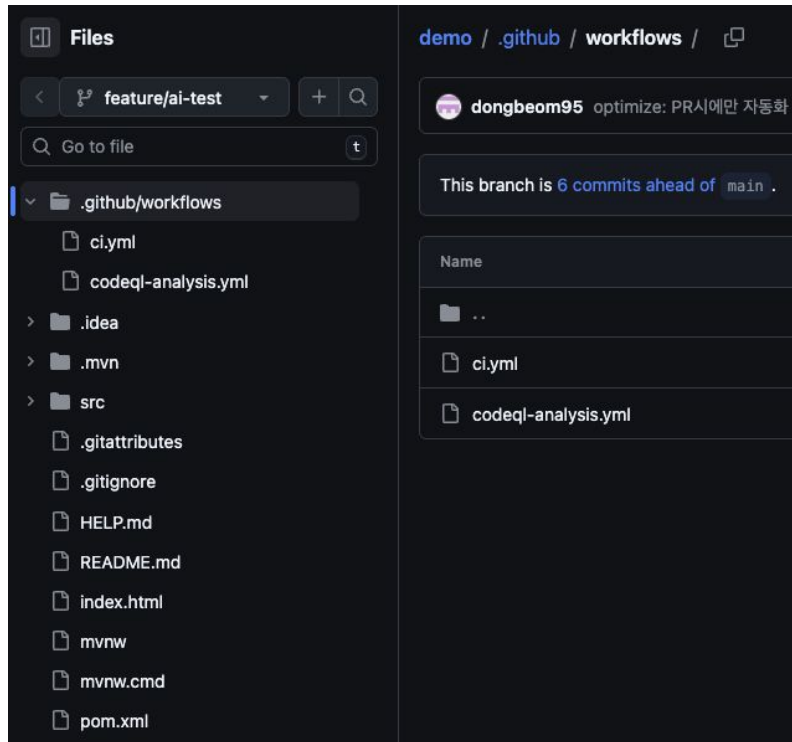


축하합니다! 배포에 성공하셨습니다!

CodeRabbit + Slack + GitHub Pages 조합을 완성하셨습니다.

이 웹사이트는 전 세계 어디서든 이 URL로 접속할 수 있습니다.

GitHub Actions



Git Actions 란 ?

Git의 특정 시점에 특정 동작을 실행시키는 기능 (트리거)

코드를 올리거나 합치는 특정 이벤트가 발생하면, 미리 정해둔 작업을 자동으로 실행

Git Actions 추가 방법

프로젝트 안에 `.github/workflows` 폴더 생성

`.yml` 확장자의 설정 파일로 작업 정의

GitHub Actions - ci.yml

```
name: Professional CI
on:
  pull_request:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven
      - name: Build with Maven
        run: chmod +x mvnw && ./mvnw -B package --file pom.xml
```

ci.yml 생성

코드가 실행 가능한 상태인지 '빌드'를 우선 시도

내 로컬에서는 되는데?를 방지하기 위함

표준 환경에서의 검증

GitHub Actions - codeql-analysis.yml

name: "CodeQL Security Scan"

on:

pull_request:

branches: ["main"]

jobs:

analyze:

name: Analyze Code

runs-on: ubuntu-latest

permissions:

actions: read

contents: read

security-events: write

strategy:

fail-fast: false

matrix:

language: ['java']

steps:

- name: Checkout repository

uses: actions/checkout@v4

- name: Initialize CodeQL

uses: github/codeql-action/init@v3

with:

languages: \${{ matrix.language }}

- name: Autobuild

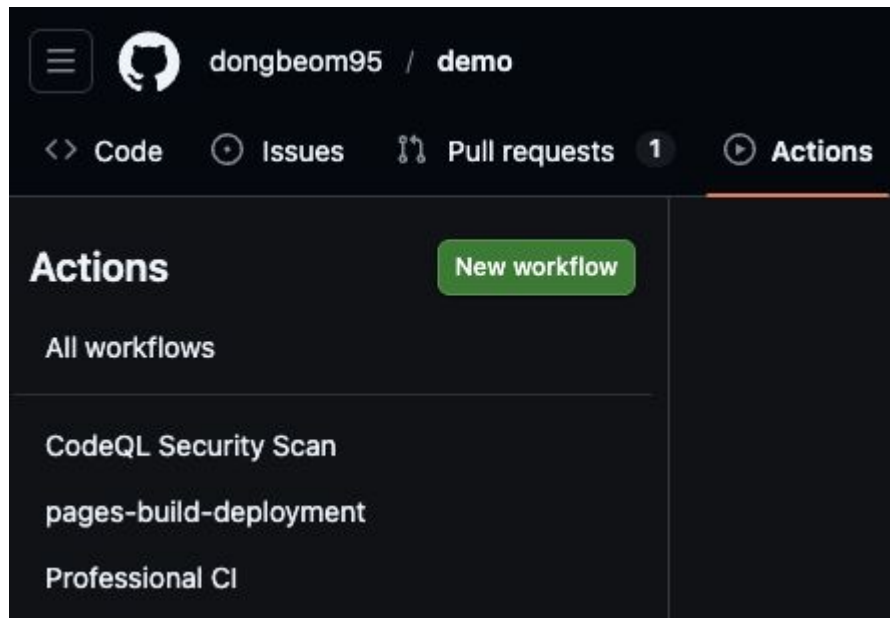
uses: github/codeql-action/autobuild@v3

codeql-analysis.yml 생성

GitHub에서 보장하는 codeql을 사용한 보안 감사

코드에 대한 해킹 위험을 정밀 진단하는 역할

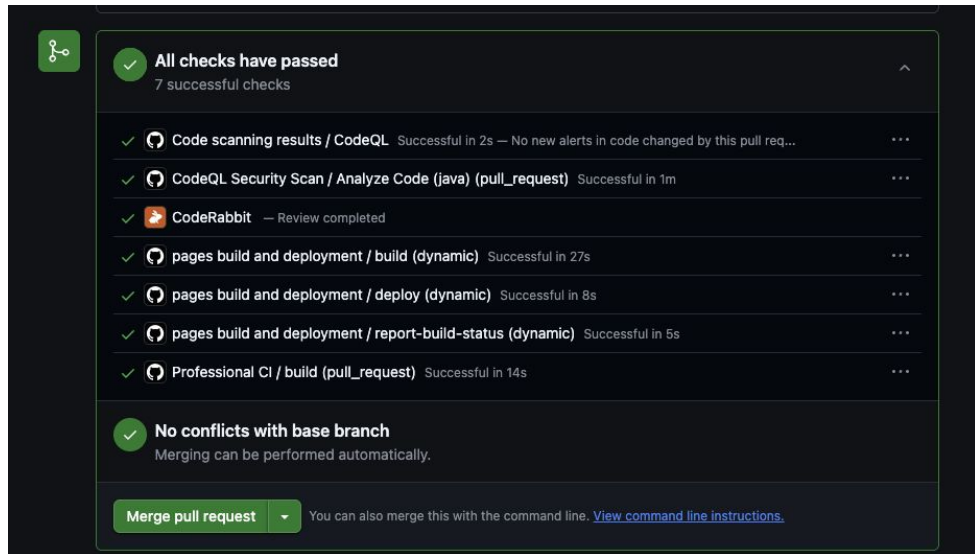
GitHub Actions



Actions 등록 확인

해당 레포지토리의 Actions 탭을 통한 별개 실행 가능

GitHub Actions



PR을 통한 Actions 동작 확인

정의한 Actions에 대한 Step별 실행 후 통과시 Merge 승인

GitHub io

Github io 란?

GitHub에서 제공하는 개인용 웹사이트 호스팅 서비스

복잡한 서버 구입이나 결제 없이, 평생 무료로 웹사이트 운영 가능

ID.github.io

GitHub 특정 사람의 공식 대문 사이트구나! 라는 인식 목적의 URL 부여

해당 저장소에 `index.html` 파일을 업로드시, 전 세계 사람들이 접속 가능한

주소 생성

다른 주소와 다른 점

블로그 주소는 남의 땅을 빌려서 글만 올리는 것

개인 도메인은 매년 돈을 내고 주소를 유지해야 하는 것

GitHub.io 무료이면서, Git의 활용을 어필 할 수 있음

GitHub Io - 이력서 배포하기

이력서 템플릿 Fork

<https://github.com/academicpages/academicpages.github.io>

위 Git을 Fork (다른 템플릿도 상관없음)

Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project. [View existing forks.](#)

Required fields are marked with an asterisk (*).

Owner *

 dongbeom95

Repository name *

 dongbeom95.github.io is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description

90 / 350 characters

☒ Copy the `master` branch only

Contribute back to academicpages/academicpages.github.io by adding your own branch. [Learn more.](#)

 You are creating a fork in your personal account.

Create fork

레포지토리 이름은 {githubId}.github.io로 설정

GitHub Io - 이력서 배포하기

GitHub Pages

[GitHub Pages](#) is designed to host your personal, organization, or project pages from a GitHub repository.

Build and deployment

Source

Deploy from a branch ▾

Branch

Make a commit to the master branch to publish your GitHub Pages site. [Learn more about configuring the publishing source for your site.](#)

 master ▾

 / (root) ▾

Save

배포 브랜치 설정

master로 설정 후 Save

GitHub Io - 이력서 배포하기

The screenshot shows the GitHub Actions page for a repository named 'dongbeom95'. The left sidebar contains the 'Actions' tab with a 'New workflow' button. Below it, a list of workflows is shown, including 'pages-build-deployment'. The main area displays 'All workflows' with a search bar. A warning message states: 'Workflows aren't being run on this forked repository. Because this repository contained workflow files when it was forked, we have disabled them from running on this fork. Make sure you understand the configured workflows and their expected usage before enabling Actions on this repository. Some GitHub features that depend on Actions may still be able to run. View the workflows directory'. A green button below the message says 'I understand my workflows, go ahead and enable them'. Below the warning, a table shows '1 workflow run' for the 'pages build and deployment' workflow, triggered by the 'pages-build-deployment #1' event, running on the 'master' branch, completed 1 minute ago, and taking 47s.

배포 확인

Fork한 레포지토리의 Actions 탭에서 배포 확인

GitHub Io - 이력서 배포하기

GitHub Pages

[GitHub Pages](#) is designed to host your personal, organization, or project pages from a GitHub repository.

Build and deployment

Source

Deploy from a branch ▾

Branch

Make a commit to the master branch to publish your GitHub Pages site. [Learn more about configuring the publishing source for your site.](#)

 master ▾

 / (root) ▾

Save

배포 브랜치 설정

master로 설정 후 Save

GitHub Io - 이력서 배포하기

The screenshot shows the GitHub repository page for the 'Academic Pages' template. The repository is named 'donbeom95.github.io'. The page features a dark theme and includes a sidebar with a profile picture placeholder, a 'Your Sidebar Name' section, and a list of social links (Earth, Red Brick University, Email, Google Scholar, ORCID, PubMed, GitHub, Bluesky). The main content area has a title 'Academic Pages is a ready-to-fork GitHub Pages template for academic personal websites' and a detailed description of the template's features and usage. It also includes a section for 'A data-driven personal website' and a 'Getting started' section with two steps: 1. Register a GitHub account and 2. Fork the template.

donbeom95.github.io

Your Name / Site Title

Publications Talks Teaching Portfolio Blog Posts CV Guide

Academic Pages is a ready-to-fork GitHub Pages template for academic personal websites

This is the front page of a website that is powered by the [Academic Pages template](#) and hosted on GitHub pages. [GitHub pages](#) is a free service in which websites are built and hosted from code and data stored in a GitHub repository, automatically updating when a new commit is made to the repository. This template was forked from the [Minimal Mistakes Jekyll Theme](#) created by Michael Rose, and then extended to support the kinds of content that academics have: publications, talks, teaching, a portfolio, blog posts, and a dynamically-generated CV. Incidentally, these same features make it a great template for anyone that needs to show off a professional template!

You can fork [this template](#) right now, modify the configuration and Markdown files, add your own PDFs and other content, and have your own site for free, with no ads!

A data-driven personal website

Like many other Jekyll-based GitHub Pages templates, Academic Pages makes you separate the website's content from its form. The content & metadata of your website are in structured Markdown files, while various other files constitute the theme, specifying how to transform that content & metadata into HTML pages. You keep these various Markdown (.md), YAML (.yml), HTML, and CSS files in a public GitHub repository. Each time you commit and push an update to the repository, the [GitHub pages](#) service creates static HTML pages based on these files, which are hosted on GitHub's servers free of charge.

Many of the features of dynamic content management systems (like Wordpress) can be achieved in this fashion, using a fraction of the computational resources and with far less vulnerability to hacking and DDoSing. You can also modify the theme to your heart's content without touching the content of your site. If you get to a point where you've broken something in Jekyll/HTML/CSS beyond repair, your Markdown files describing your talks, publications, etc. are safe. You can rollback the changes or even delete the repository and start over - just be sure to save the Markdown files! You can also write scripts that process the structured data on the site, such as [this one](#) that analyzes metadata in pages about talks to display [a map of every location you've given a talk](#).

For those users that need more advanced functionality, the template also supports the following popular tools:

- [MathJax](#) for mathematical equations
- [Mermaid](#) for diagramming
- [Plotly](#) for plotting

Getting started

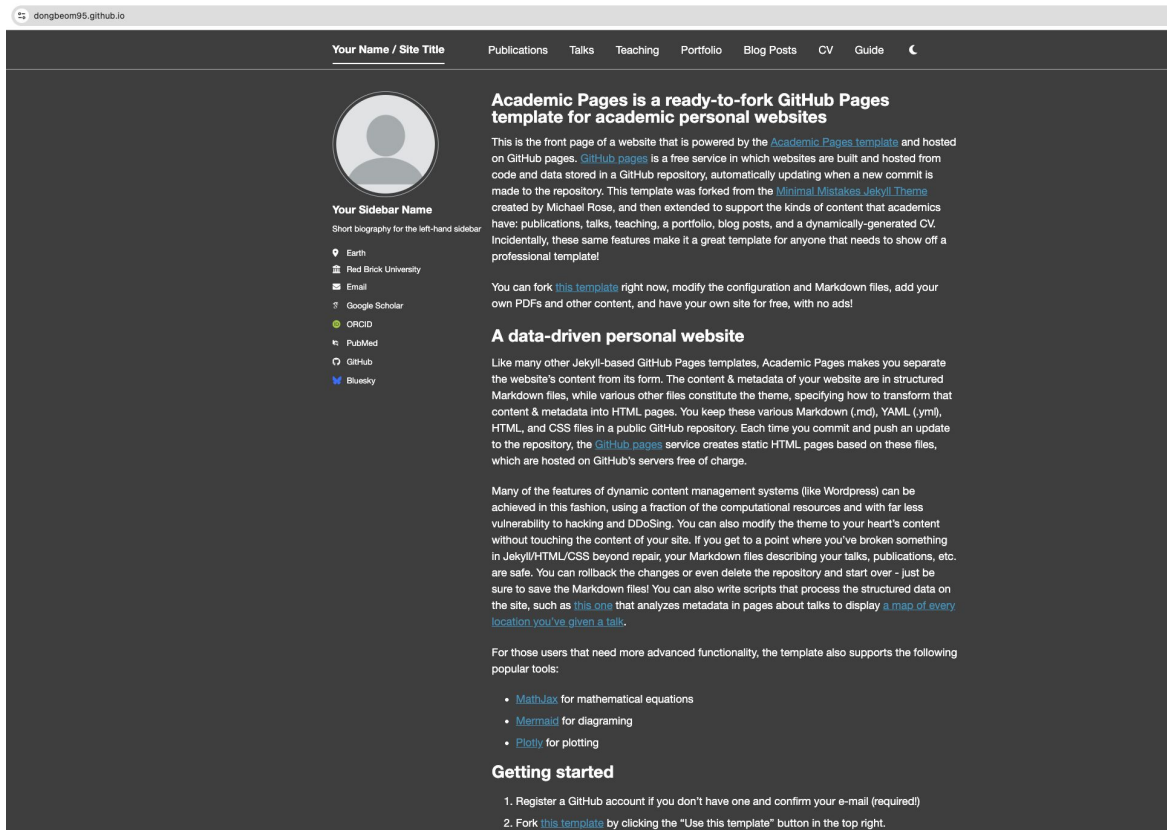
1. Register a GitHub account if you don't have one and confirm your e-mail (required)
2. Fork [this template](#) by clicking the "Use this template" button in the top right.

이력서 URL 접속

donbeom95.github.io 정상 접속 확인

템플릿을 기반으로 소스를 수정하여 나만의 이력서로 변경

GitHub Io - 이력서 배포하기



이력서 URL 접속

dongbeom95.github.io 정상 접속 확인

템플릿을 기반으로 소스를 수정하여 나만의 이력서로 변경

Ex) <https://hyunseob.github.io/resume/>

Code Rabbit

Code Rabbit이란?

GitHub에 상주하며 내가 올린 코드(PR)를 실시간으로 리뷰해주는 **AI 코드 리뷰어**입니다.

복잡한 설치나 코딩(GitHub Actions) 없이, 앱 설치 한 번으로 동작이 가능합니다.

Benefits

즉각적인 피드백: "내 코드가 맞나?" 고민하며 팀장을 기다릴 필요가 없습니다. 1분 안에 보고서가 올라옵니다.

자동 요약 기능: 내가 수정 내역을 일일이 설명하지 않아도 AI가 "이 부분은 무엇을 위한 수정이군요!"라고 대신 보고해줍니다.

한국어 지원: 언어 장벽 없이 한글로 질문하고 답변받으며 과외받듯 배울 수 있습니다.

작동 방법

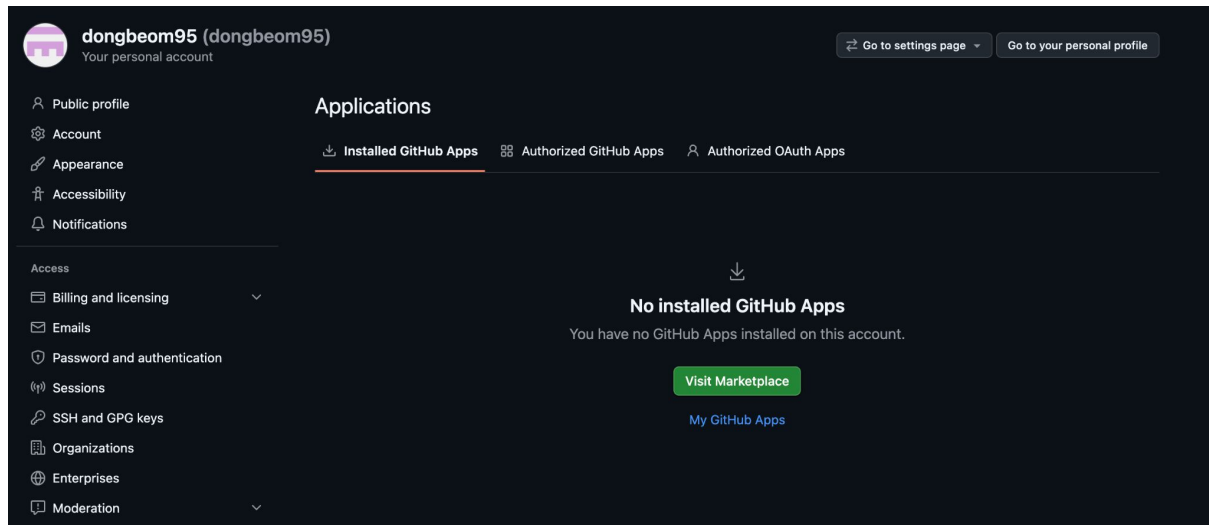
Code 작성: 나만의 작업 공간(Branch)에서 내용을 수정합니다.

PR 생성: "제 코드 좀 봐주세요!"라고 요청(Pull Request)을 날립니다.

AI 리뷰: CodeRabbit 토끼 아이콘과 함께 AI의 상세 리뷰가 댓글로 달립니다.



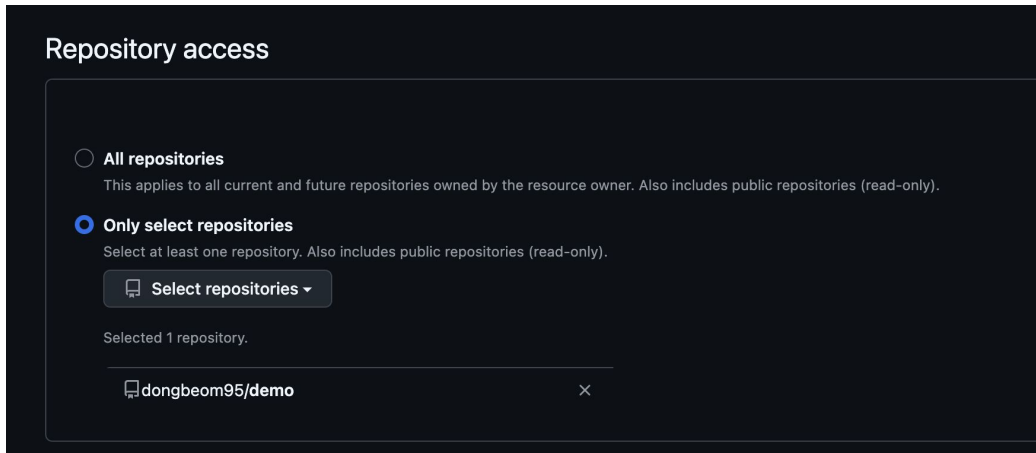
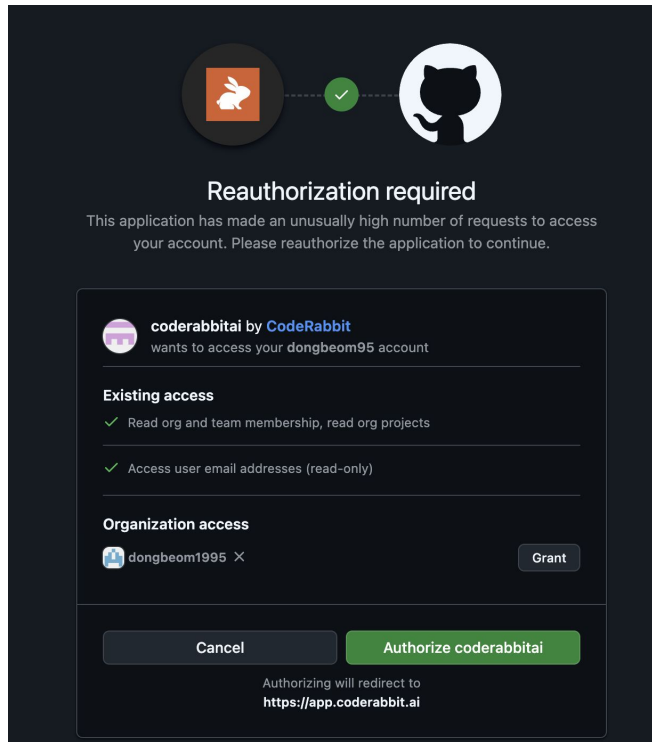
Code Rabbit 설치



Settings - Applications - installed GitHub Apps

Visit Marketplace 클릭 후 coderabbit install

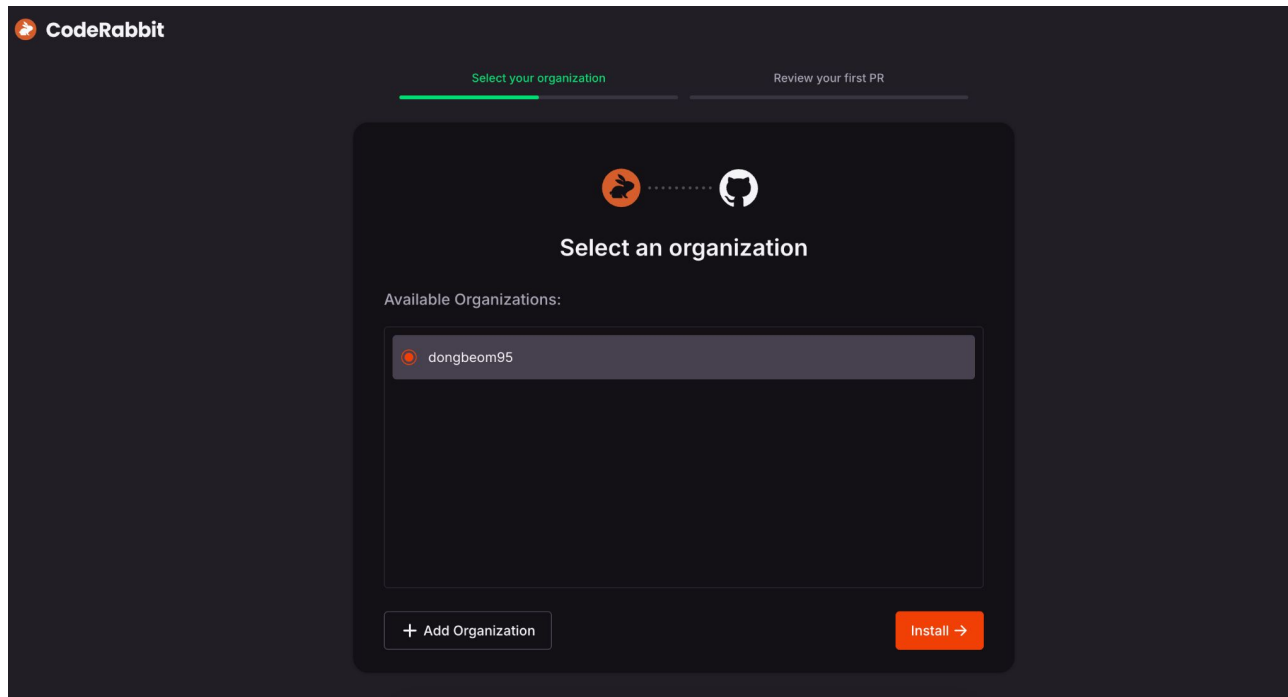
Code Rabbit 설치



설치 후 Github 권한 부여

coderabbit을 사용할 레포지토리 선택

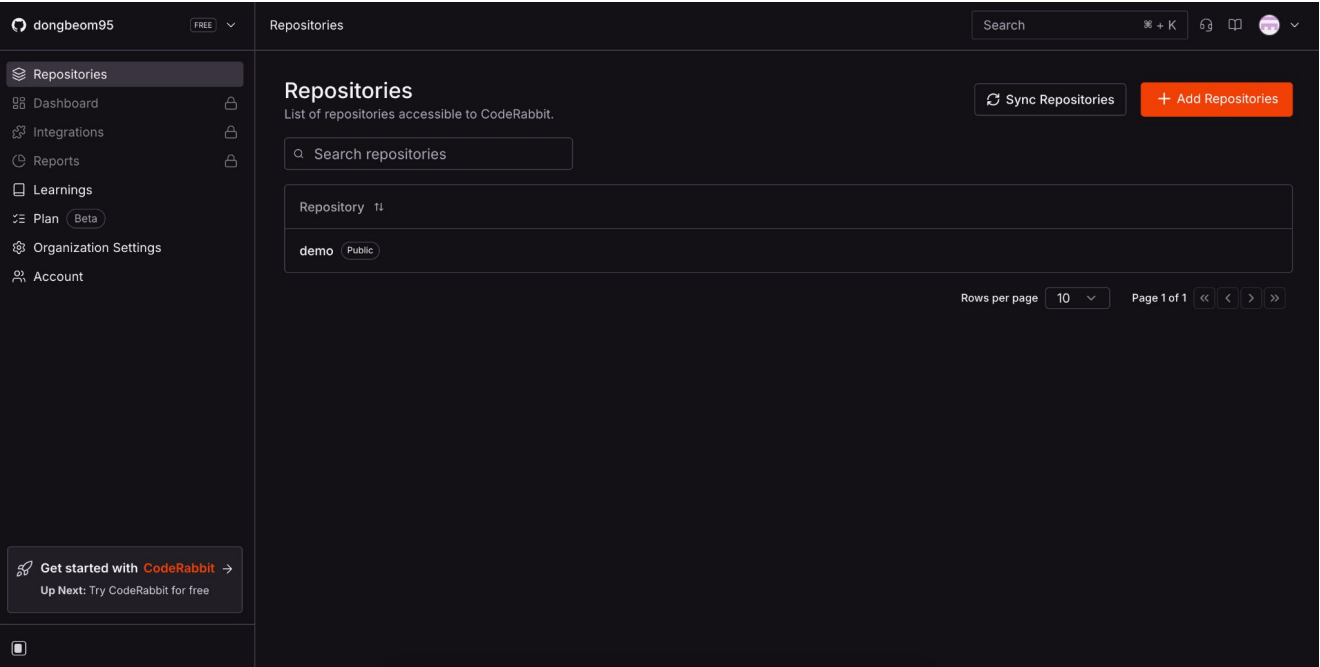
Code Rabbit 설치



Code Rabbit Dashboard

CodeRabbit 에서 권한 부여

Code Rabbit 설치확인



Code Rabbit Dashboard

권한 부여 레포지토리 확인

Code Rabbit 동작확인

 **coderrabbitai** bot commented 2 minutes ago • edited

▼ Walkthrough

Walkthrough

A Korean-language section titled "CodeRabbit AI 피드백 테스트" has been added to README.md, describing a collaboration workflow test for verifying AI summarization capabilities for a non-expert lesson.

Changes

Cohort / File(s)	Summary
Documentation Update README.md	Added new section with Korean text explaining a CodeRabbit AI feedback test for collaboration workflow verification.

Estimated code review effort

👍 1 (Trivial) | ⌚ ~2 minutes

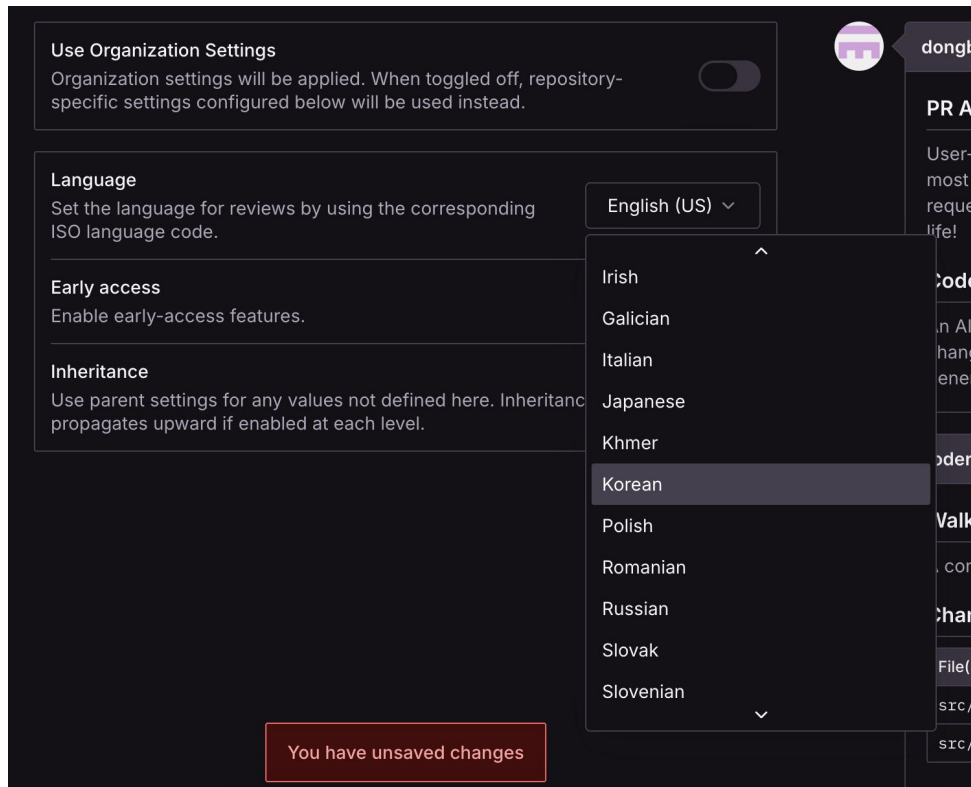
Poem

📖+ A Korean note hops into view,
Testing how AI reviews what's new,
Simple words on a README line,
Documentation looks divine! 🐇

Code Rabbit Review

PR을 통한 CodeReview 동작 확인

Code Rabbit 설정변경

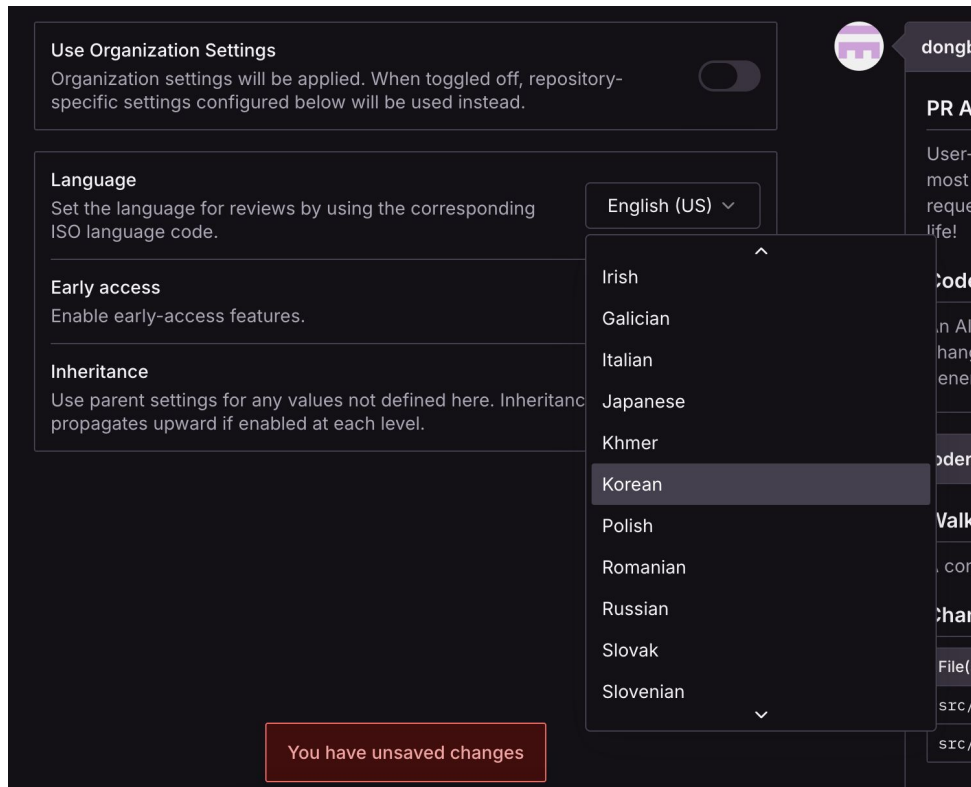


Code Rabbit Setting

CodeRabbit Dashboard -> Repository 선택 후 설정

한국어 변경과 PR리뷰 스타일 조정

Code Rabbit 설정변경

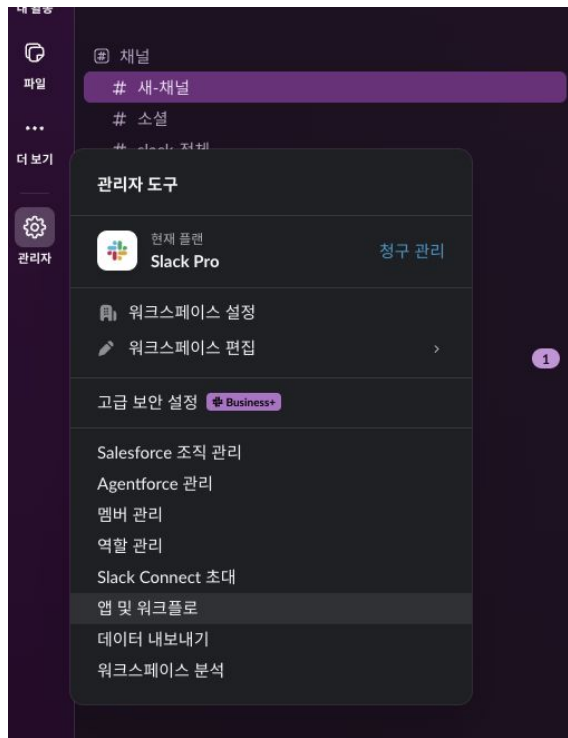


Code Rabbit Setting

CodeRabbit Dashboard -> Repository 선택 후 설정

한국어 변경과 PR리뷰 스타일 조정

Slack Integration



Slack Github 설정

관리자 -> 앱 및 워크플로 선택

Slack Integration

 [찾아보기](#) [카테고리](#) [컬렉션](#) [관리](#) [구축](#)

 demo

PRO AI 앱을 포함한 무제한 앱 추가 기능은 유료 기능이며, 5월 14일까지는 무료 평가판으로 사용할 수 있습니다. 자세히 알아보기

< 앱 알아보기



Slack에 추가

자세히 알아보기

지원되는 언어
영어

요금
무료 및 유료 플랜 제공

자세히 알아보기 및 지원

- 앱 지원 받기
- 개발자 웹 사이트로 이동
- support@github.com
- 개인정보 처리방침

카테고리

개발자 도구

커뮤니케이션

인기

생산성

AI 앱 및 어시스턴트

GitHub

설명 기능 권한 보안 및 규정 준수



Ask @GitHub
to fix bugs, add test coverage & refactor

Upgrade required
If you're running the version of the app released before Oct 28, 2025, upgrade at <https://slack.github.com/>.
Bring your code to the conversations you care about with the GitHub app for Slack. With two of your most important workspaces connected, you'll stay updated on what's happening and can easily take action, without leaving Slack.

Work with Copilot in Slack
Mention @GitHub in any Slack thread to turn conversations into action. Copilot uses your conversation as context to create pull requests, open issues, and answer questions, all without leaving Slack.

- Generate pull requests from Slack discussions
- Create issues using natural language, with titles, labels, assignees, and milestones
- Break down work into parent and sub-issues from a single message

Slack Marketplace

GitHub 검색 후 Slack 추가

Slack Integration

Github Slack 권한 부여



"GitHub" 앱의 Slack 액세스 허용

✔ Slack에서 승인한 앱입니다

앱은 Slack Marketplace에 등록되기 전에 품질 검토를 거칩니다. 앱 검토에 대해 자세히 알아보세요.

Workspace



Review app permissions

[권한 관리](#)

"GitHub" 앱이 볼 수 있는 정보

- 🔍 채널과 대화에 관한 콘텐츠와 정보 ▶
- 📁 내 워크스페이스에 관한 콘텐츠와 정보 ▶

"GitHub" 앱이 취할 수 있는 작업

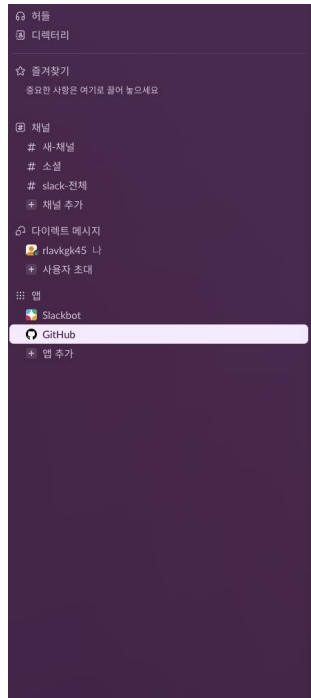
- 🔍 채널과 대화에서 작업 수행 ▶
- 📁 내 워크스페이스에서 작업 수행 ▶

Slack은 귀하에게 부여된 권한을 GitHub 앱과 공유하게 됩니다. Slack의 **개인 정보 처리방침**을 확인하세요. Slack이 귀하의 ID를 공유하도록 허용하는 데에 동의하는 경우, GitHub의 **개인 정보 보호 계약** 및 **서비스 약관**에도 함께 동의하게 됩니다. 설치하기 전에 GitHub의 약관을 검토하세요.

취소

허용

Slack Integration



 GitHub

@GitHub 님과의 다이렉트 메시지 내역의 맨 첫 부분입니다.

📍 GitHub 님은 어떻게 작업하나요?

오늘 ▾

 GitHub 오후 10:43

Hi there 🙌 here are some ideas of what you can do:

Create issues and pull requests with GitHub Copilot

GitHub Copilot can create GitHub issues and pull requests directly from your Slack thread.

To get started, mention the app in any message:

`@GitHub [your prompt]`

Examples:

- `@GitHub add error handling to the login function in owner/repo`
- `@GitHub raise an issue to fix the login bug in owner/repo`

To iterate on Copilot's work, mention `@GitHub` again in the same thread.

Examples:

- `@GitHub add tests`
- `@GitHub refactor the code to use async/await`

Set a default repository for Copilot in this channel by running `@GitHub settings`. This repository will be used when no repository is specified in your prompt.

Subscribe to repositories

Subscribe	<code>/github subscribe owner/repo</code>
Unsubscribe	<code>/github unsubscribe owner/repo</code>
Show subscriptions	<code>/github subscribe list</code>

Create and edit issues

Create a new issue	<code>/github open owner/repo</code>
Close an issue as completed	<code>/github close [issue link]</code>
Reopen an issue	<code>/github reopen [issue link]</code>

Accounts

Connect your GitHub account	<code>/github signin</code>
Disconnect your GitHub account	<code>/github signout</code>

Slack 연동 확인

Slack Github 앱 추가 확인

Slack Integration

Accounts

Connect your GitHub account

Disconnect your GitHub account

/github signin

/github signout

Learn more

Contact support

 **rlavkgk45** 오후 10:46

/github signin

 나에게만 표시

 **GitHub** 오후 10:46

Connect your GitHub account

Connect GitHub account

 What to expect

You'll be sent to github.com to authorize GitHub (Slack application) to access your GitHub account. After authorization, copy the 6-digit verification code you receive and paste it below.

Enter verification code

B I U     

#새-채널에 메시지 보내기

+

Aa









GitHub 명령어 사용

/github signin 을 통한 권한 부여 진행

16
7

Slack Integration



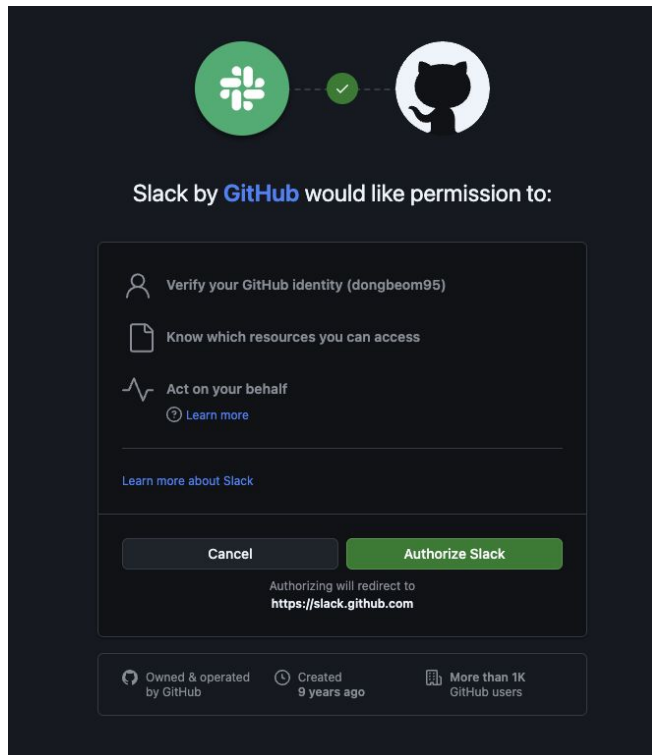
Finish connecting your GitHub account to your Slack account

on the `demo-vsm2064.slack.com` workspace

This will connect your accounts so that you can use Slack to view rich previews for private GitHub links, open and close issues, and other features that depend on your access to GitHub. [Learn more](#)

Connect GitHub account

GitHub Inc. ©2026
[Terms](#) / [Privacy](#)



GitHub 연동 권한 부여

Connect GitHub account 클릭

Slack Integration

나에게만 표시

 **GitHub** 오후 10:46

Connect your GitHub account

Connect GitHub account

 What to expect

You'll be sent to github.com to authorize **GitHub (Slack application)** to access your GitHub account. After authorization, copy the 6-digit verification code you receive and paste it below. 📌

Enter verification code

나에게만 표시

 **GitHub** 오후 10:47

✓ Success! Your GitHub account **dongbeam95** is now connected.

You're all set to use `@GitHub`.

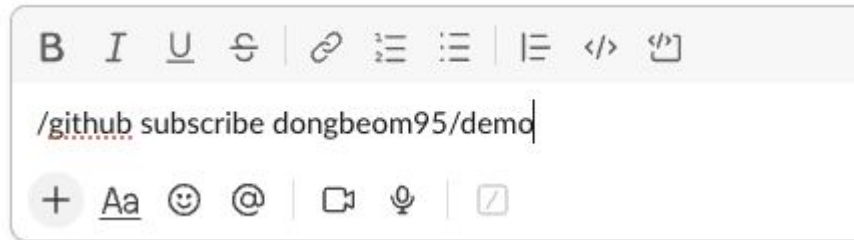
Type `/github help` to explore what you can do.

B I U ↺ ↻ ↶ ↷ ↸ ↹ ↺ ↻ ↶ ↷ ↸ ↹

#새-채널에 메시지 보내기

GitHub 연동 완료

Slack Integration



 **rlavkgk45** 오후 10:48
/github subscribe dongbeom95/demo

🔔 나에게만 표시
 **GitHub** 오후 10:48
Either the app isn't installed on your repository or the repository does not exist. Install it to proceed.

[Install GitHub App](#)

🔔 나에게만 표시
✅ Success! The app is installed.
✅ Subscribed to [dongbeom95/demo](#). This channel will receive notifications for [issues](#), [pulls](#), [commits](#), [releases](#), [deployments](#)

[Learn More](#)



레포지토리 구독

명령어를 통한 레포지토리 구독 실행

Slack Integration

 **rlavkgk45** 오후 10:54
/github subscribe dongbeom95/demo reviews, comments

 **GitHub** 오후 10:54
This channel will receive notifications from [dongbeom95/demo](#) for:
[issues](#), [pulls](#), [commits](#), [releases](#), [deployments](#), [reviews](#), [comments](#)
[Learn More](#)

구독 항목 추가

reviews, comments 구독 항목 추가

B

I

U

🔒

🔗

☰

☰

☰

</>

📄

|

+

Aa

😊

@

📺

🎤

📌

Slack Integration



GitHub  오후 10:54

Pull request opened by [dongbeom95](#) on 4월 15일

#5 docs: AI 리뷰를 유도하기 위한 README 수정

Summary by CodeRabbit

- Documentation

- Added a new section with information about the project's collaboration workflow testing and verification process.

Comments

4

 [dongbeom95/demo](#) | 오늘 오후 10:07 | 봇이 추가한 GitHub

Comment



1개의 댓글 오늘 오후 10:54에

comments 알림 확인

PR comment 추가 후 알림 수신 확인

B I U      

>

+ Aa  @   